Sorting Out Secure Analytics with Thousands of Users

October 31, 2025

Sam Buxbaum

Roadmap

1. Deployment of MPC for Political Polling

2. Oblivious Sorting

3. ORQ

Privacy-Preserving Machine Learning on Web Browsing for Public Opinion

To appear at CSCML 25!

Paper

GitHub

Privacy-Preserving Machine Learning on Web Browsing for Public Opinion

Sam Buxbaum[1], Lucas M. Tassis[1], Lucas Boschelli[2], Giovanni Comarela[3], Mayank Varia[1], Mark Crovella[1], and Dino P. Christenson[4]

[1] Boston University {sambux,ltassis,varia,crovella}bu.edu
[2] Maritz boschellil@wustl.edu
[3] Universidade Federal do Espirito Santo gc@inf.ufes.br
[4] Washington University dinopc@wustl.edu

Abstract. We present a real-world deployment of secure multiparty computation to predict political preference from private web browsing data. To estimate aggregate preferences for the 2024 U.S. presidential election candidates, we collect and analyze secret-shared data from nearly 8000 users from August 2024 through February 2025, with over 2000

Plan

Plan

1. Background on Political Polling

Plan

1. Background on Political Polling

2. Our Approach and Contributions

Plan

1. Background on Political Polling
2. Our Approach and Contributions
3. Our System

Plan

1. Background on Political Polling
2. Our Approach and Contributions
3. Our System
4. The Learning Problem

Plan

1. Background on Political Polling
2. Our Approach and Contributions
3. Our System
4. The Learning Problem
5. Learning Under MPC

Plan

1. Background on Political Polling
2. Our Approach and Contributions
3. Our System
4. The Learning Problem
5. Learning Under MPC
6. Paying Our Users

Plan

1. Background on Political Polling
2. Our Approach and Contributions
3. Our System
4. The Learning Problem
5. Learning Under MPC
6. Paying Our Users
7. Next Steps and Future Work

Traditional Political Polling

Traditional Political Polling

- Data collection takes time

Traditional Political Polling

- Data collection takes time
- Data collection is human-intensive

Traditional Political Polling

- Data collection takes time
- Data collection is human-intensive
- Poor geographic and temporal coverage

Traditional Political Polling

- Data collection takes time
- Data collection is human-intensive
- Poor geographic and temporal coverage

West Virginia 2024 Presidential Election Polls

Harris vs. Trump

Source	Date	Sample	Harris	Trump	Other
Research America	8/30/2024	400 LV ±4.9%	34%	61%	5%

Traditional Political Polling

- Data collection takes time
- Data collection is human-intensive
- Poor geographic and temporal coverage

West Virginia 2024 Presidential Election Polls

Harris vs. Trump

Source	Date	Sample	Harris	Trump	Other
Research America	8/30/2024	400 LV ±4.9%	34%	61%	5%

Michigan 2024 Presidential Election Polls

Instantly compare a poll to prior one by same pollster

Harris vs. Trump

Source	Date	Sample	Harris	Trump	Other
Average of 23 Polls†			**48.6%**	**46.8%**	-
FAU / Mainstreet	11/04/2024	713 LV	49%	47%	4%
Emerson College	11/04/2024	790 LV ±3.4%	50%	48%	2%
Research Co.	11/04/2024	450 LV ±4.6%	49%	47%	4%
InsiderAdvantage	11/03/2024	800 LV ±3.7%	47%	47%	6%
Trafalgar Group	11/03/2024	1,079 LV ±2.9%	47%	48%	5%
MIRS / Mich. News Source	11/03/2024	585 LV ±4%	50%	48%	2%
NY Times / Siena College	11/03/2024	998 LV ±3.7%	47%	47%	6%
Morning Consult	11/03/2024	1,108 LV ±3%	49%	48%	3%
AtlasIntel	11/02/2024	1,198 LV ±3%	48%	50%	2%
Redfield & Wilton	11/01/2024	1,731 LV ±2.2%	47%	47%	6%
The Times (UK) / YouGov	11/01/2024	942 LV ±3.9%	48%	45%	7%
EPIC-MRA	11/01/2024	600 LV ±4%	48%	45%	7%
Marist Poll	11/01/2024	1,214 LV ±3.5%	51%	48%	1%
AtlasIntel	10/31/2024	1,136 LV ±3%	49%	49%	2%
Echelon Insights	10/31/2024	600 LV ±4.4%	48%	48%	4%
MIRS / Mich. News Source	10/31/2024	1,117 LV ±2.5%	47%	49%	4%
UMass Lowell	10/31/2024	600 LV ±4.5%	49%	45%	6%
Washington Post	10/31/2024	1,003 LV ±3.7%	47%	46%	7%
Fox News	10/30/2024	988 LV ±3%	49%	49%	2%
CNN	10/30/2024	726 LV ±4.7%	48%	43%	9%
Suffolk University	10/30/2024	500 LV ±4.4%	47%	47%	6%

Web Browsing for Political Polling

Web Browsing for Political Polling

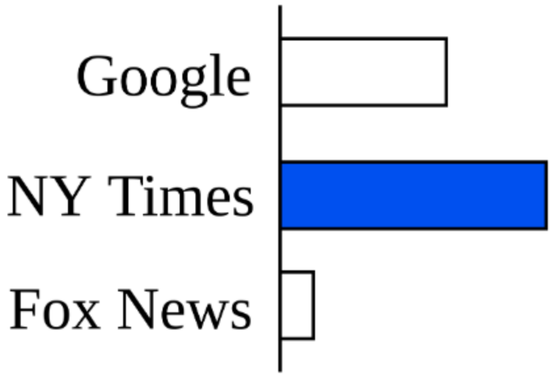
- Can website visits predict political leanings?

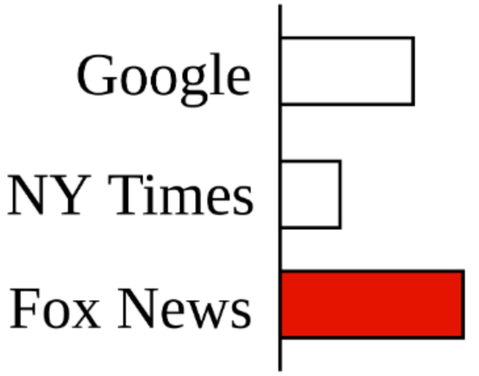
Web Browsing for Political Polling

- Can website visits predict political leanings?
- Example - news websites

Web Browsing for Political Polling

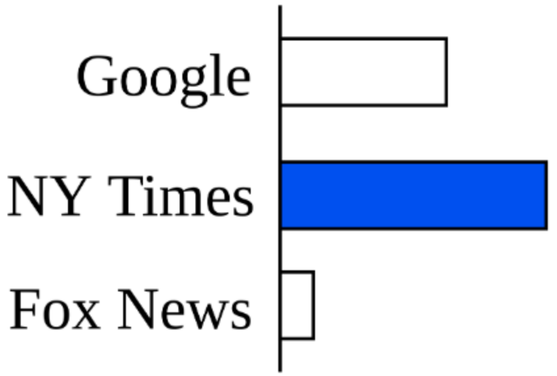
- Can website visits predict political leanings?
- Example - news websites

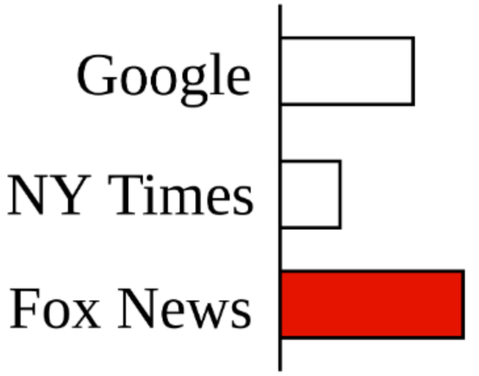




Web Browsing for Political Polling

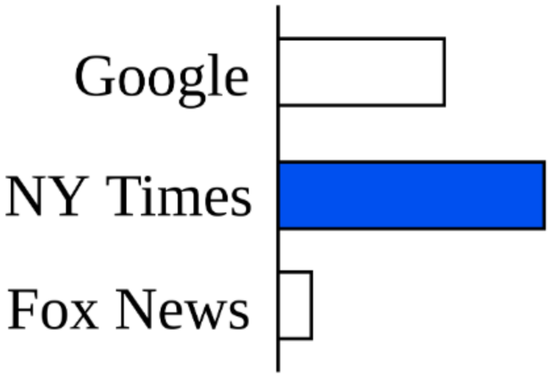
- Can website visits predict political leanings?
- Example - news websites
- More data

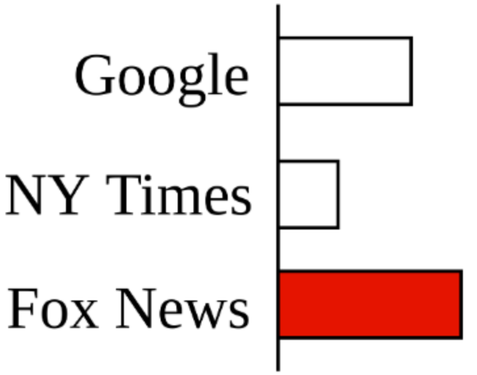




Web Browsing for Political Polling

- Can website visits predict political leanings?
- Example - news websites
- More data
- Fully automated





Prior Work

Prior Work

- Web browsing behavior can predict voting results

Prior Work

- Web browsing behavior can predict voting results
- Quantifying the 'Comey letter' (Comarela et al.)

Prior Work

- Web browsing behavior can predict voting results
- Quantifying the 'Comey letter' (Comarela et al.)

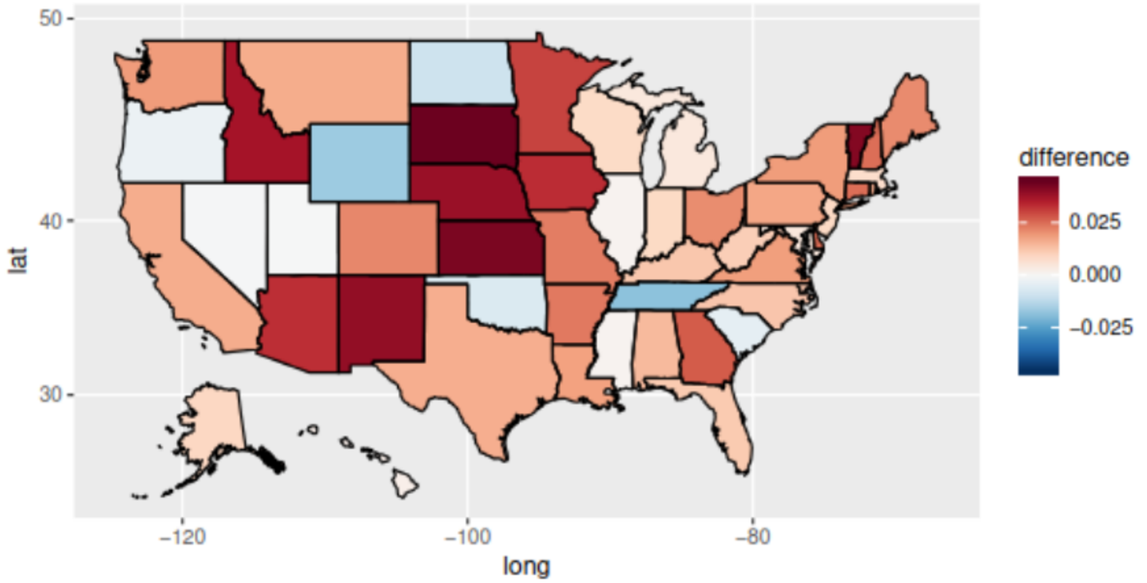


Figure 8: Impact of the 'Comey letter' at the state level.

Prior Work

- Web browsing behavior can predict voting results
- Quantifying the 'Comey letter' (Comarela et al.)

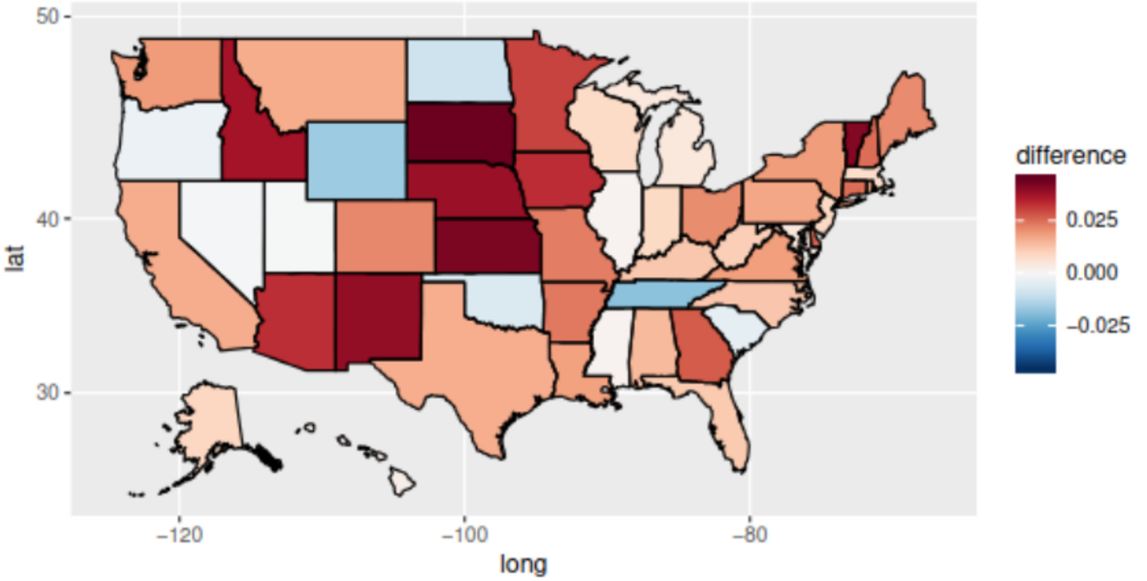
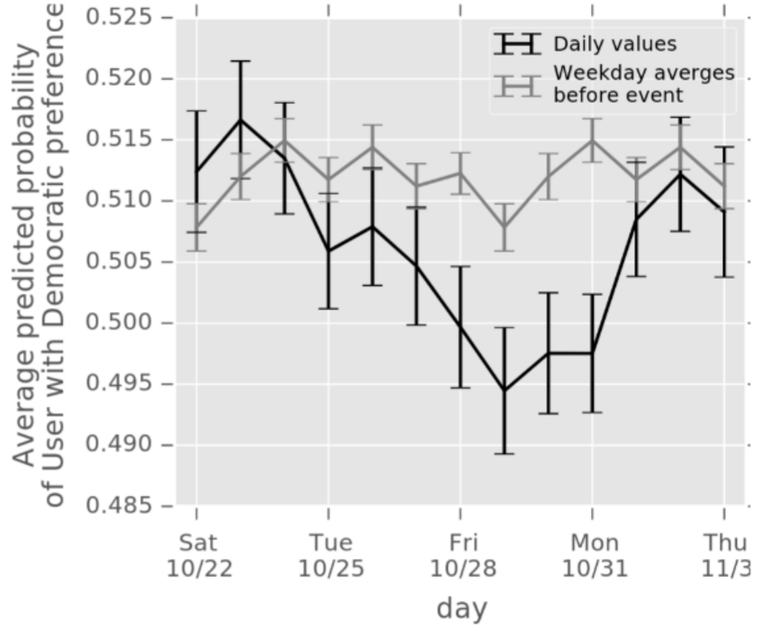
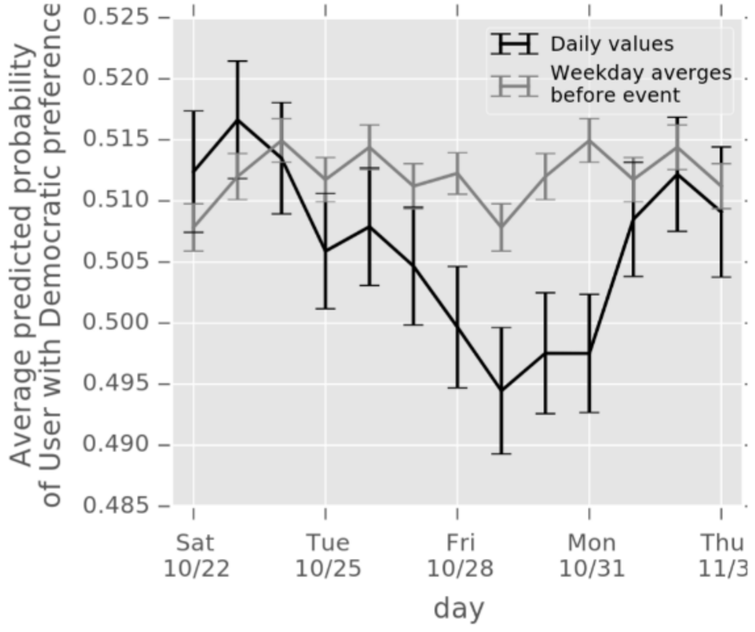


Figure 8: Impact of the 'Comey letter' at the state level.



Prior Work

- Web browsing behavior can predict voting results
- Quantifying the 'Comey letter' (Comarela et al.)
- Social media referrals are the best signal



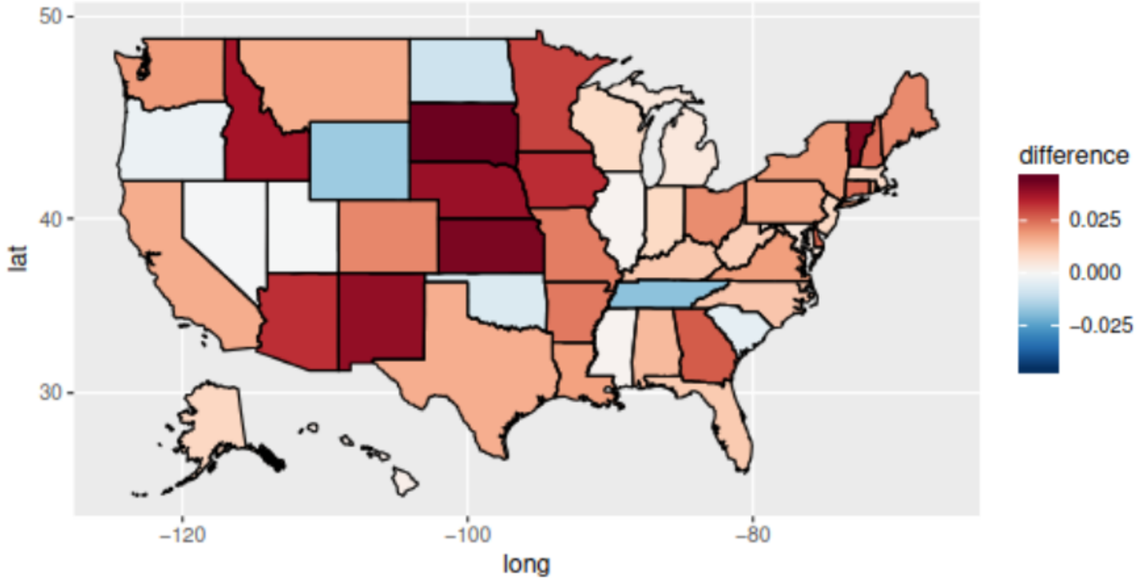
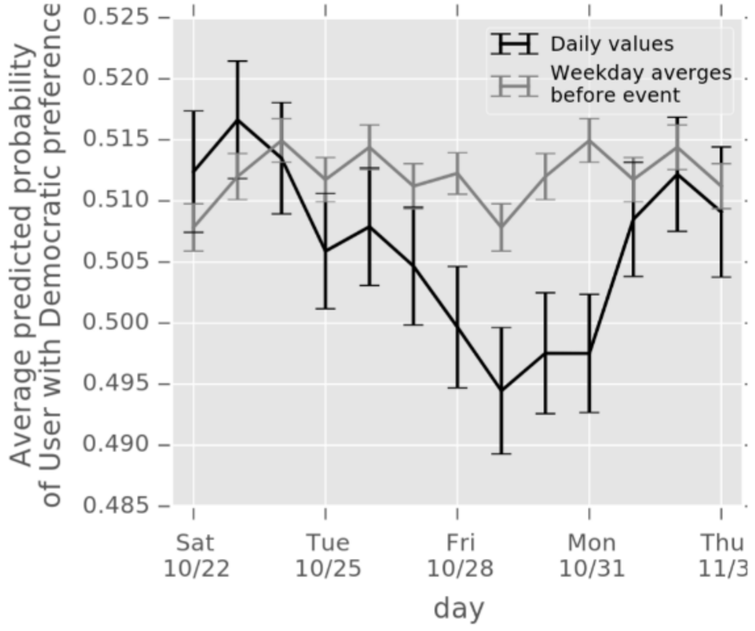


Figure 8: Impact of the 'Comey letter' at the state level.

Prior Work

- Web browsing behavior can predict voting results
- Quantifying the 'Comey letter' (Comarela et al.)
- Social media referrals are the best signal
- Used large plaintext dataset



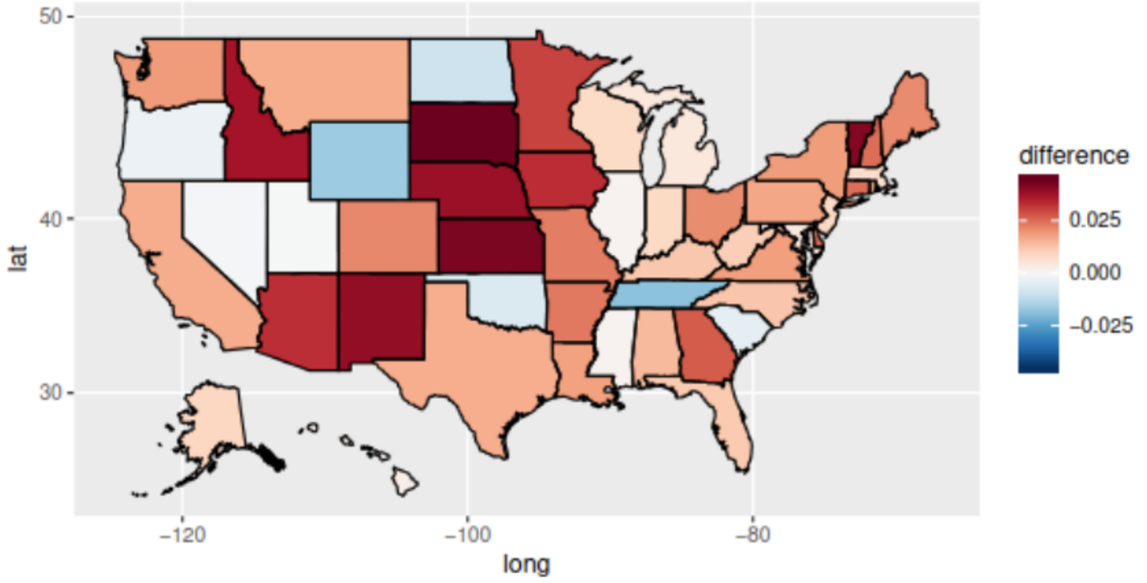


Figure 8: Impact of the 'Comey letter' at the state level.

Two Approaches to Political Polling

Two Approaches to Political Polling

Traditional Polling

- Slow
- Expensive
- Coarse-grained insights

Two Approaches to Political Polling

Traditional Polling

- Slow
- Expensive
- Coarse-grained insights

VS

Web Behavior Analysis

- Immediate
- Cheap
- Fine-grained insights

Two Approaches to Political Polling

Traditional Polling

- Slow
- Expensive
- Coarse-grained insights

VS

Web Behavior Analysis

- Immediate
- Cheap
- Fine-grained insights

What about privacy?

Our Contributions

Our Contributions

- We built a system for securely predicting political preferences from web browsing data

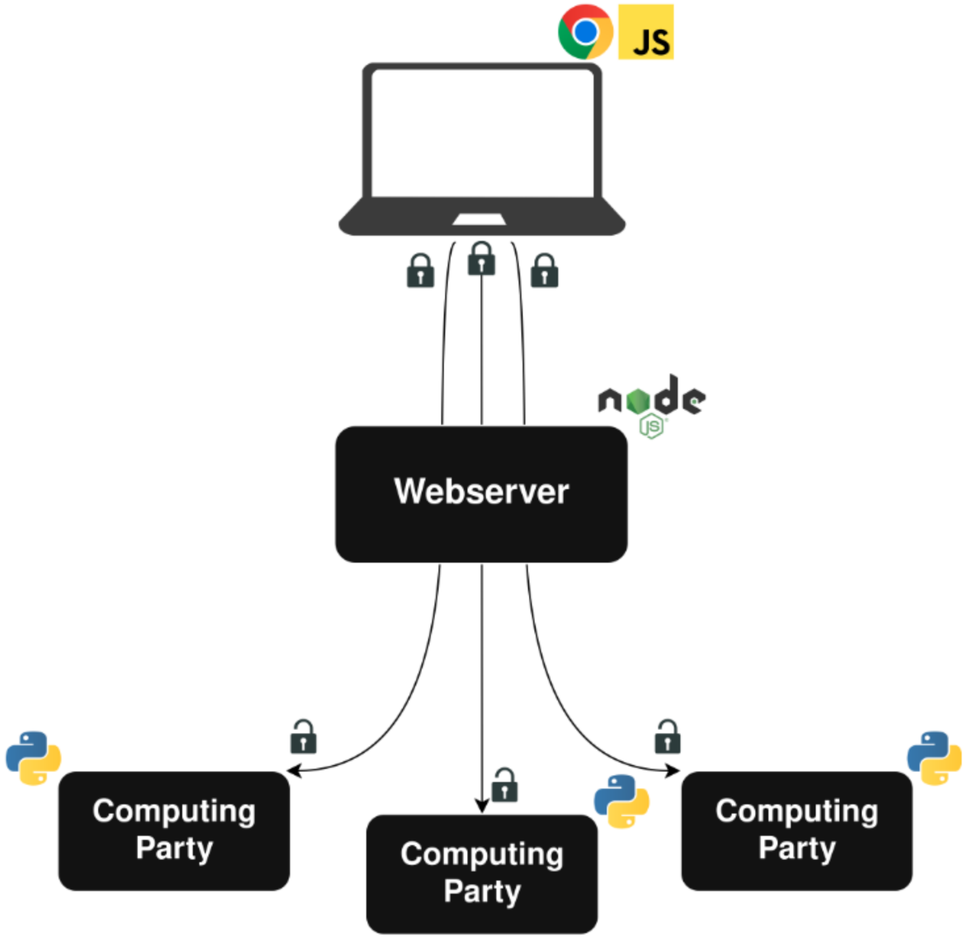
Our Contributions

- We built a system for securely predicting political preferences from web browsing data
- We collected and analyzed data from almost 8000 unique users

Our Contributions

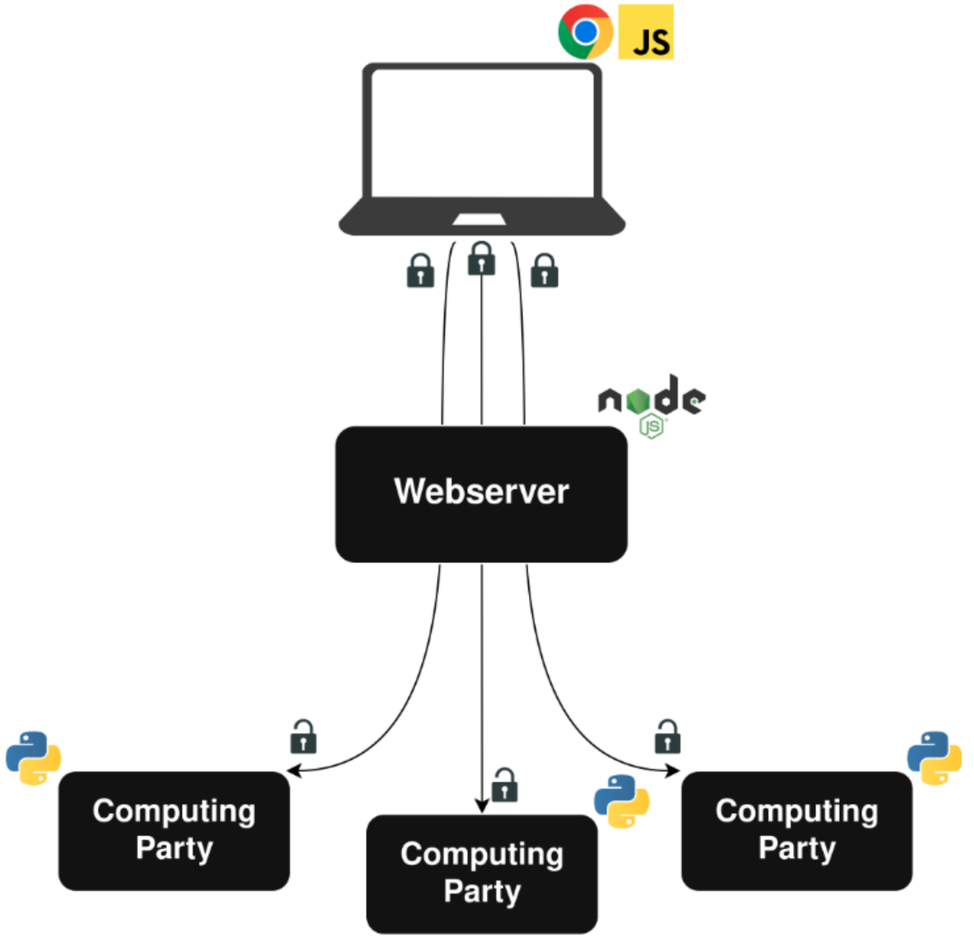
- We built a system for securely predicting political preferences from web browsing data
- We collected and analyzed data from almost 8000 unique users
- All analysis took place under MPC

System Design



System Design

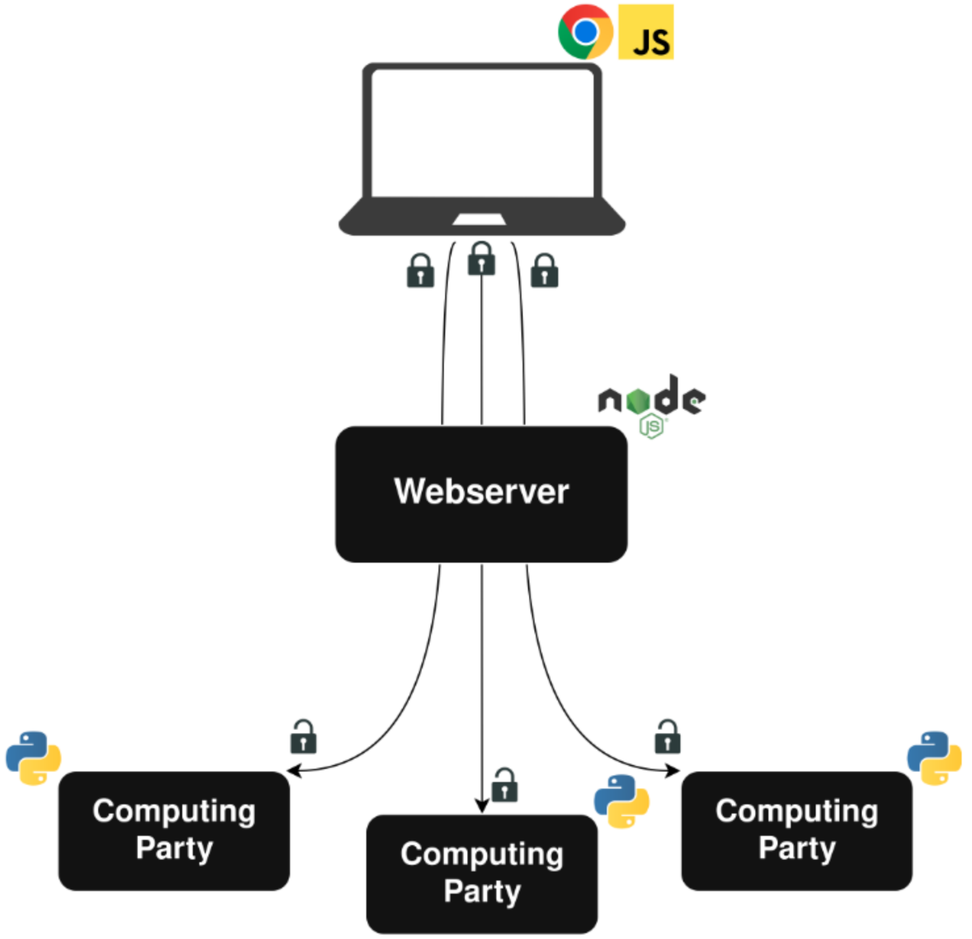
Client Plugin



System Design

Client Plugin

Webserver

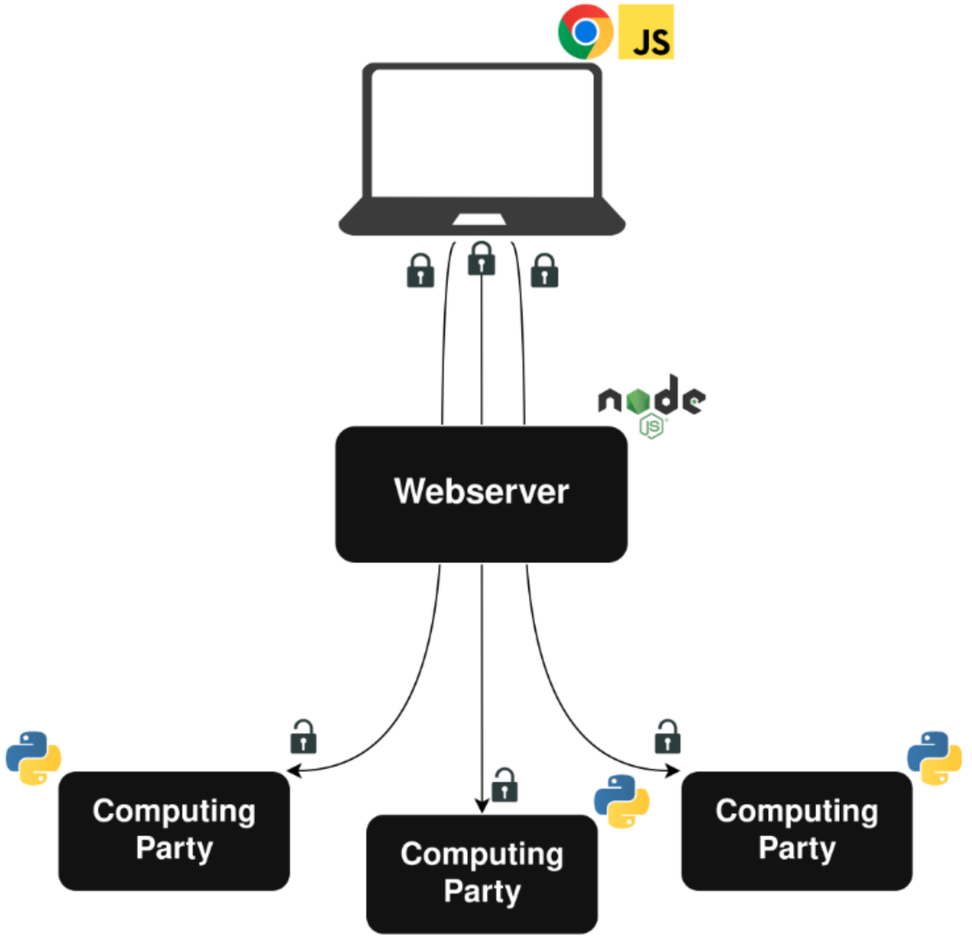


System Design

Client Plugin

Webserver

MPC Backend

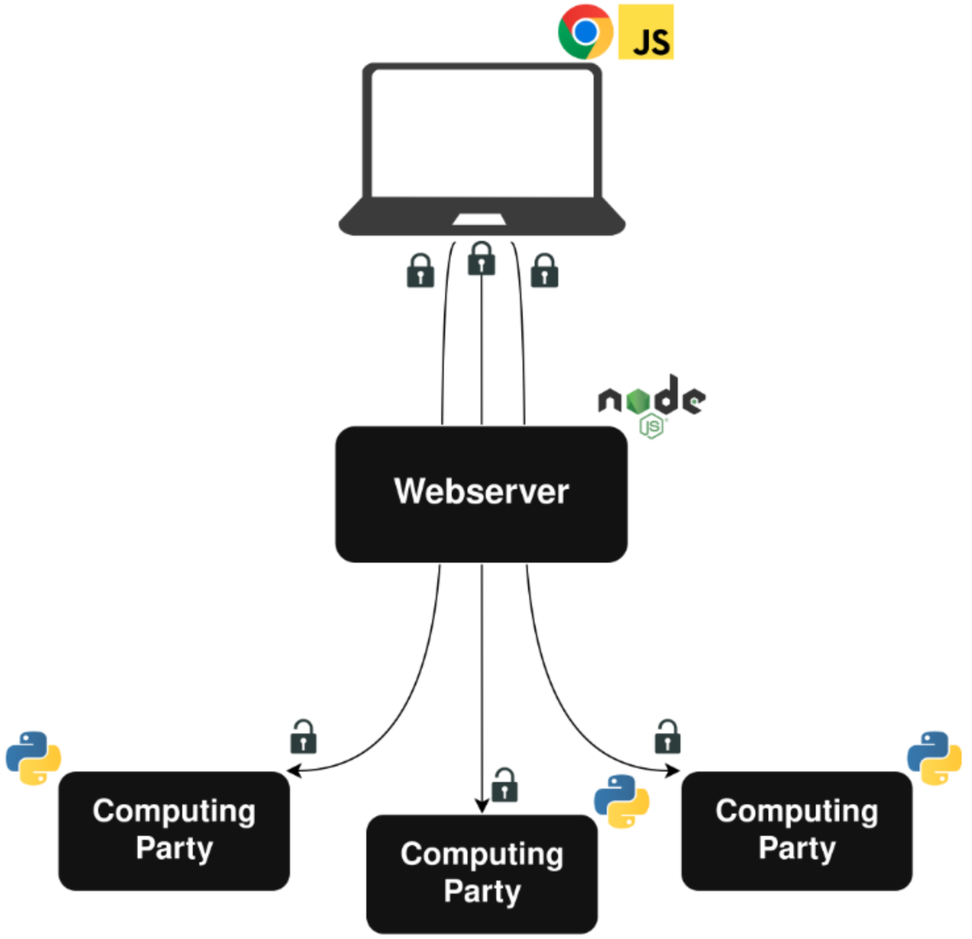


System Design

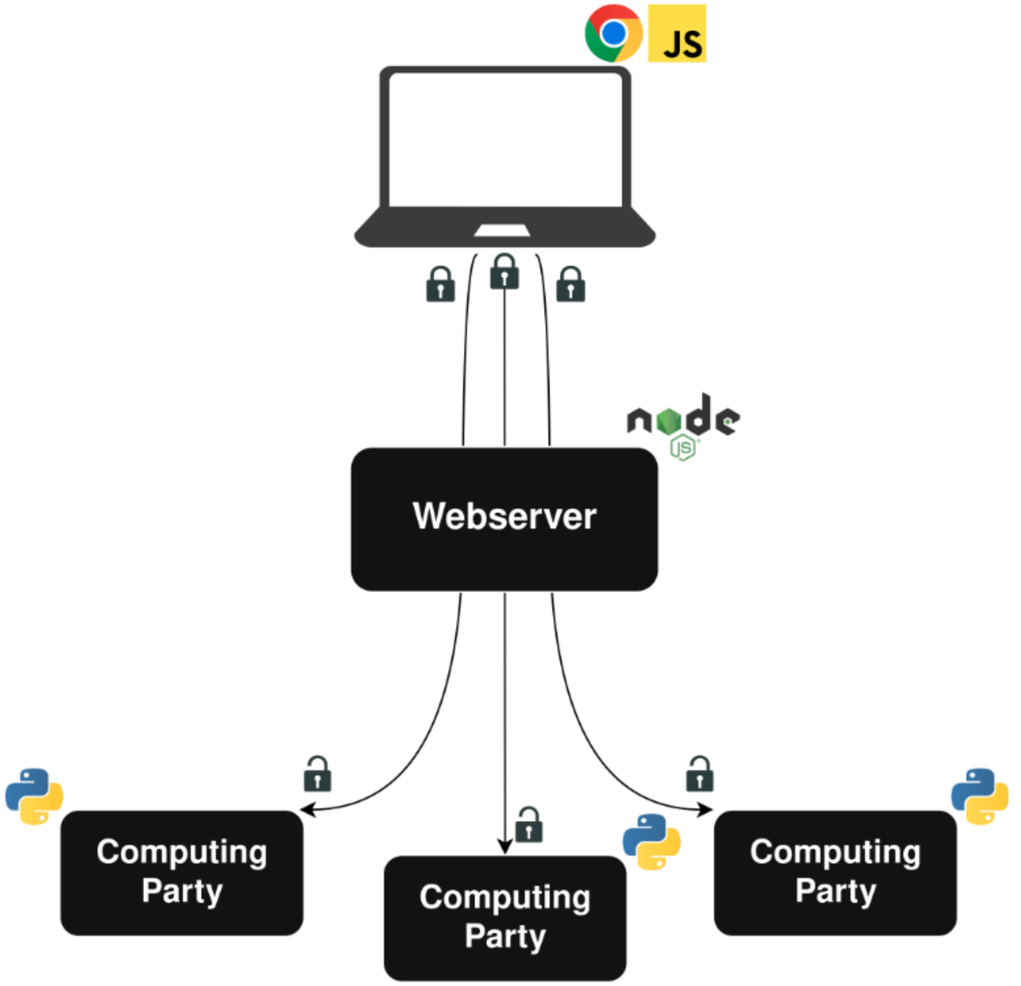
Client Plugin

Webserver

MPC Backend

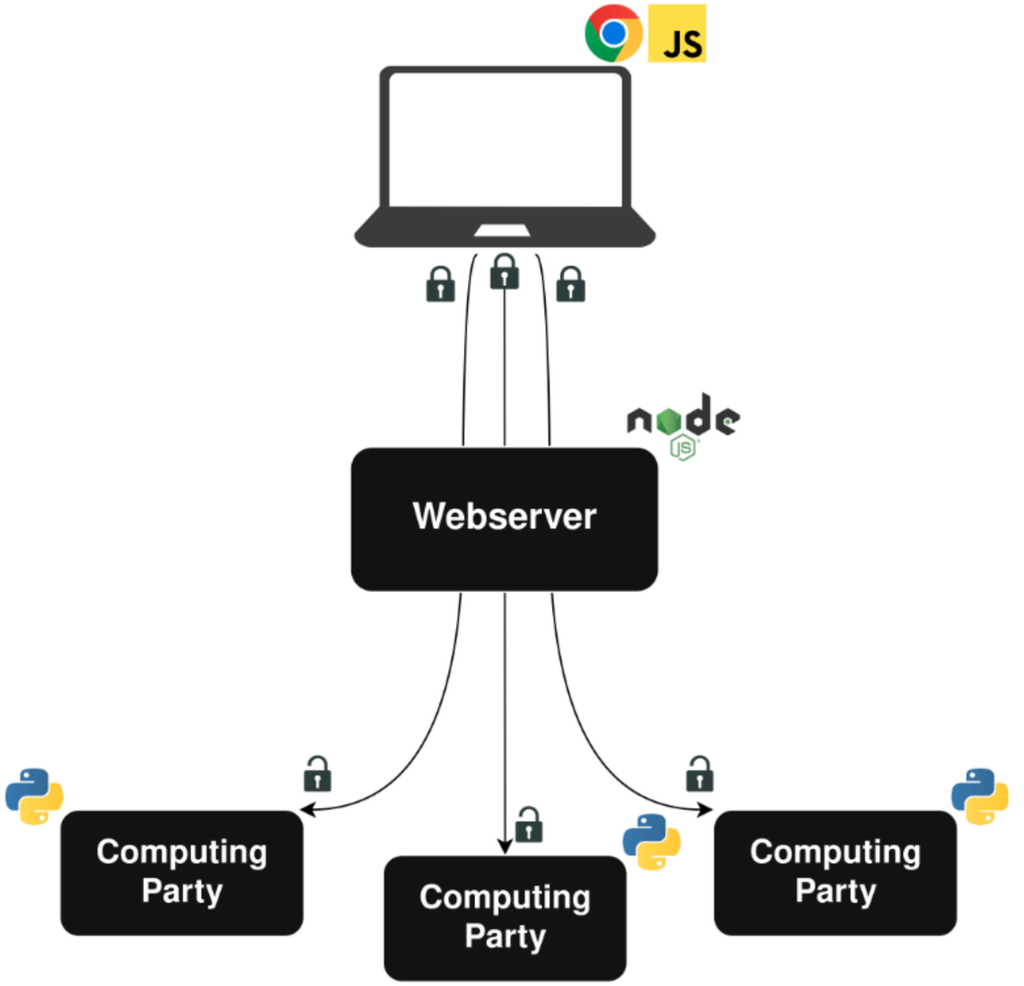


Client Plugin



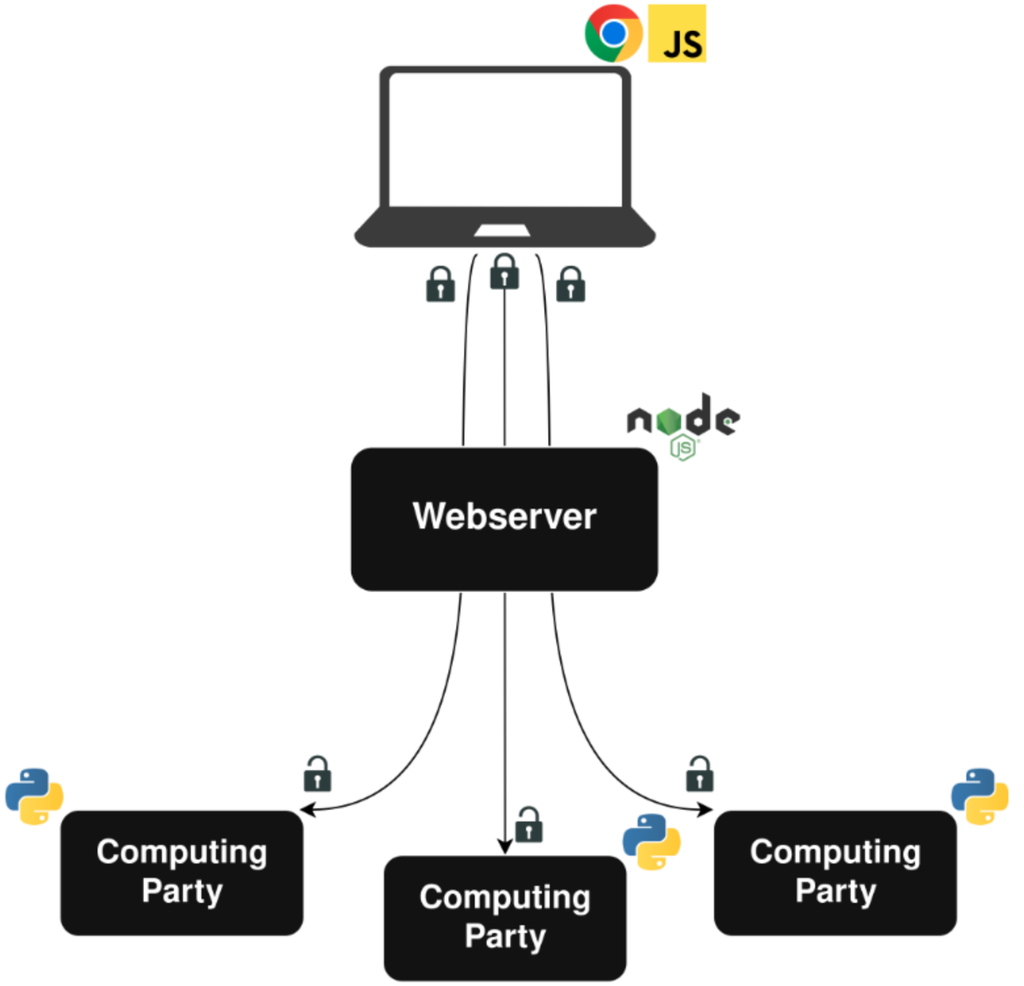
Client Plugin

- Custom-built Chrome plugin to monitor browsing



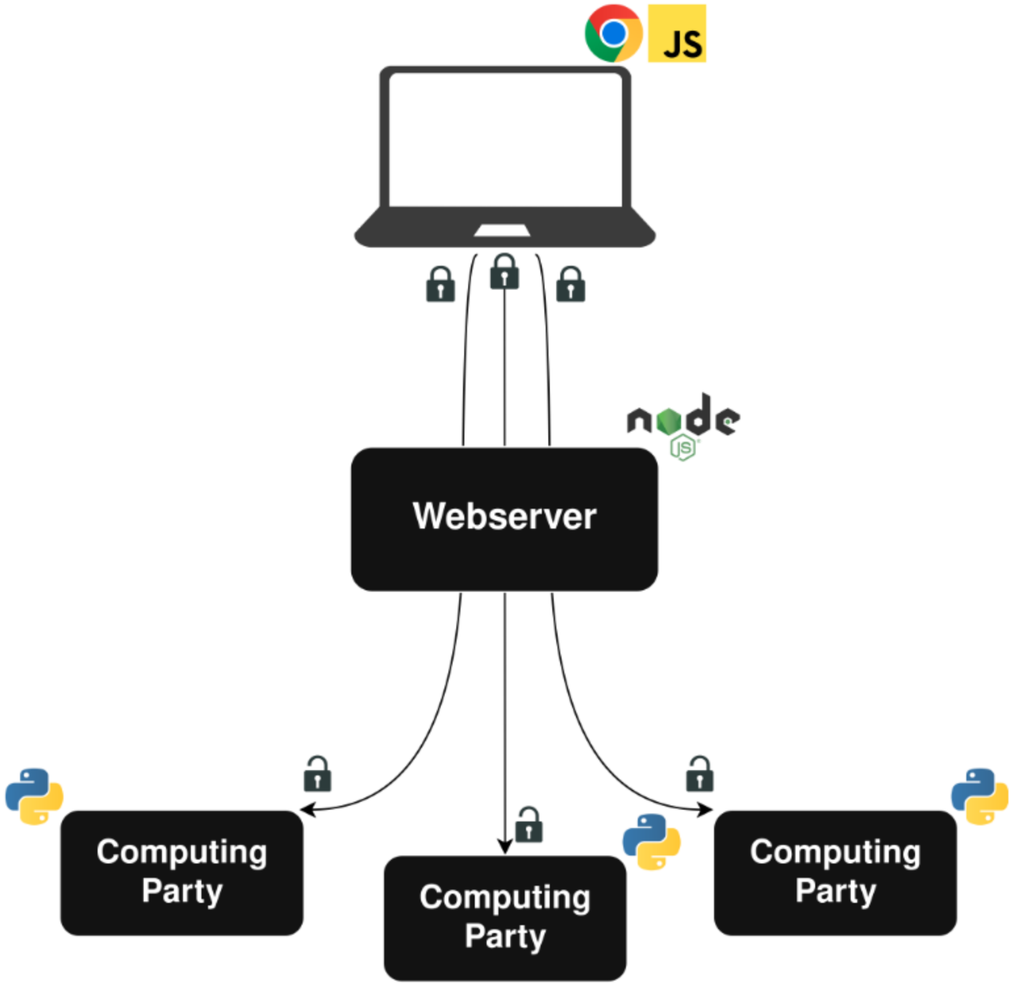
Client Plugin

- Custom-built Chrome plugin to monitor browsing
- Daily data uploads of secret-shared histograms

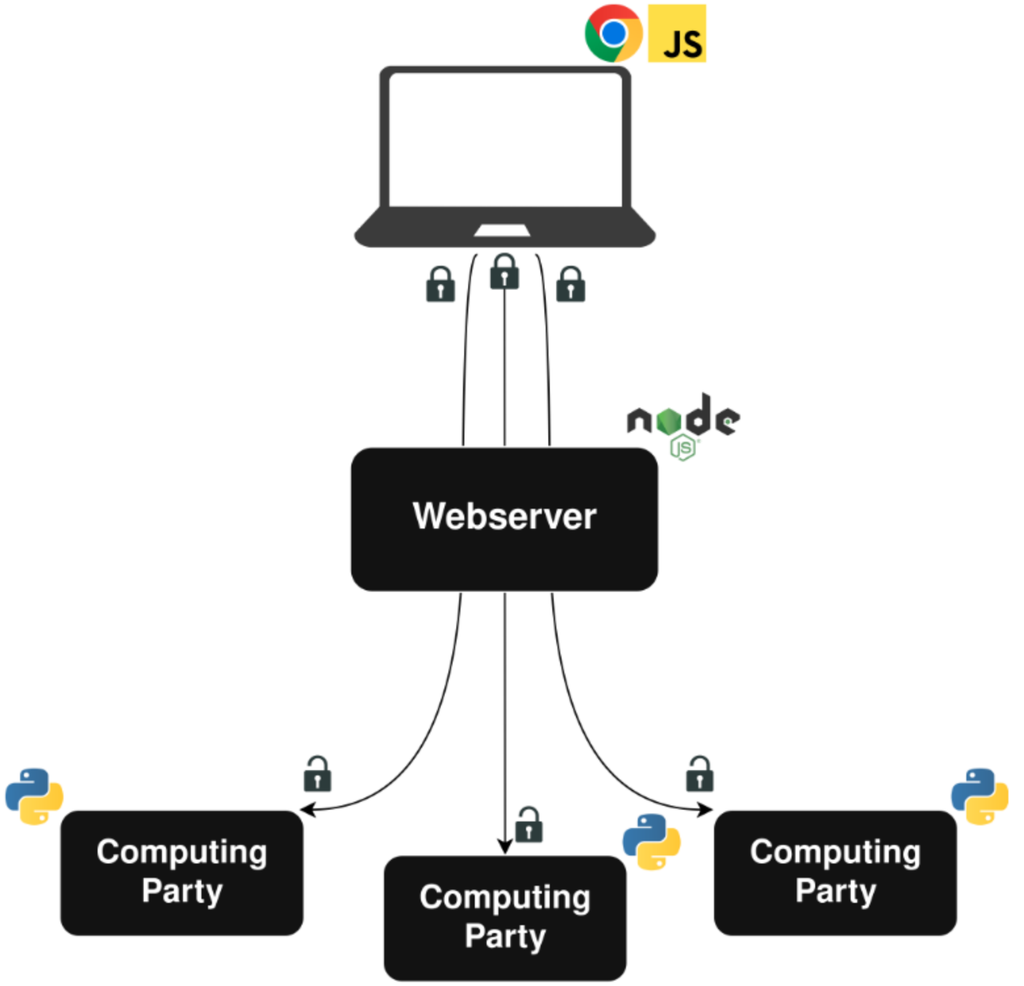


Client Plugin

- Custom-built Chrome plugin to monitor browsing
- Daily data uploads of secret-shared histograms
- Client-side secret sharing and encryption

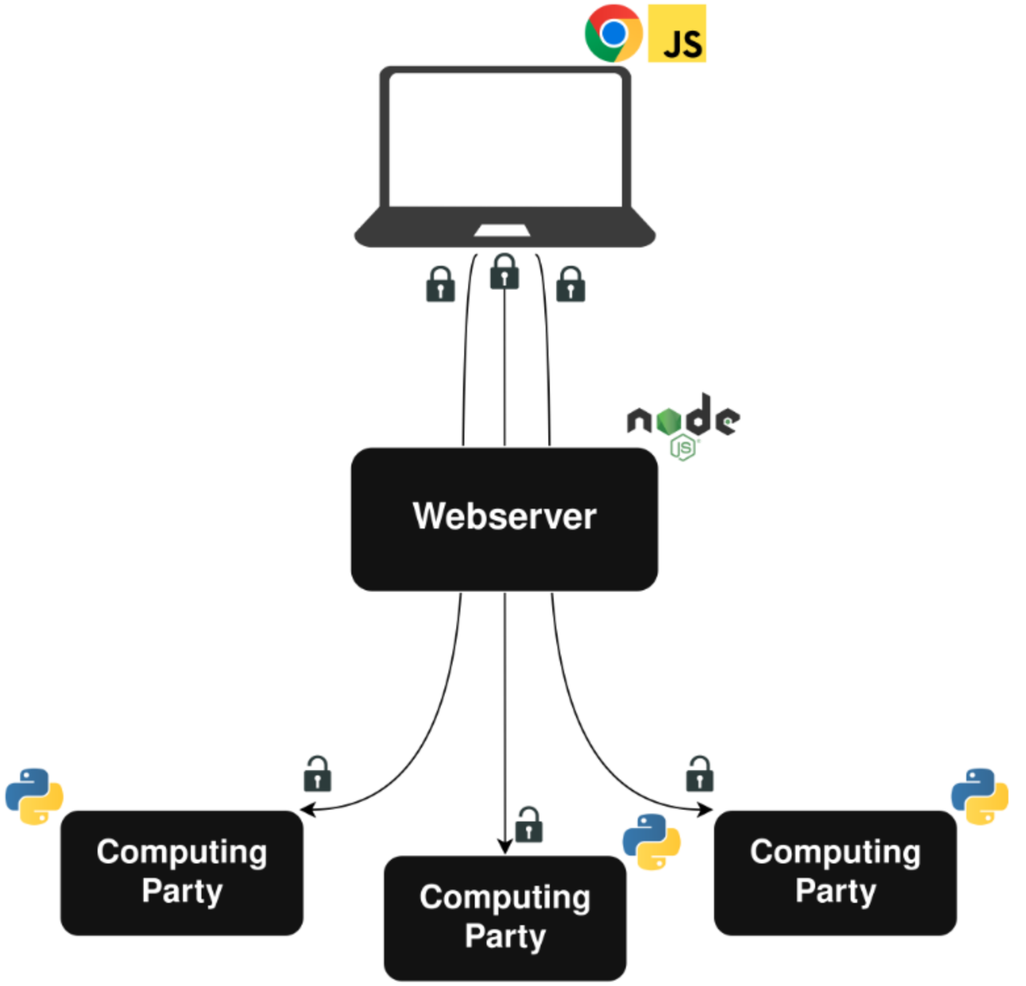


Webserver


JS
node
Webserver
Computing Party
Computing Party
Computing Party

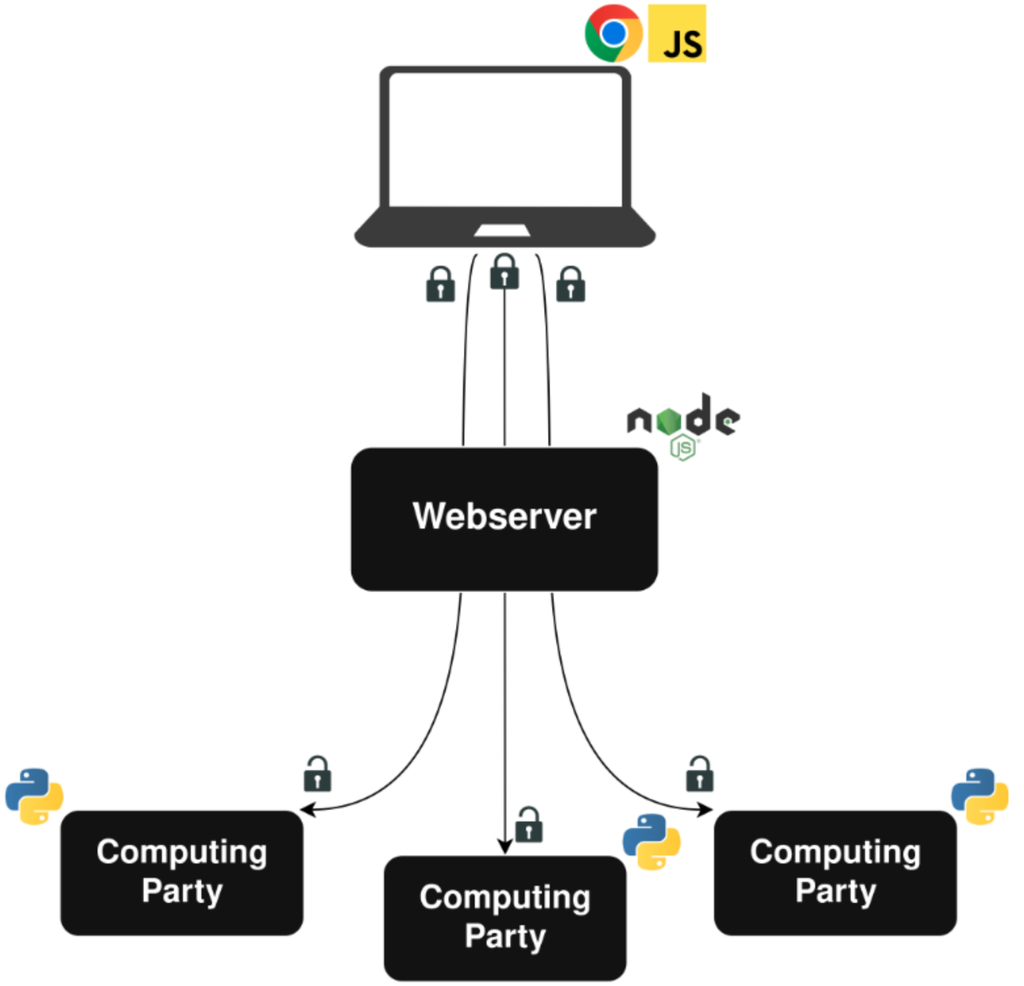
Webserver

- Simplifies interaction with clients



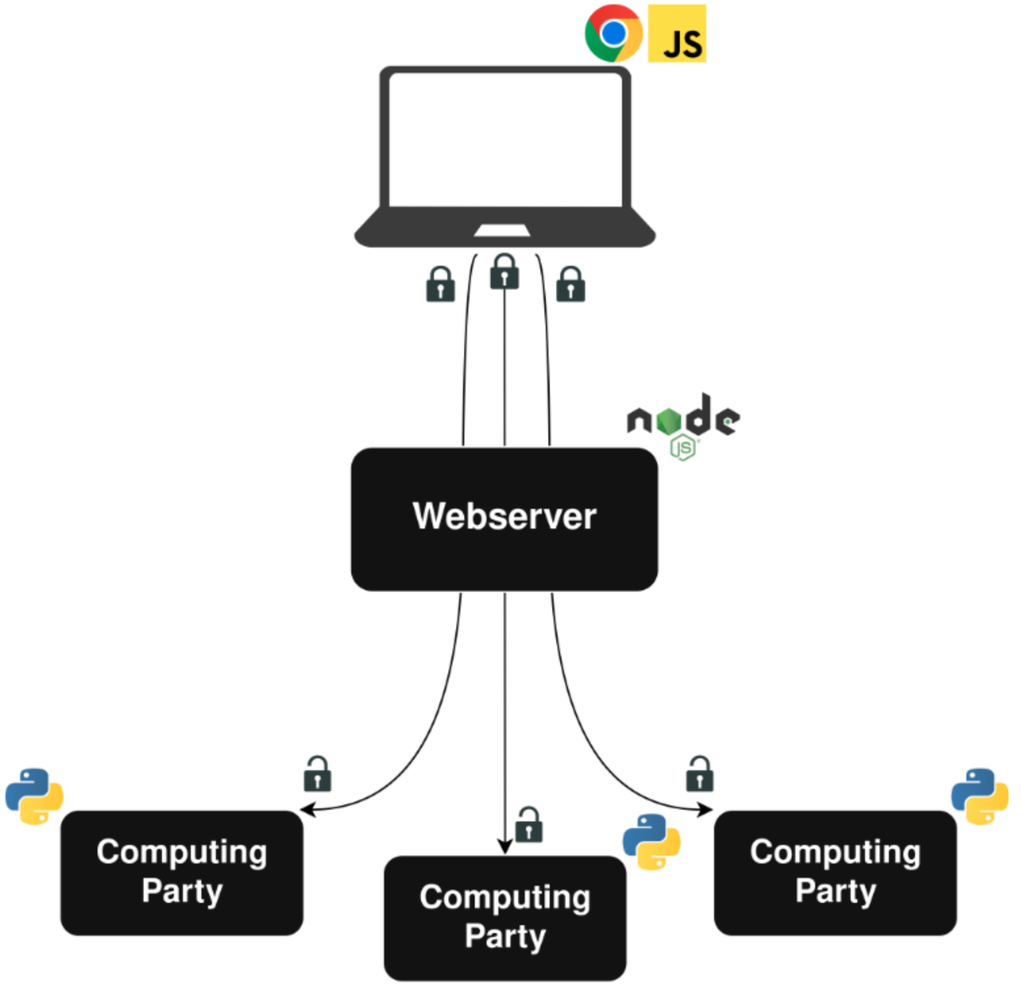
Webserver

- Simplifies interaction with clients
- Collects basic metadata

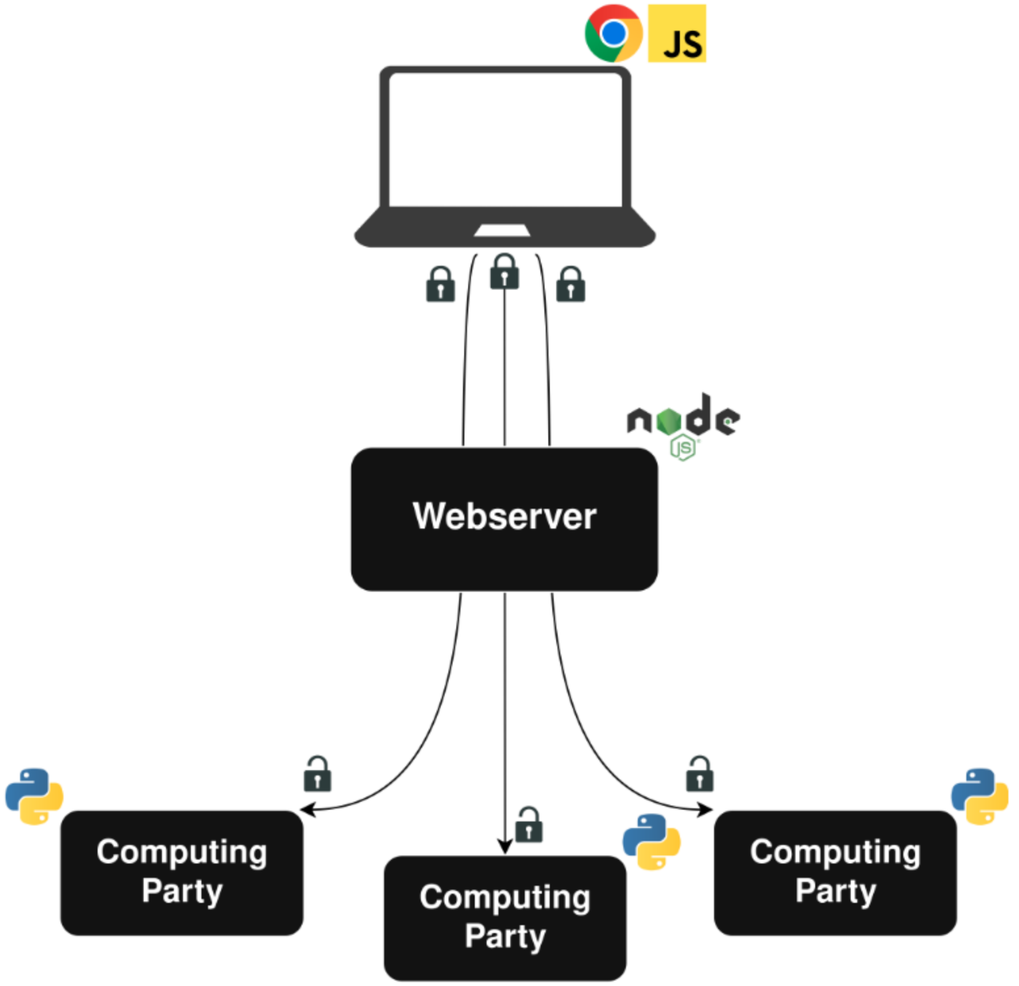


Webserver

- Simplifies interaction with clients
- Collects basic metadata
- Never sees any private data

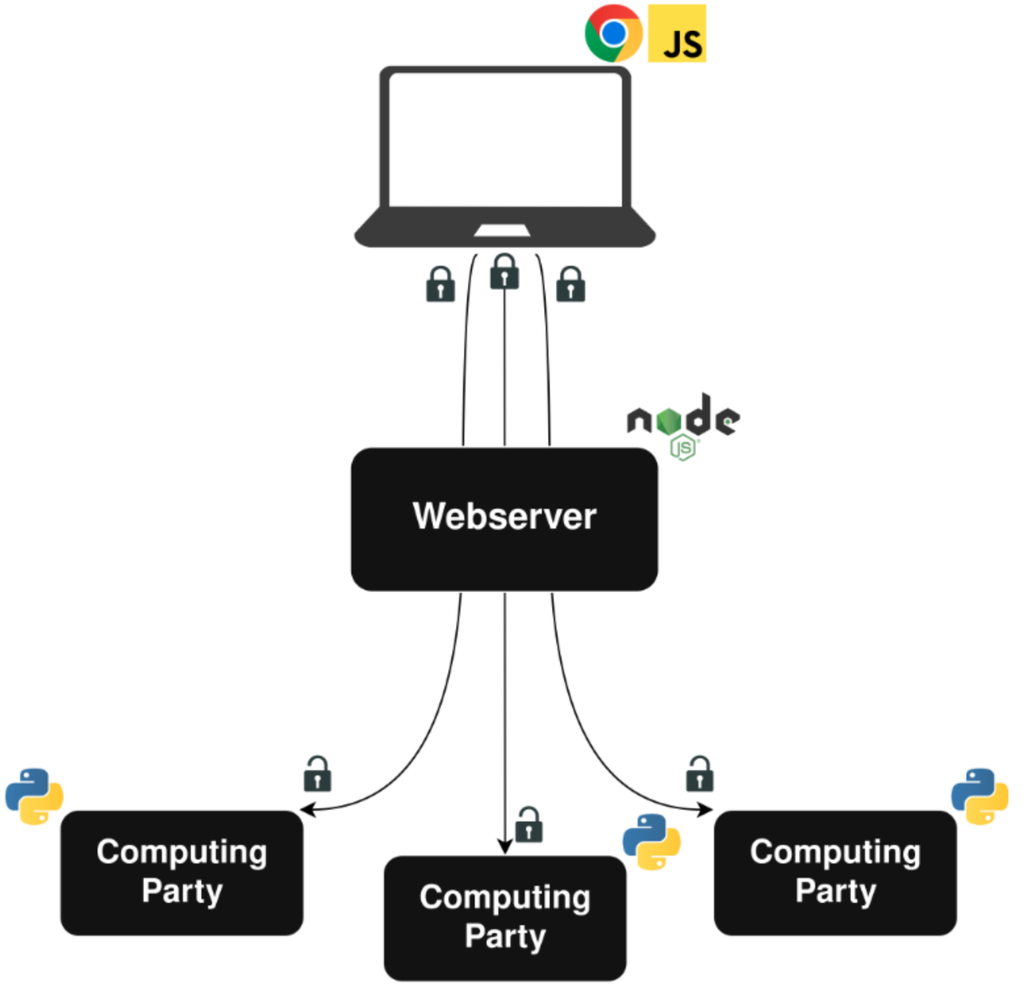


MPC Backend


JS
node
Webserver
Computing Party
Computing Party
Computing Party

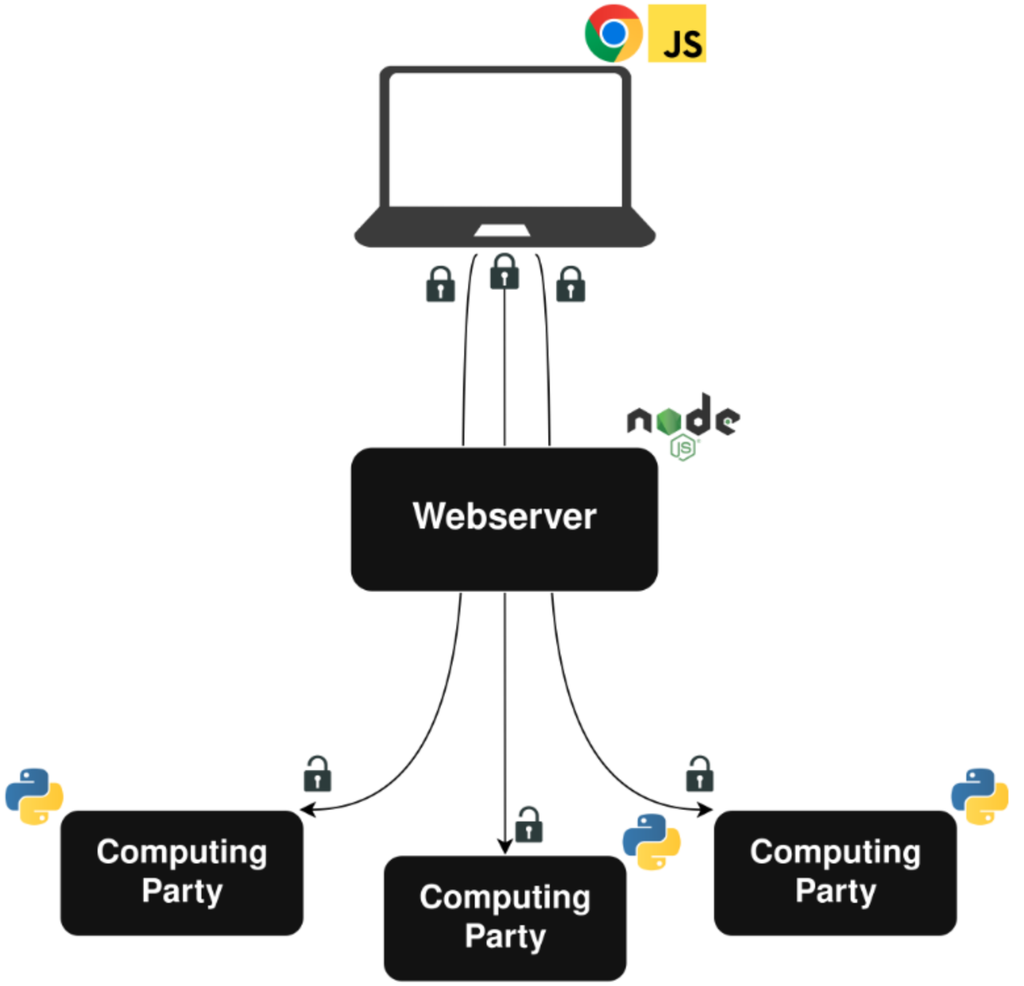
MPC Backend

- Three party computation with an honest majority



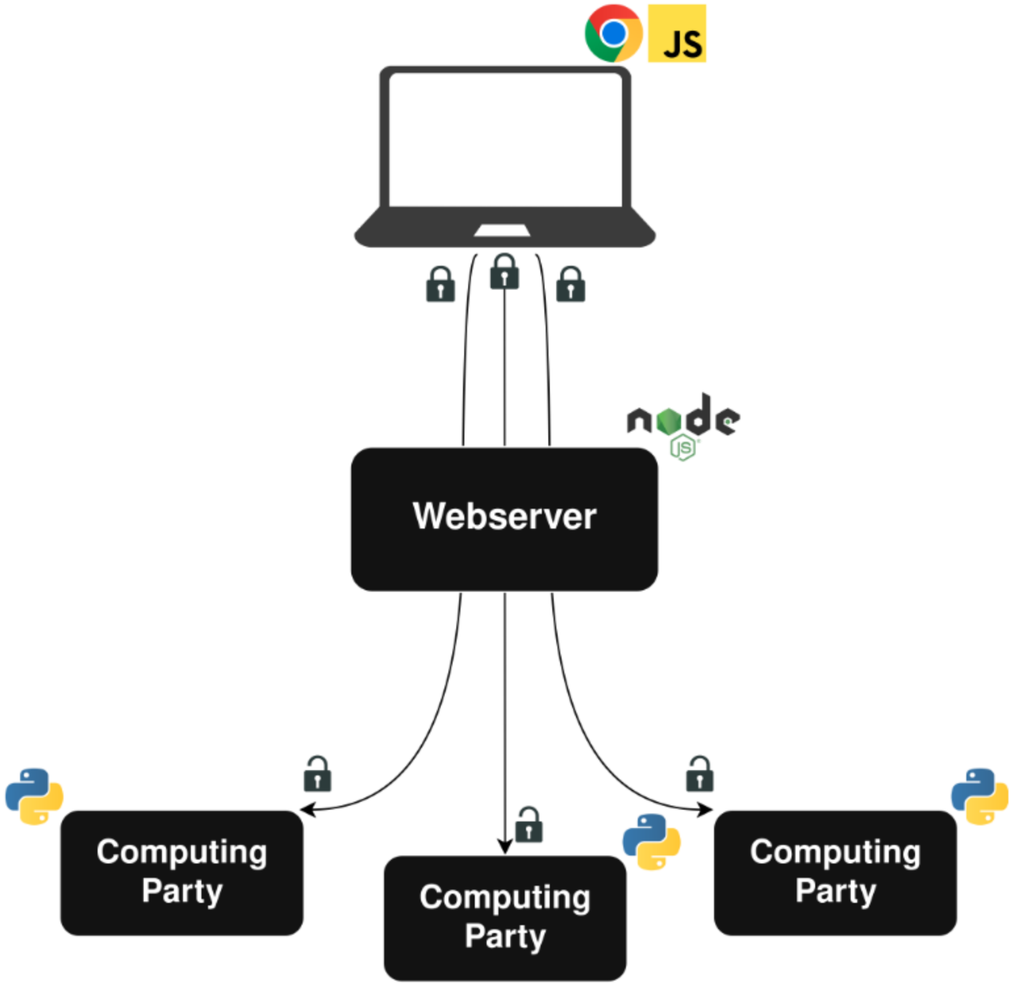
MPC Backend

- Three party computation with an honest majority
- We used and augmented the CrypTen library



MPC Backend

- Three party computation with an honest majority
- We used and augmented the CrypTen library
- We implemented an algorithm for LLP under MPC



The Learning Process

The Learning Process

We’re working with the learning from label proportions (LLP) problem.

The Learning Process

We’re working with the learning from label proportions (LLP) problem.

- The first implementation under MPC of an LLP model

The Learning Process

We’re working with the learning from label proportions (LLP) problem.

- The first implementation under MPC of an LLP model
- Mix of out-of-the-box components and custom implementations

Learning from Label Proportions (LLP)

Learning from Label Proportions (LLP)

- Each user uploads an *unlabeled* 1,034-element vector every day

Learning from Label Proportions (LLP)

- Each user uploads an *unlabeled* 1,034-element vector every day
  - Number of visits to the top 517 sites

Learning from Label Proportions (LLP)

- Each user uploads an *unlabeled* 1,034-element vector every day
  - Number of visits to the top 517 sites
  - Number of social media referrals to the top 517 sites

Learning from Label Proportions (LLP)

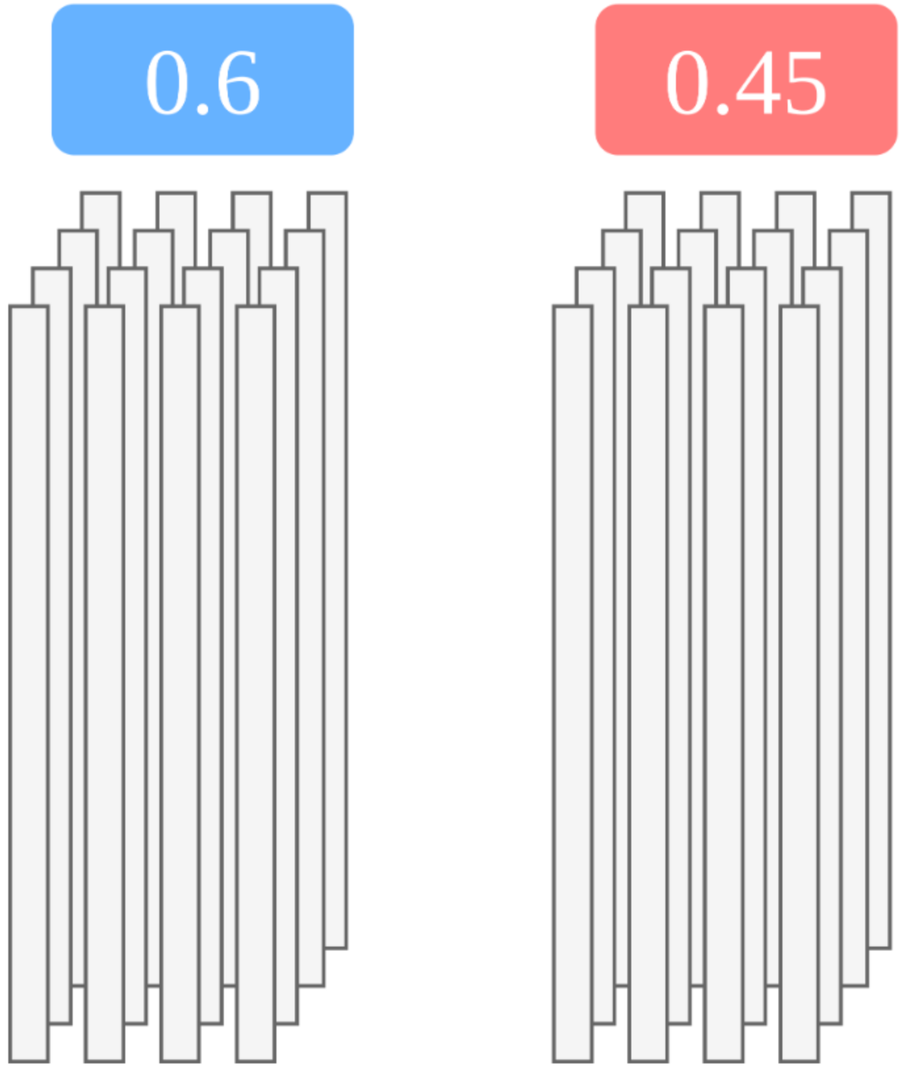
- Each user uploads an *unlabeled* 1,034-element vector every day
  - Number of visits to the top 517 sites
  - Number of social media referrals to the top 517 sites
- Unlabeled vectors are grouped by state

Learning from Label Proportions (LLP)

- Each user uploads an *unlabeled* 1,034-element vector every day
  - Number of visits to the top 517 sites
  - Number of social media referrals to the top 517 sites
- Unlabeled vectors are grouped by state
- Each state has a ground-truth label

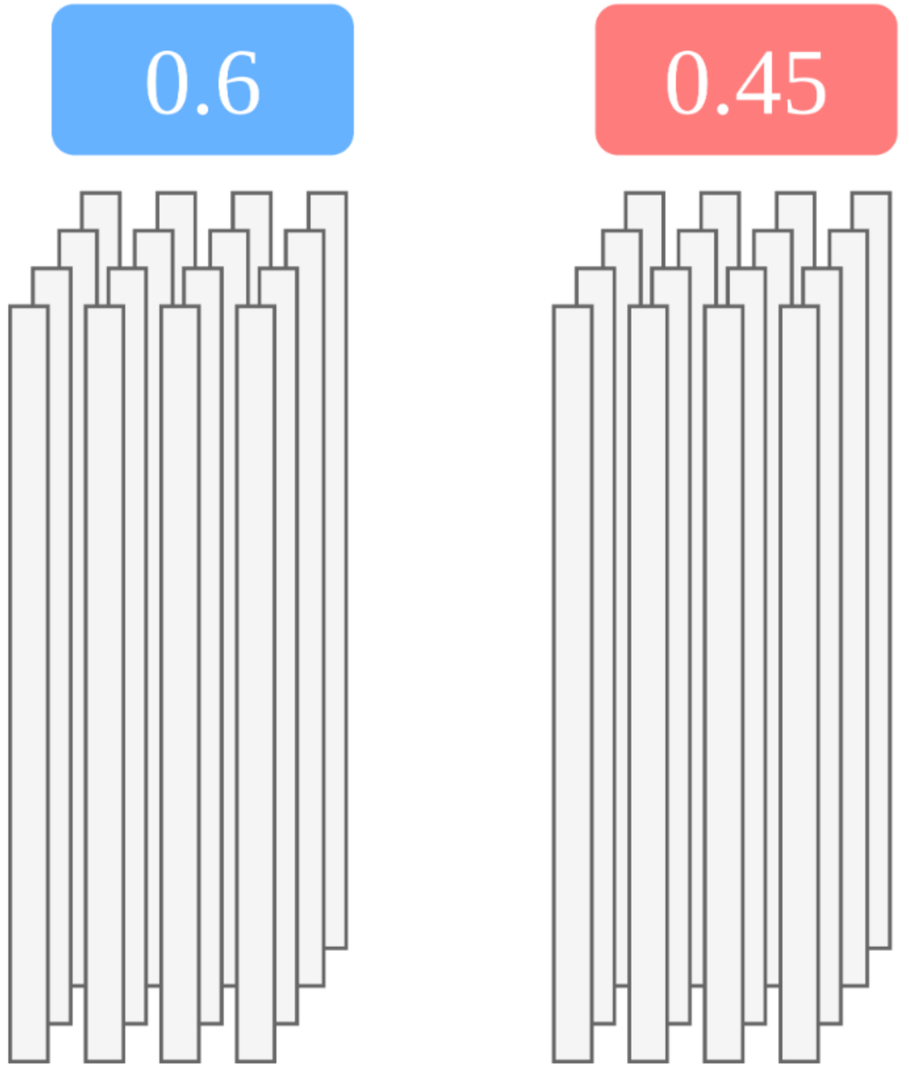
Learning from Label Proportions (LLP)

- Each user uploads an *unlabeled* 1,034-element vector every day
  - Number of visits to the top 517 sites
  - Number of social media referrals to the top 517 sites
- Unlabeled vectors are grouped by state
- Each state has a ground-truth label



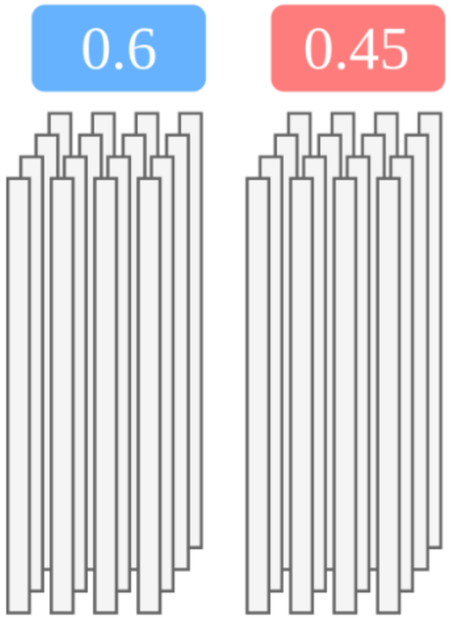
Learning from Label Proportions (LLP)

- Each user uploads an *unlabeled* 1,034-element vector every day
  - Number of visits to the top 517 sites
  - Number of social media referrals to the top 517 sites
- Unlabeled vectors are grouped by state
- Each state has a ground-truth label
- Train on aggregate ground truth

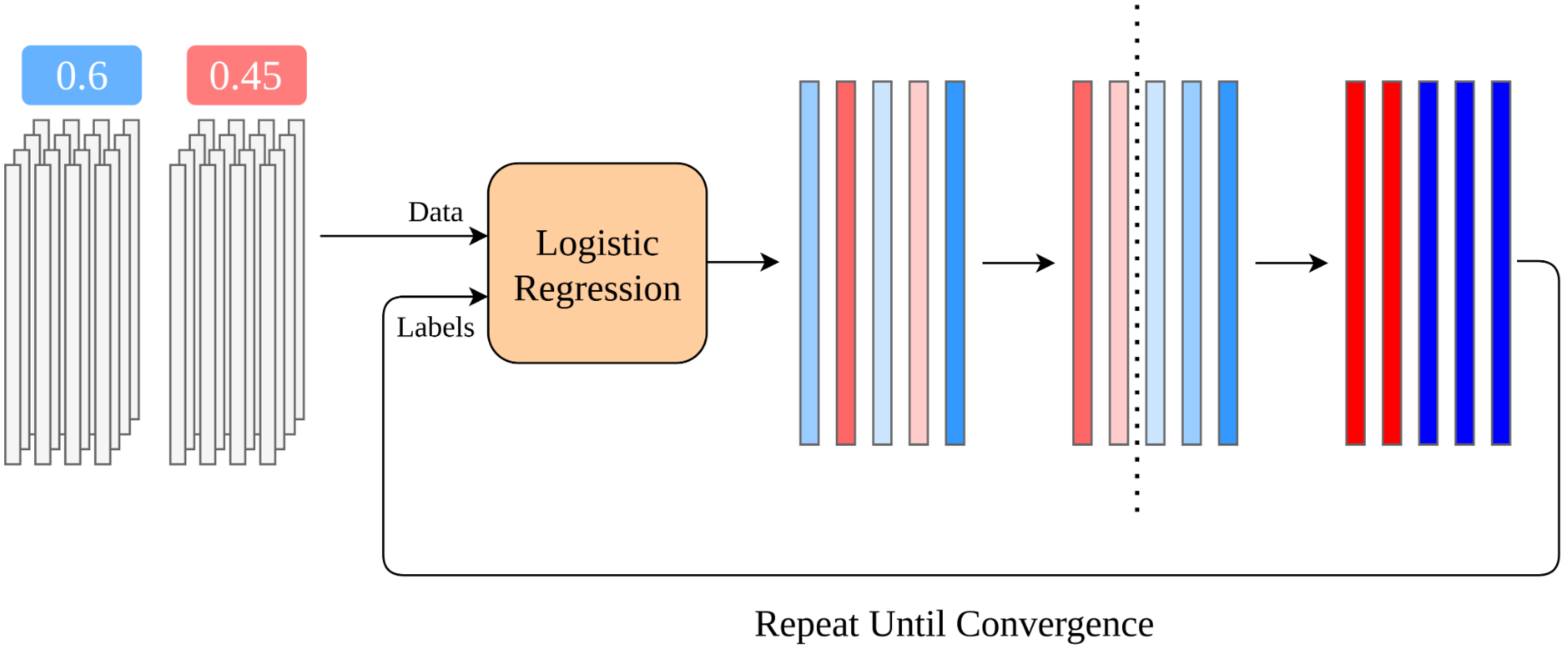


The Plaintext Algorithm

The Plaintext Algorithm



The Plaintext Algorithm



Translating to MPC

```
initialize_labels()

repeat until convergence:
    model = train(data, labels)
    predictions = inference(model, data)

    for each state:
        compute_threshold(truth[state], predictions[state])

    for each user:
        labels[user] = predictions[user] ⩾ thresholds[state]
```

Translating to MPC

```
initialize_labels()

repeat until convergence:
    model = train(data, labels)
    predictions = inference(model, data)

    for each state:
        compute_threshold(truth[state], predictions[state])

    for each user:
        labels[user] = predictions[user] ⩾ thresholds[state]
```

- Plaintext initialization

Translating to MPC

```
initialize_labels()

repeat until convergence:
    model = train(data, labels)
    predictions = inference(model, data)

    for each state:
        compute_threshold(truth[state], predictions[state])

    for each user:
        labels[user] = predictions[user] ⩾ thresholds[state]
```

- Plaintext initialization
- Training and inference as black boxes

Translating to MPC

```
initialize_labels()

repeat until convergence:
    model = train(data, labels)
    predictions = inference(model, data)

    for each state:
        compute_threshold(truth[state], predictions[state])

    for each user:
        labels[user] = predictions[user] ⩾ thresholds[state]
```

- Plaintext initialization
- Training and inference as black boxes
- Computing thresholds requires oblivious sorting

Translating to MPC

```
initialize_labels()

repeat until convergence:
    model = train(data, labels)
    predictions = inference(model, data)

    for each state:
        compute_threshold(truth[state], predictions[state])

    for each user:
        labels[user] = predictions[user] ⩾ thresholds[state]
```

- Plaintext initialization
- Training and inference as black boxes
- Computing thresholds requires oblivious sorting
- Updating labels requires oblivious comparison

Translating to MPC

```
initialize_labels()

repeat until convergence:
    model = train(data, labels)
    predictions = inference(model, data)

    for each state:
        compute_threshold(truth[state], predictions[state])

    for each user:
        labels[user] = predictions[user] ⩾ thresholds[state]
```

Convergence Check

- Plaintext initialization
- Training and inference as black boxes
- Computing thresholds requires oblivious sorting
- Updating labels requires oblivious comparison

Translating to MPC

```
initialize_labels()

repeat until convergence:
    model = train(data, labels)
    predictions = inference(model, data)

    for each state:
        compute_threshold(truth[state], predictions[state])

    for each user:
        labels[user] = predictions[user] ⩾ thresholds[state]
```

- Plaintext initialization
- Training and inference as black boxes
- Computing thresholds requires oblivious sorting
- Updating labels requires oblivious comparison

Convergence Check

- Vector oblivious comparison

Translating to MPC

```
initialize_labels()

repeat until convergence:
    model = train(data, labels)
    predictions = inference(model, data)

    for each state:
        compute_threshold(truth[state], predictions[state])

    for each user:
        labels[user] = predictions[user] ⩾ thresholds[state]
```

- Plaintext initialization
- Training and inference as black boxes
- Computing thresholds requires oblivious sorting
- Updating labels requires oblivious comparison

Convergence Check

- Vector oblivious comparison
- Sum the comparison results

Translating to MPC

```
initialize_labels()

repeat until convergence:
    model = train(data, labels)
    predictions = inference(model, data)

    for each state:
        compute_threshold(truth[state], predictions[state])

    for each user:
        labels[user] = predictions[user] ⩾ thresholds[state]
```

- Plaintext initialization
- Training and inference as black boxes
- Computing thresholds requires oblivious sorting
- Updating labels requires oblivious comparison

Convergence Check

- Vector oblivious comparison
- Sum the comparison results
- Compare sum with convergence threshold

Payment

Payment

- Users are paid and recruited through Mechanical Turk

Payment

- Users are paid and recruited through Mechanical Turk
  - Requires knowledge of non-anonymous MTurk ID

Payment

- Users are paid and recruited through Mechanical Turk
  - Requires knowledge of non-anonymous MTurk ID
  - Which users on which days

Payment

- Users are paid and recruited through Mechanical Turk
  - Requires knowledge of non-anonymous MTurk ID
  - Which users on which days

- BU IRB process took too long

Payment

- Users are paid and recruited through Mechanical Turk
  - Requires knowledge of non-anonymous MTurk ID
  - Which users on which days

- BU IRB process took too long
  - BU was not allowed to hold personally identifiable information

Payment

- Users are paid and recruited through Mechanical Turk
  - Requires knowledge of non-anonymous MTurk ID
  - Which users on which days

- BU IRB process took too long
  - BU was not allowed to hold personally identifiable information

Communication by email

Data Partition

Data Partition

Data Partition

Input	Web Browsing Data	Registered IDs

Data Partition

	BOSTON UNIVERSITY	WashU
Input	Web Browsing Data	Registered IDs
Output	Aggregation by ID	ID Set per Day

Calculating Payments

Calculating Payments

WashU needed to know which users sent data on which days.

Calculating Payments

WashU needed to know which users sent data on which days.

Users send private tags.

Calculating Payments

WashU needed to know which users sent data on which days.

Users send private tags.

```
tag = Hash(id || date)
```

Calculating Payments

WashU needed to know which users sent data on which days.

Users send private tags.

```
tag = Hash(id || date)
```

≈ 96 bits of entropy

Calculating Payments

WashU needed to know which users sent data on which days.

Users send private tags.

```
tag = Hash(id || date)
```

≈ 96 bits of entropy

- BU sends all collected tags to WashU

Calculating Payments

WashU needed to know which users sent data on which days.

Users send private tags.

```
tag = Hash(id || date)
```

≈ 96 bits of entropy

- BU sends all collected tags to WashU
- WashU computes all combinations of users and dates and checks for matches

Computing Anonymous IDs

Computing Anonymous IDs

BU wants to connect data from the same user over multiple days.

Computing Anonymous IDs

BU wants to connect data from the same user over multiple days.

WashU maps IDs to anonymized IDs between 1 and n.

Computing Anonymous IDs

BU wants to connect data from the same user over multiple days.

WashU maps IDs to anonymized IDs between 1 and n.

- WashU sends all possible tags for all anonymous IDs

Computing Anonymous IDs

BU wants to connect data from the same user over multiple days.

WashU maps IDs to anonymized IDs between 1 and n.

- WashU sends all possible tags for all anonymous IDs
  - All **`(date, hash)`** combinations

Computing Anonymous IDs

BU wants to connect data from the same user over multiple days.

WashU maps IDs to anonymized IDs between 1 and n.

- WashU sends all possible tags for all anonymous IDs
  - All `(date, hash)` combinations
- BU aggregates data from same anonymous ID for each week

Looking Ahead to 2028

Looking Ahead to 2028

- Anonymous payments

Looking Ahead to 2028

- Anonymous payments
- More fine-grained data

Looking Ahead to 2028

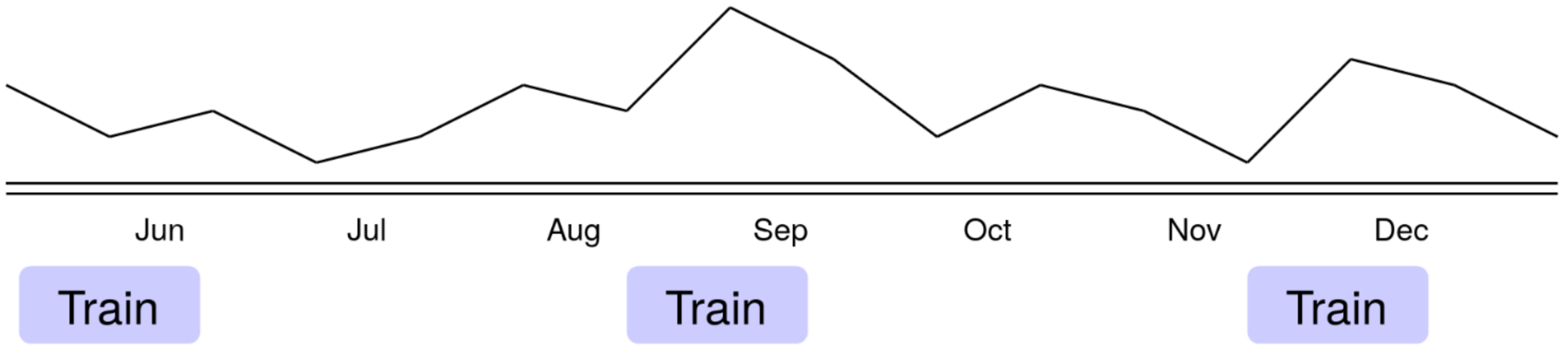
- Anonymous payments
- More fine-grained data
- Redesigning the inference pipeline

Looking Ahead to 2028

- Anonymous payments
- More fine-grained data
- Redesigning the inference pipeline
- Scaling the training pipeline

Looking Ahead to 2028

- Anonymous payments
- More fine-grained data
- Redesigning the inference pipeline
- Scaling the training pipeline



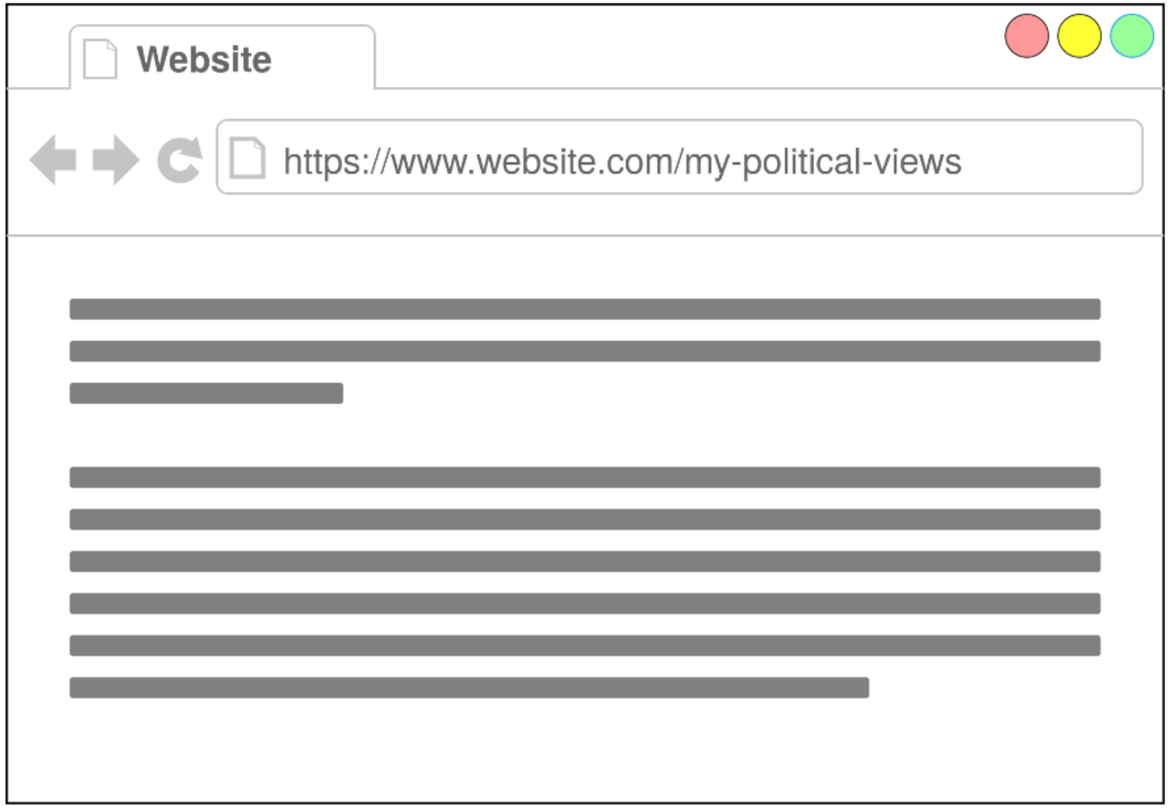
More Fine-Grained Data

More Fine-Grained Data

- Beyond domains

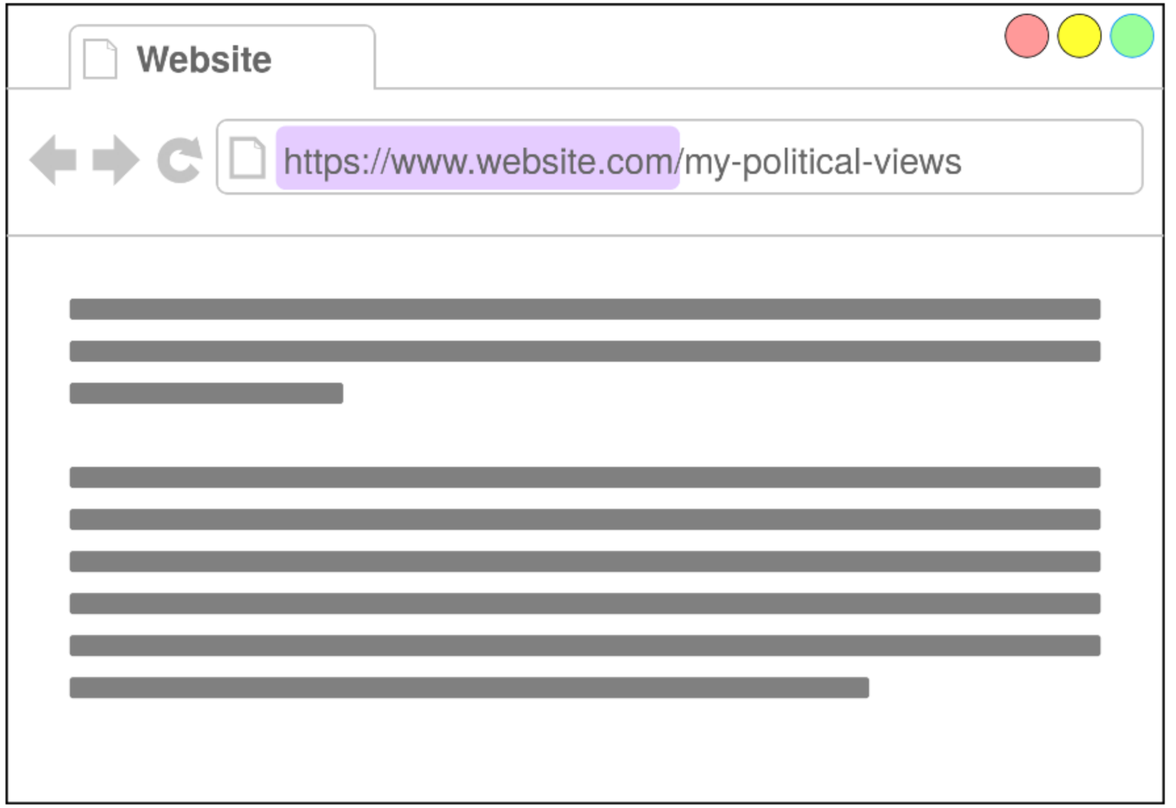
More Fine-Grained Data

- Beyond domains



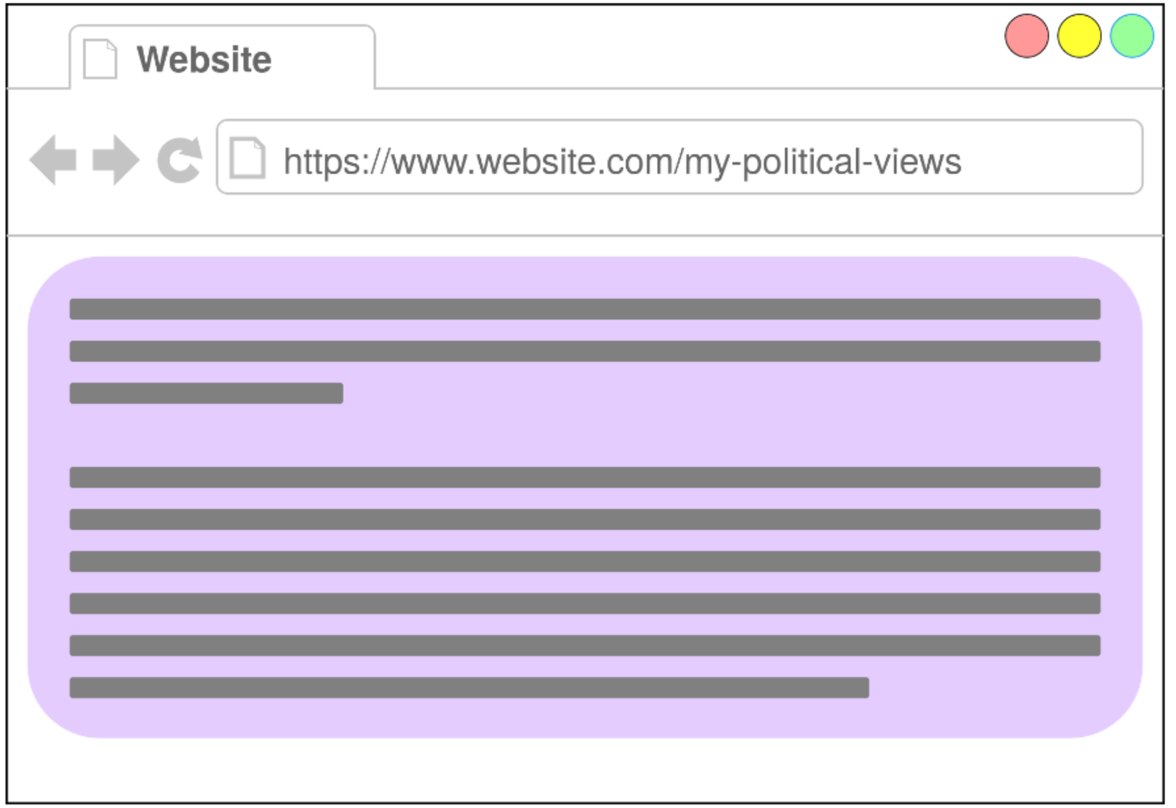
More Fine-Grained Data

- Beyond domains



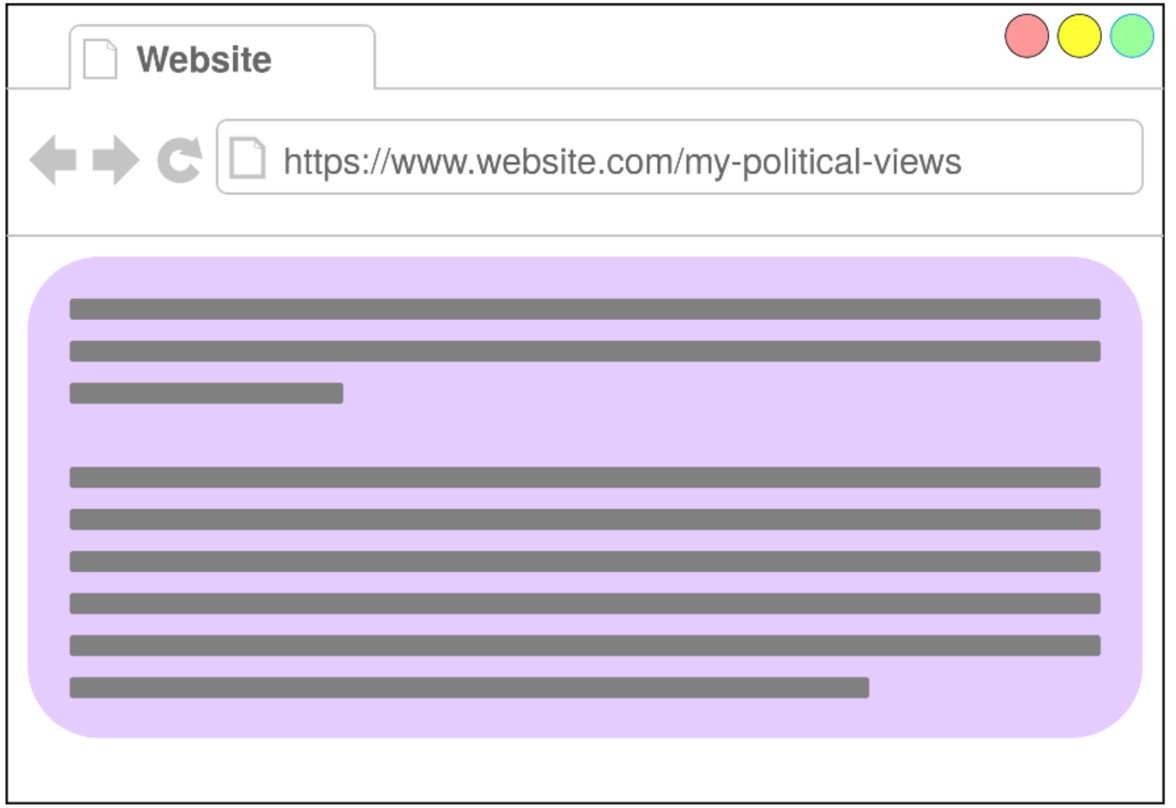
More Fine-Grained Data

- Beyond domains



More Fine-Grained Data

- Beyond domains
- LLM in browser to generate embeddings



A New Inference Approach

A New Inference Approach

- Users can be treated independently

A New Inference Approach

- Users can be treated independently
- Run inference locally, aggregate

A New Inference Approach

- Users can be treated independently
- Run inference locally, aggregate
  - Secure aggregation

A New Inference Approach

- Users can be treated independently
- Run inference locally, aggregate
  - Secure aggregation
  - Work-in-progress open source implementation

A New Inference Approach

- Users can be treated independently
- Run inference locally, aggregate
  - Secure aggregation
  - Work-in-progress open source implementation
- Attach ZK proofs of correct inference evaluation

Scaling Up Training

Scaling Up Training

- 2k $\rightarrow$ 10k clients

Scaling Up Training

- 2k $\rightarrow$ 10k clients
- Bottleneck: oblivious sorting

Scaling Up Training

- 2k $\rightarrow$ 10k clients
- Bottleneck: oblivious sorting
  - 1 minute per 10,000 elements

Scaling Up Training

- 2k $\rightarrow$ 10k clients
- Bottleneck: oblivious sorting
  - 1 minute per 10,000 elements

State of the art is $1000\times$ faster!

Oblivious Sorting

Roadmap

~~1. Deployment of MPC for Political Polling~~

2. Oblivious Sorting

3. ORQ

Oblivious Sorting

Oblivious Sorting

Sorting control flow must be data-independent.

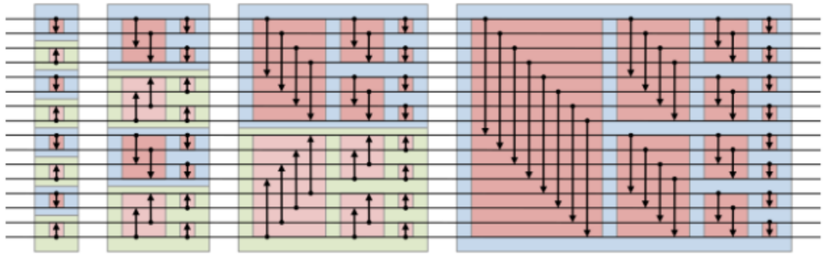
Oblivious Sorting

Sorting control flow must be data-independent.

Sorting Networks

Oblivious Sorting

Sorting control flow must be data-independent.

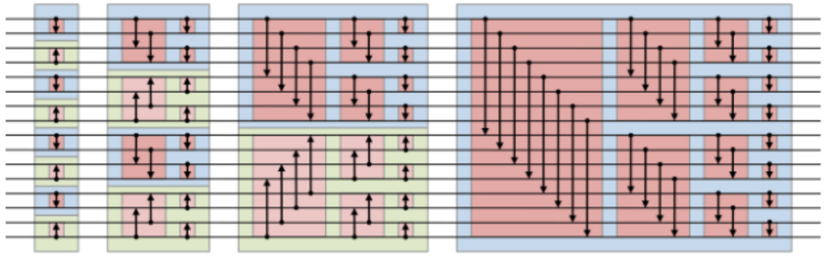


Sorting Networks

- Circuit with a fixed topology of compare-and-swap gates

Oblivious Sorting

Sorting control flow must be data-independent.

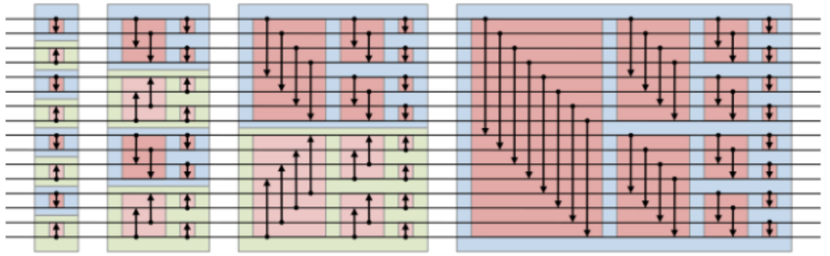


Sorting Networks

- Circuit with a fixed topology of compare-and-swap gates
- Requires $O(n \log^2 n)$ comparison gates

Oblivious Sorting

Sorting control flow must be data-independent.



Sorting Networks

- Circuit with a fixed topology of compare-and-swap gates
- Requires $O(n \log^2 n)$ comparison gates

Goal: Match the $O(n \log n)$ plaintext algorithms.

Our Sorting Contributions

Our Sorting Contributions

- Propose optimized parallel protocols for oblivious quicksort and radixsort

Our Sorting Contributions

- Propose optimized parallel protocols for oblivious quicksort and radixsort
- State-of-the-art implementation

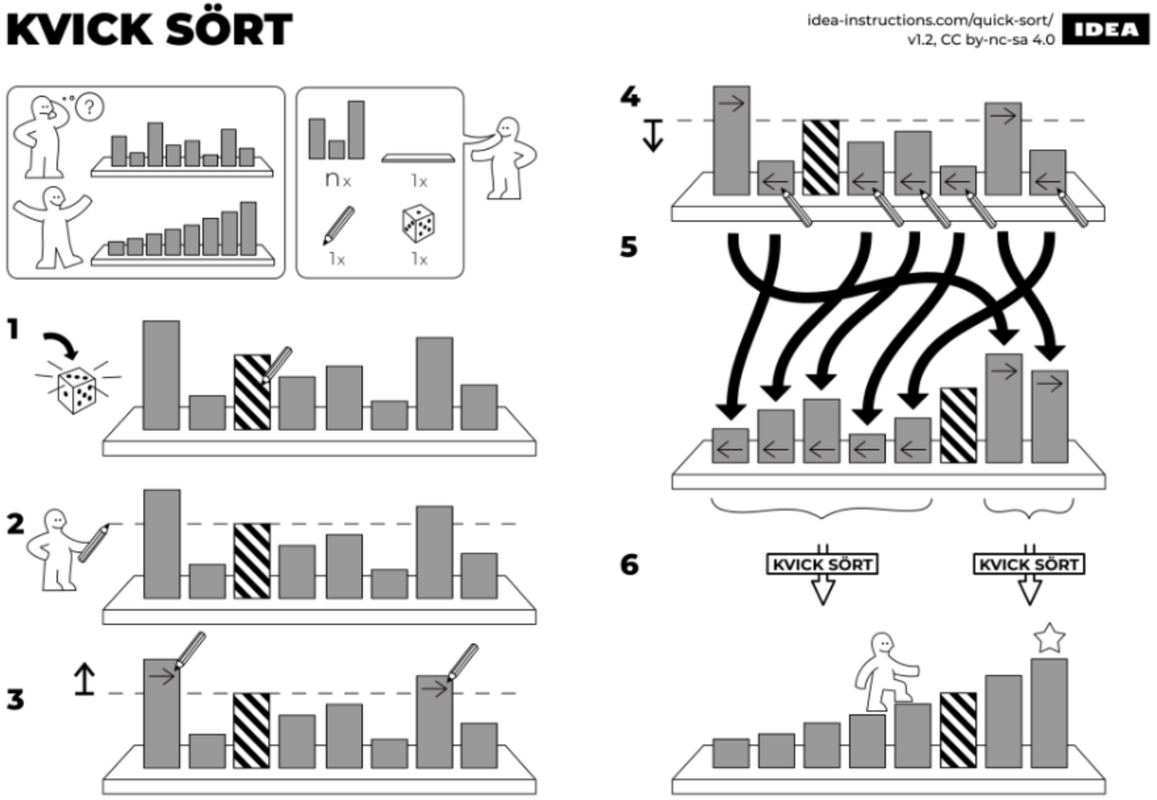
Our Sorting Contributions

- Propose optimized parallel protocols for oblivious quicksort and radixsort
- State-of-the-art implementation
  - Sort half a billion elements

Our Sorting Contributions

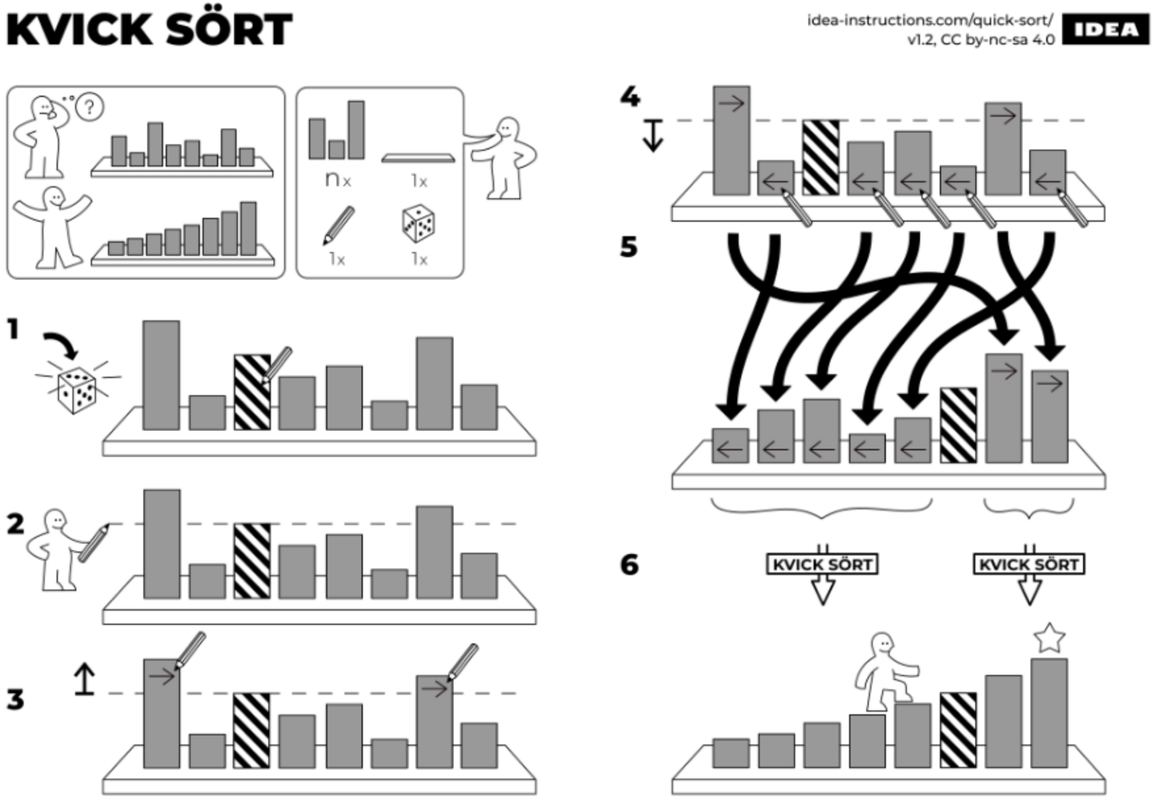
- Propose optimized parallel protocols for oblivious quicksort and radixsort
- State-of-the-art implementation
  - Sort half a billion elements
  - Open source!

Oblivious Quicksort



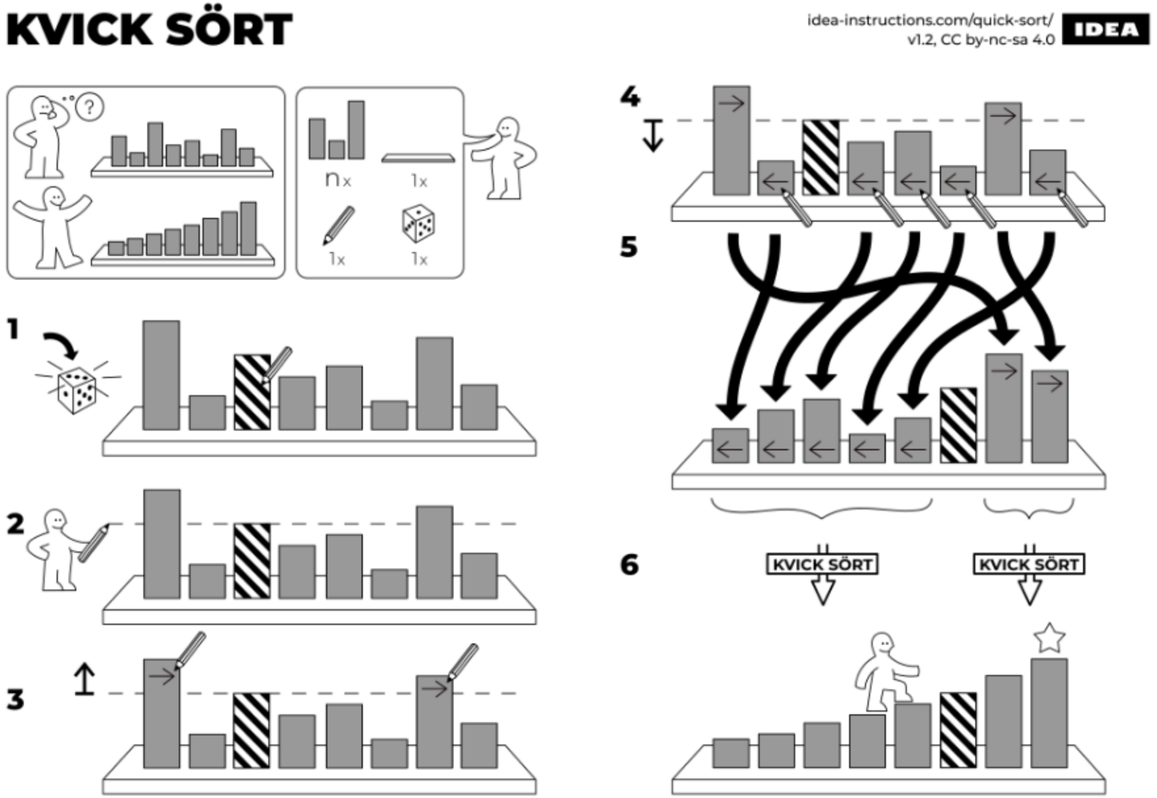
Oblivious Quicksort

- Control flow is non-oblivious



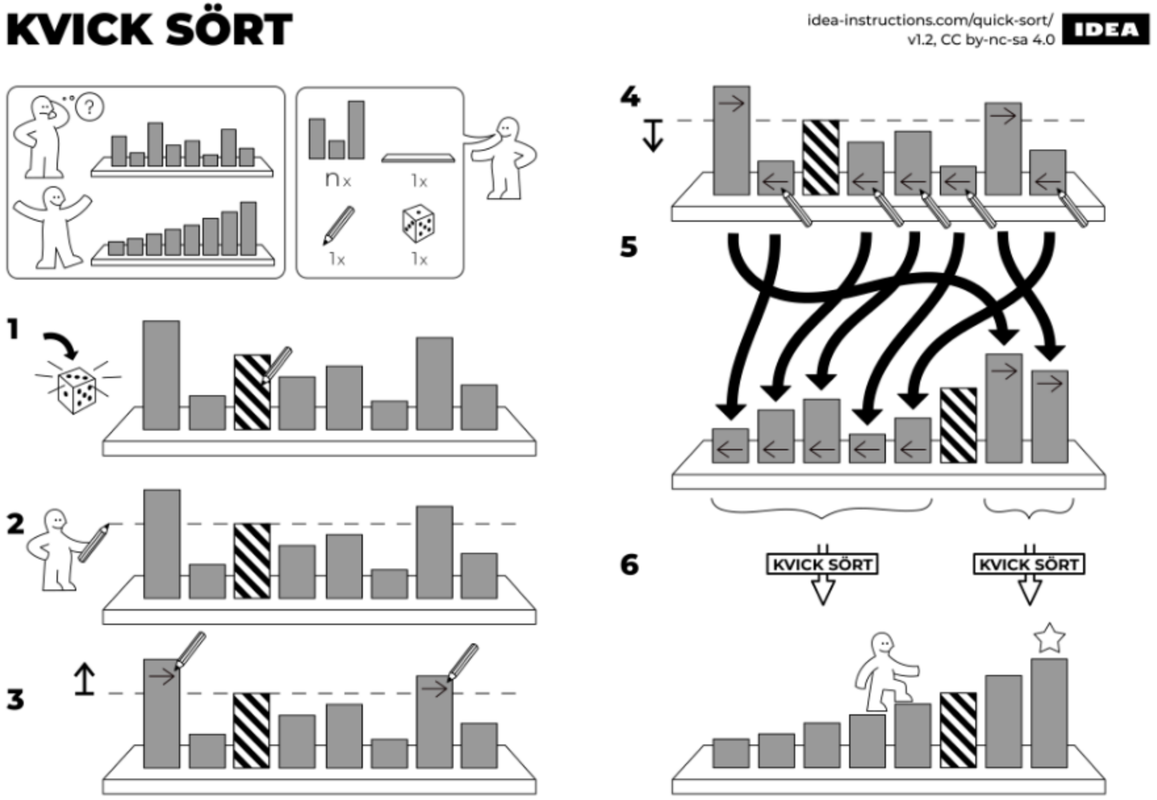
Oblivious Quicksort

- Control flow is non-oblivious
- Shuffle-then-sort paradigm



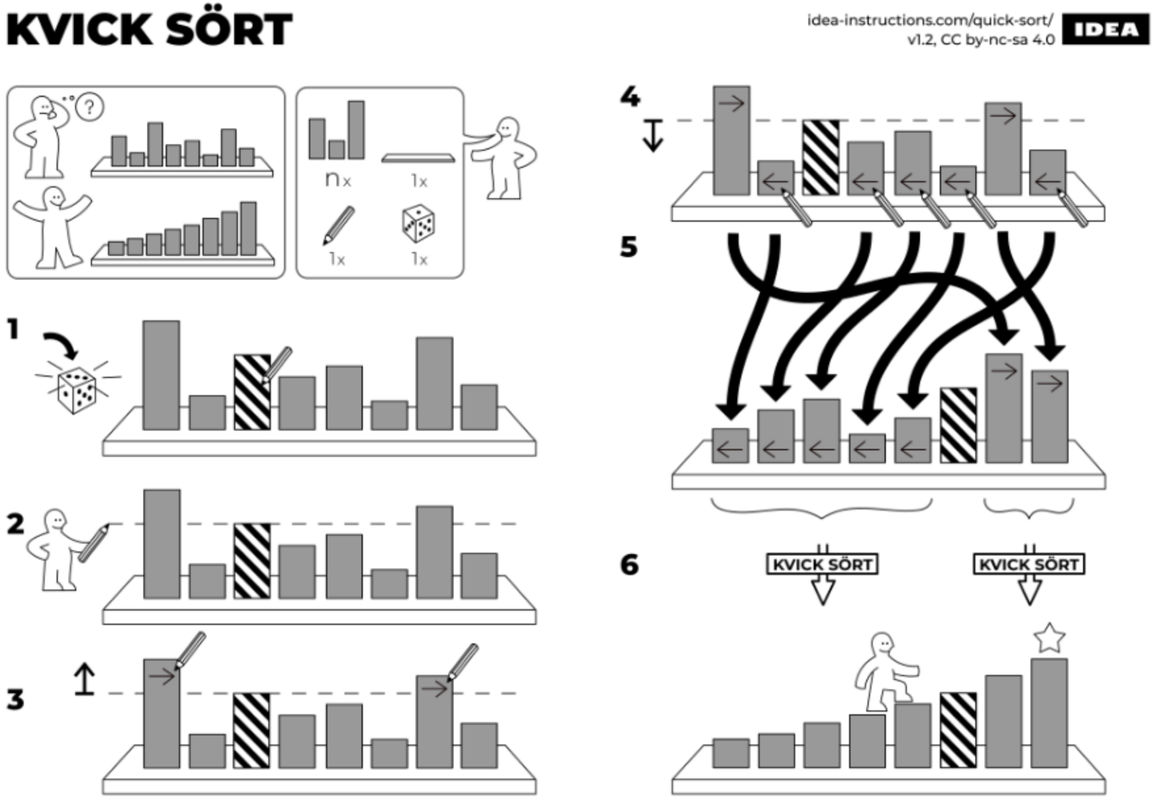
Oblivious Quicksort

- Control flow is non-oblivious
- Shuffle-then-sort paradigm
  - Reveals sorted order of shuffled elements



Oblivious Quicksort

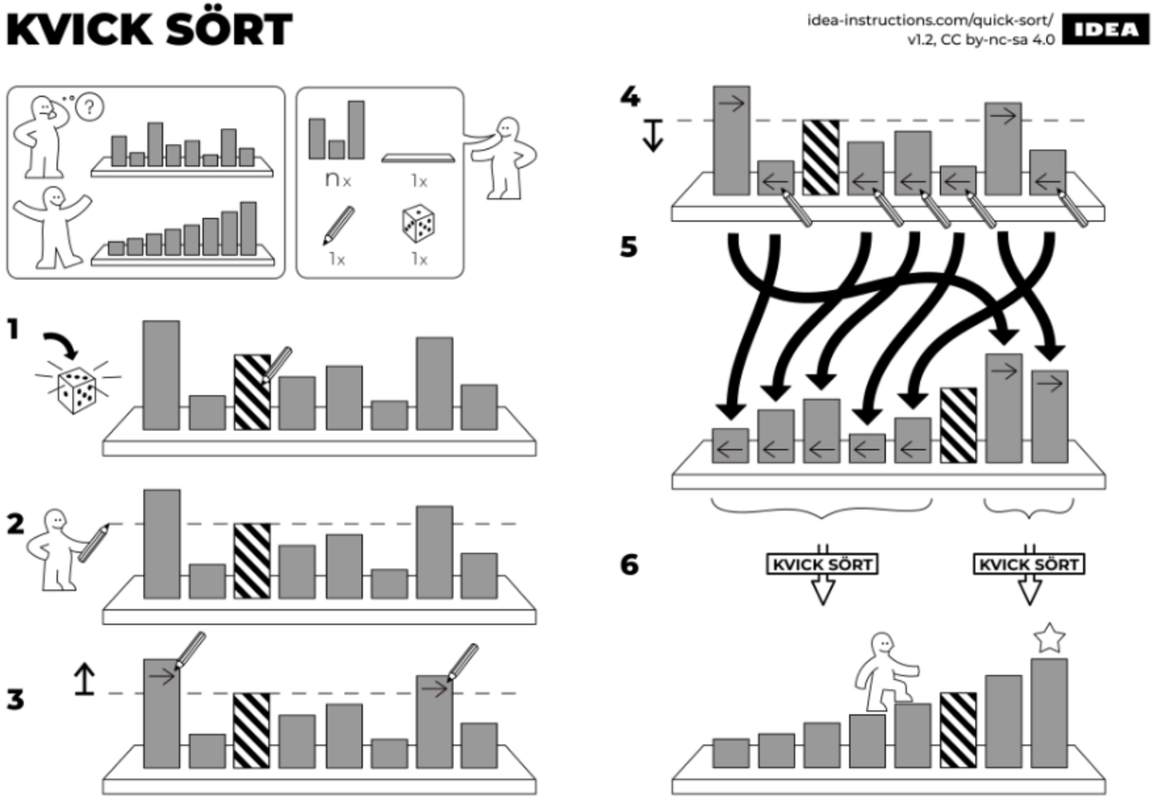
- Control flow is non-oblivious
- Shuffle-then-sort paradigm
  - Reveals sorted order of shuffled elements
  - Uncorrelated with original order



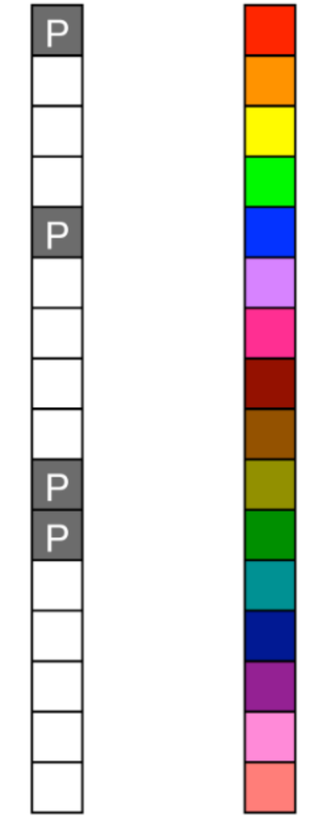
Oblivious Quicksort

- Control flow is non-oblivious
- Shuffle-then-sort paradigm
  - Reveals sorted order of shuffled elements
  - Uncorrelated with original order

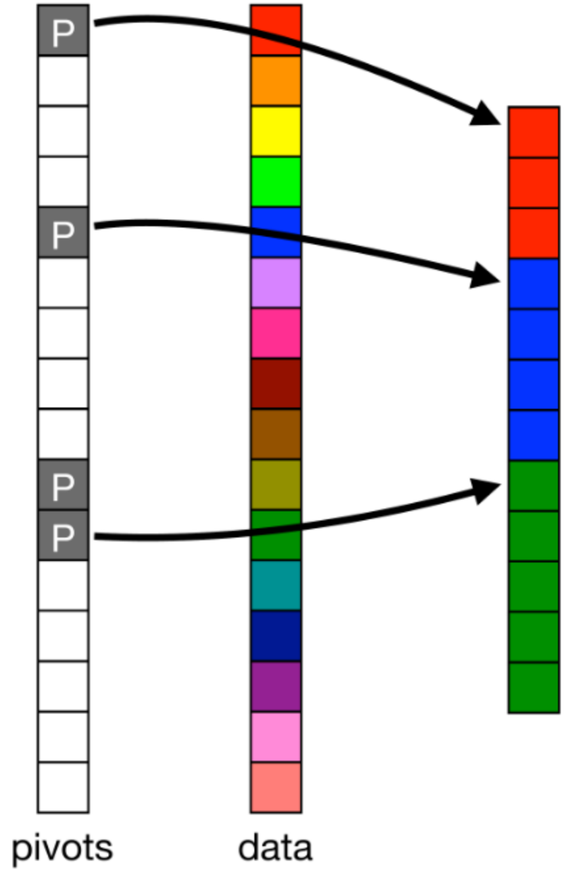
$$[\vec{x}] \to [\pi(\vec{x})] \to [\sigma(\pi(\vec{x}))]$$



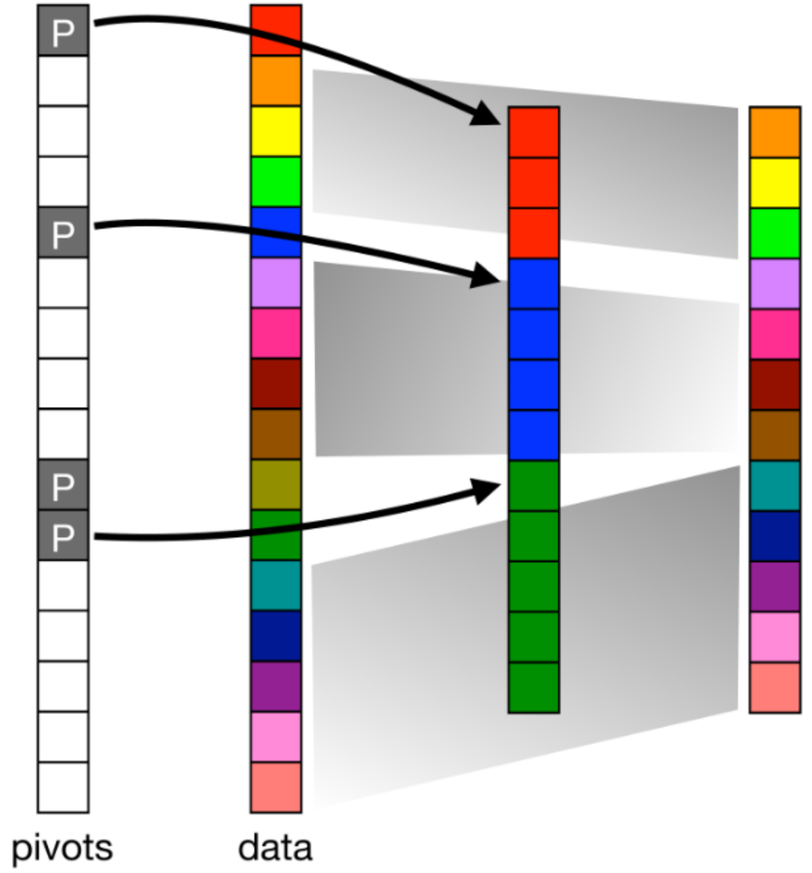
Iterative Quicksort



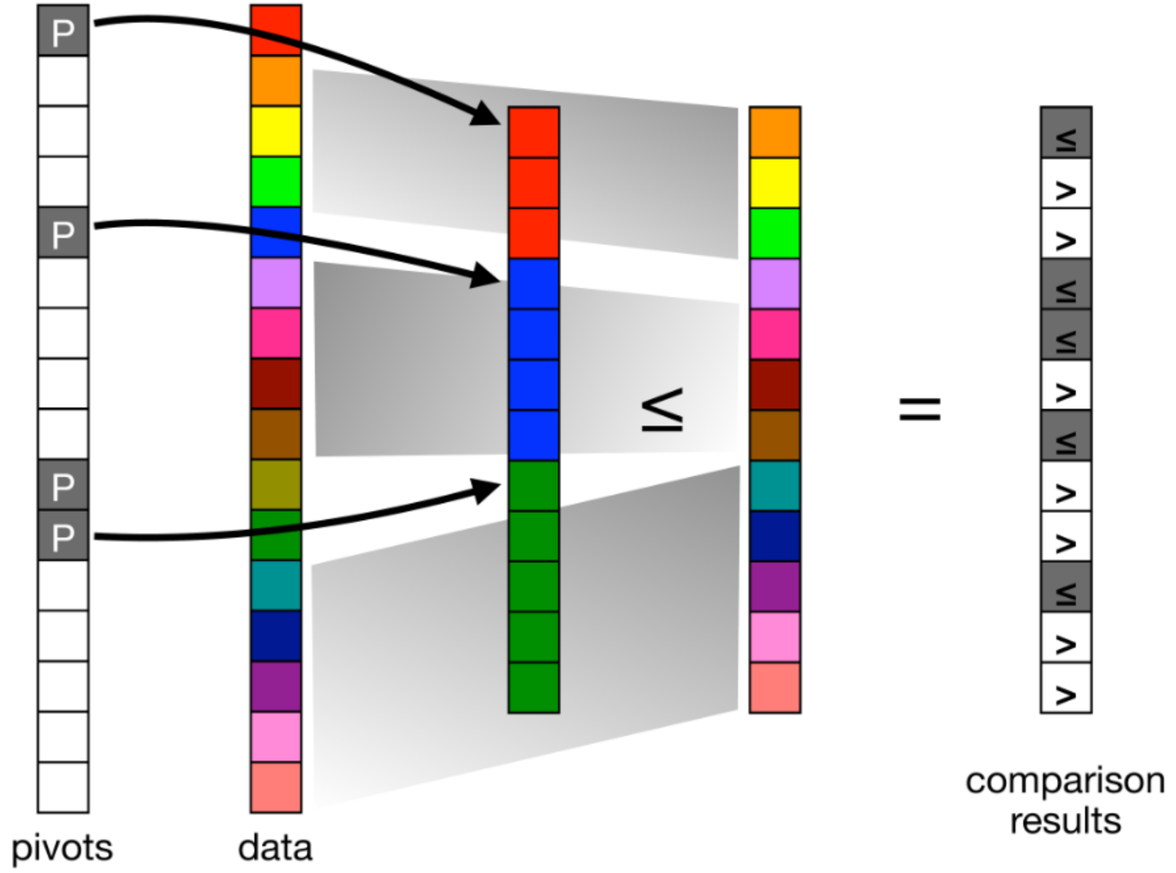
Iterative Quicksort



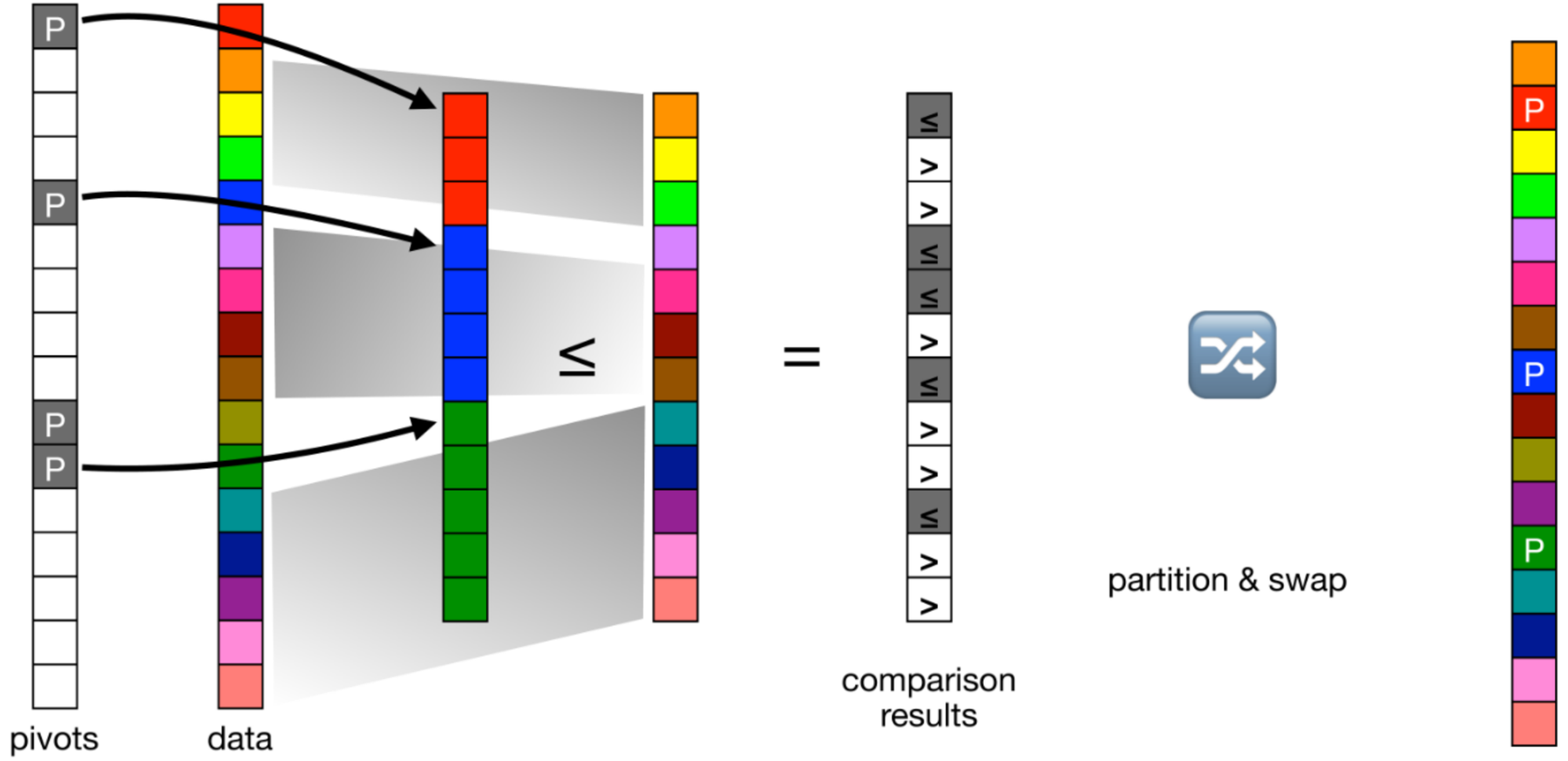
Iterative Quicksort



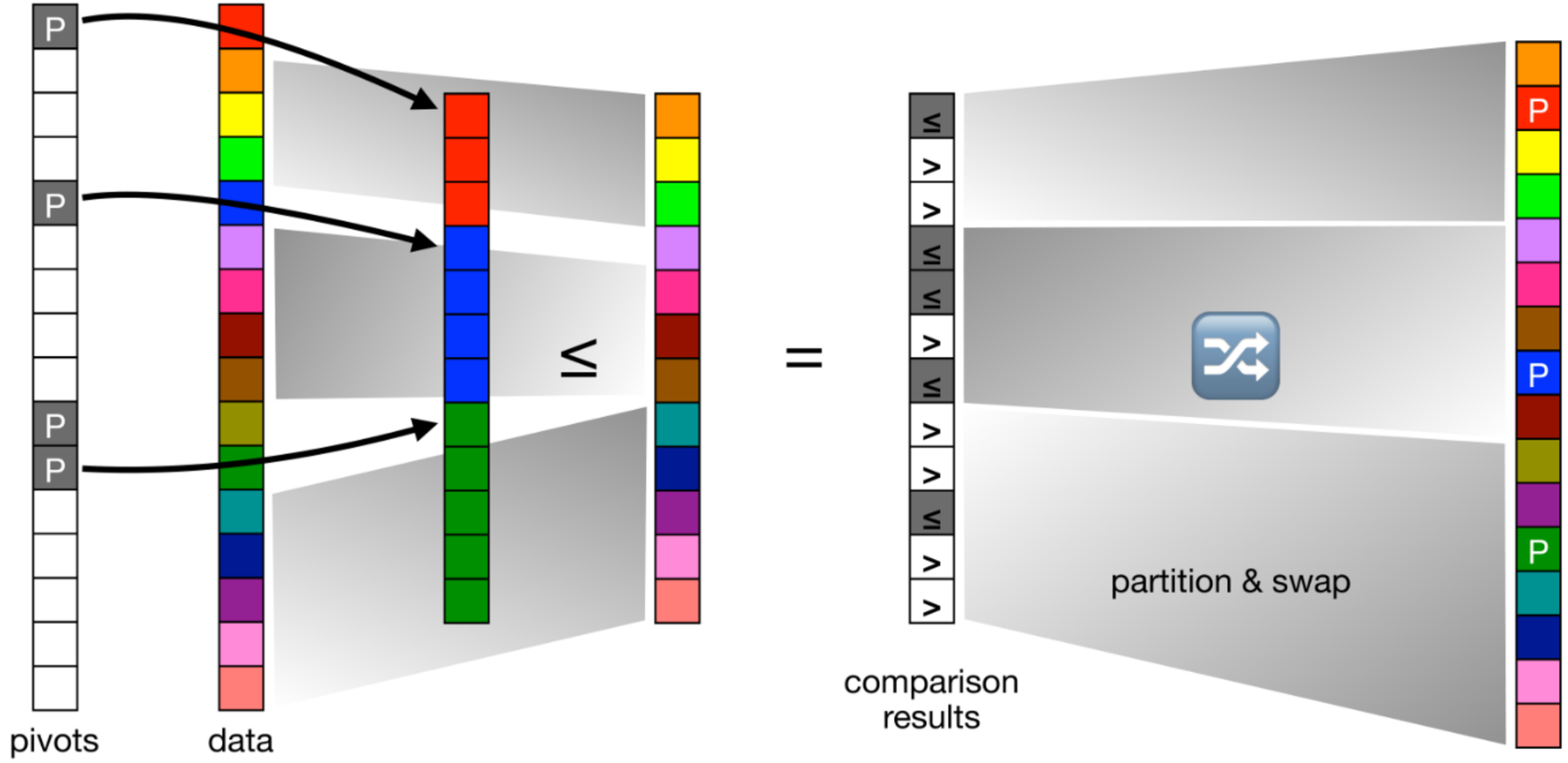
Iterative Quicksort



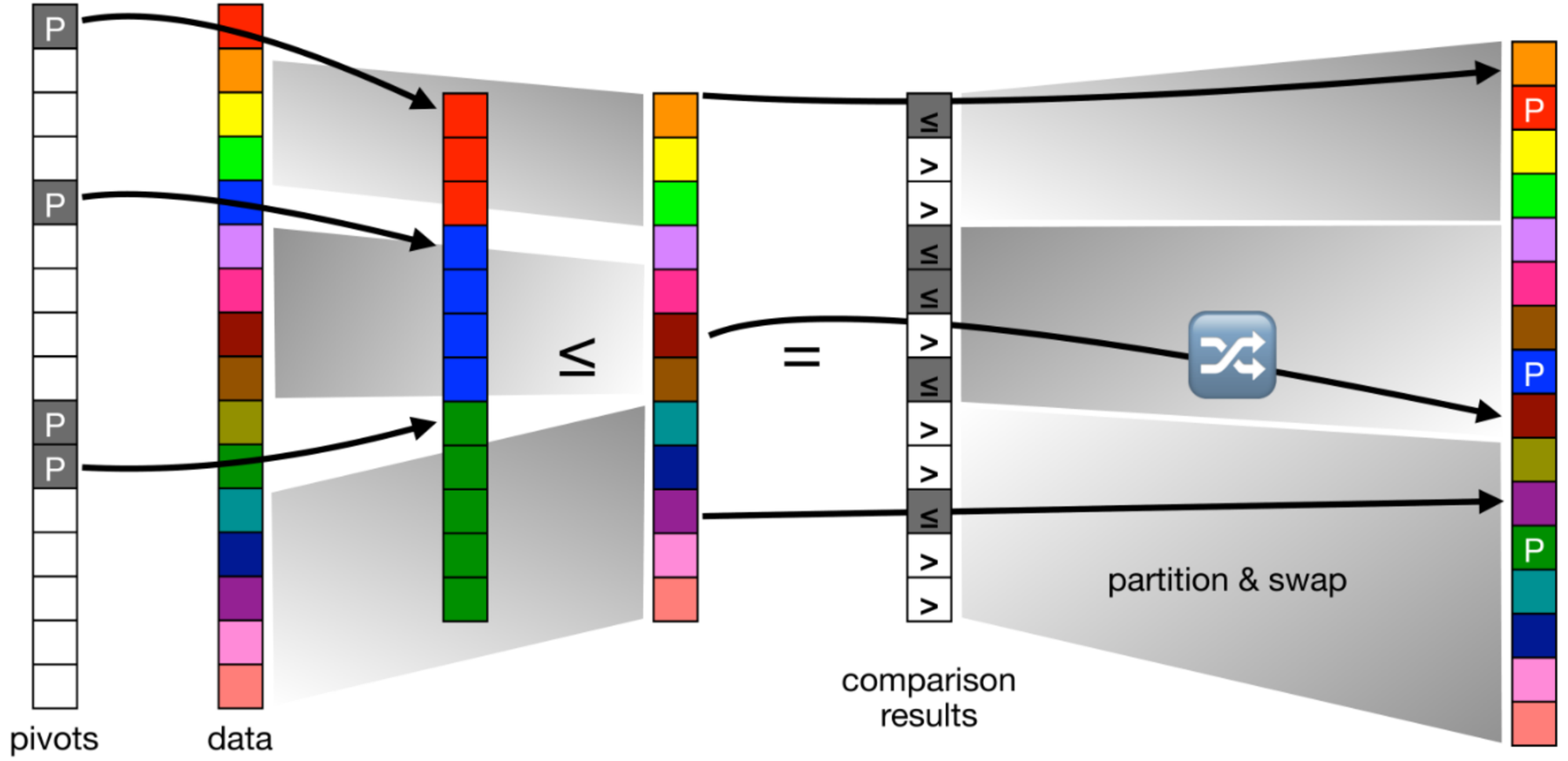
Iterative Quicksort



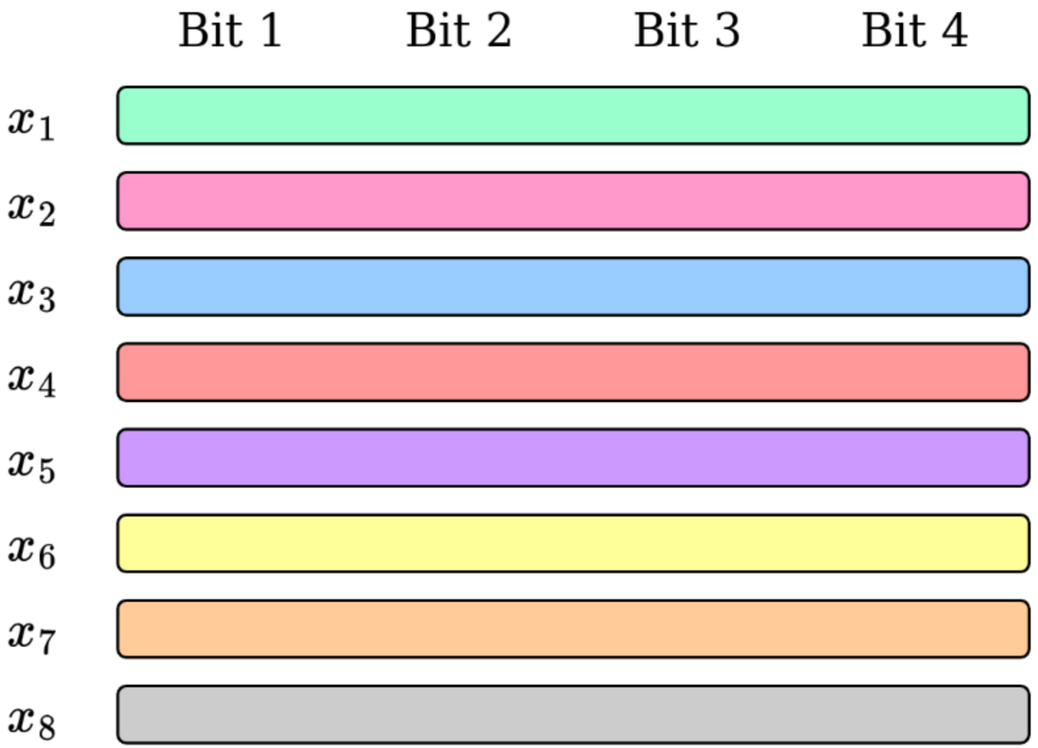
Iterative Quicksort



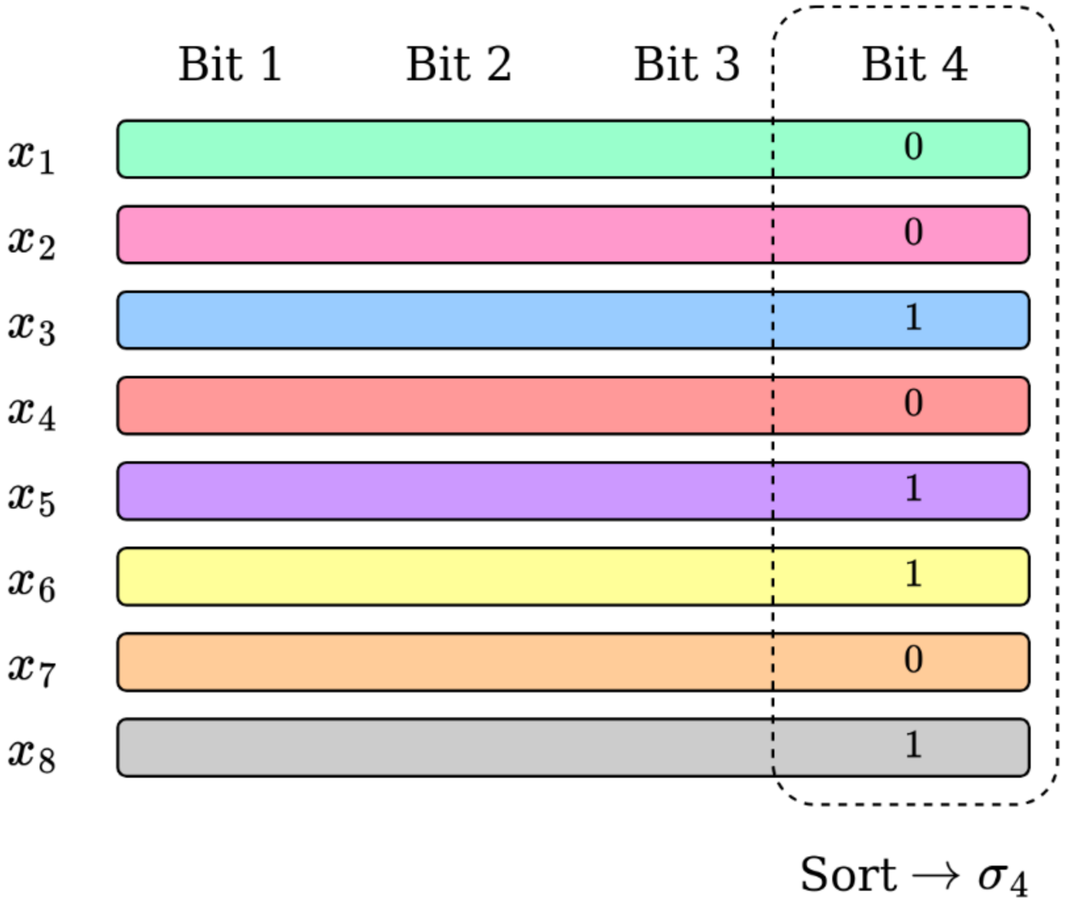
Iterative Quicksort



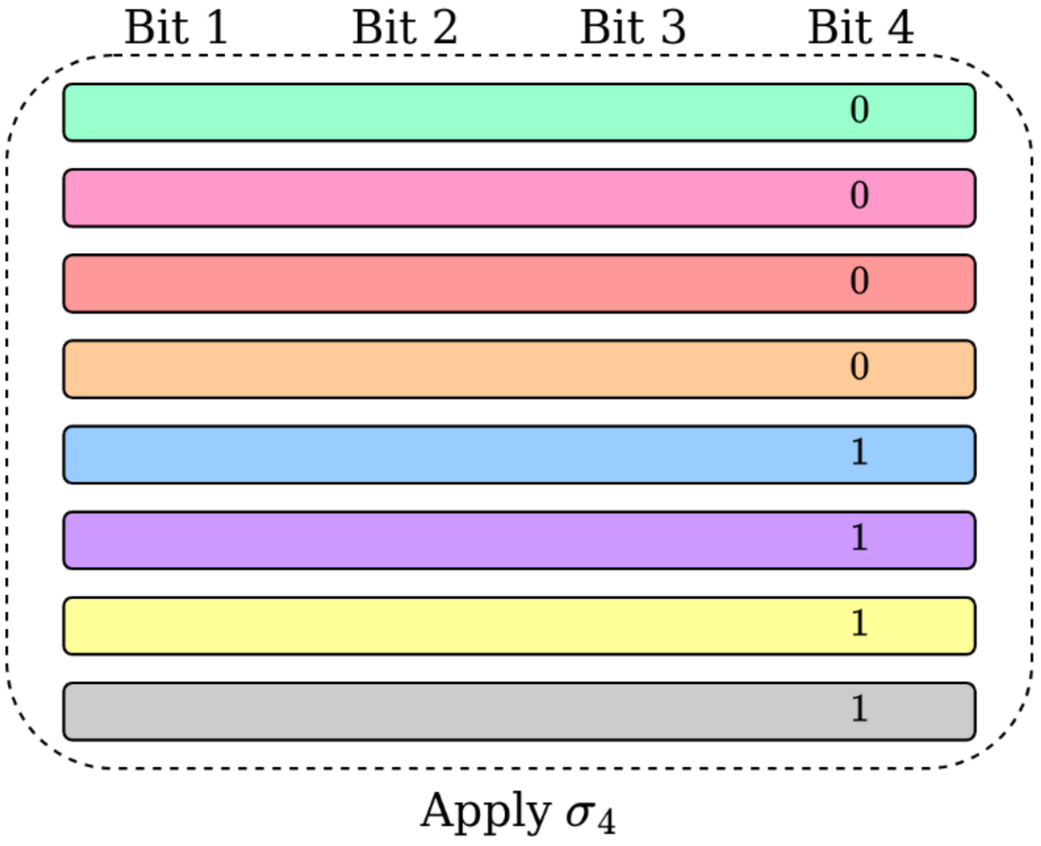
Radix Sort



Radix Sort

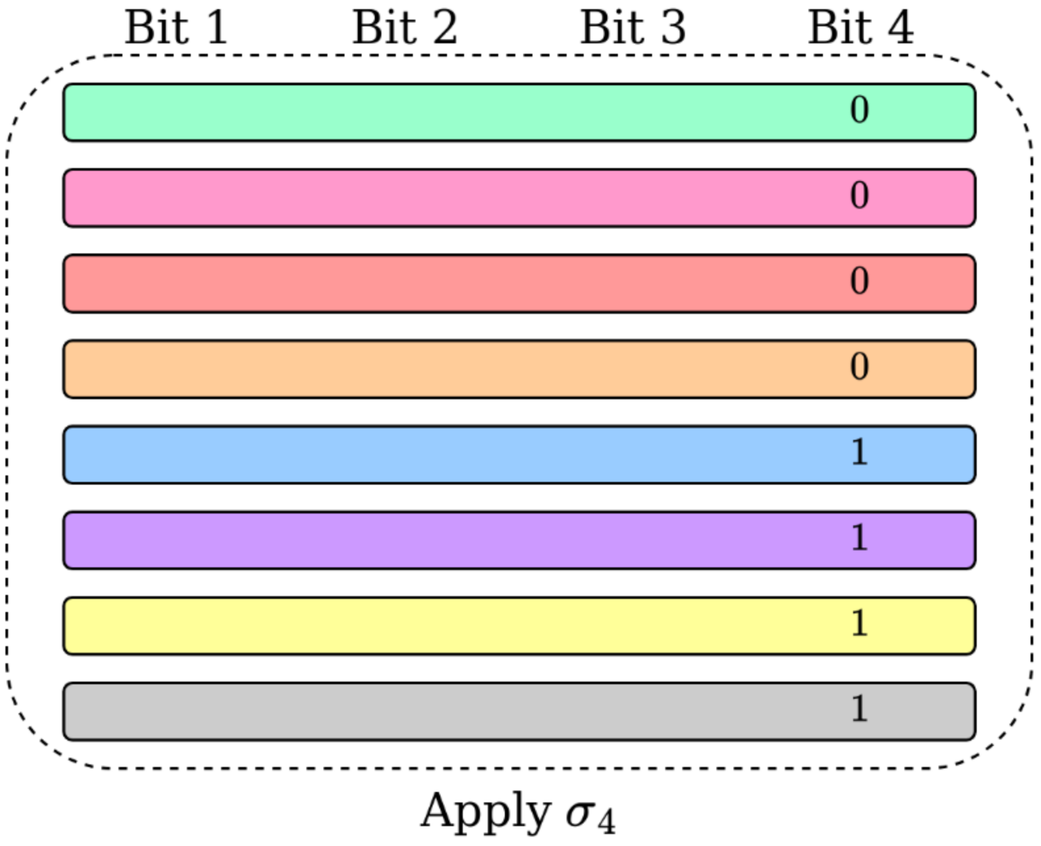

Bit 1
Bit 2
Bit 3
Bit 4
x_1
x_2
x_3
x_4
x_5
x_6
x_7
x_8
0
0
1
0
1
1
0
1
Sort $\rightarrow \sigma_4$

Radix Sort



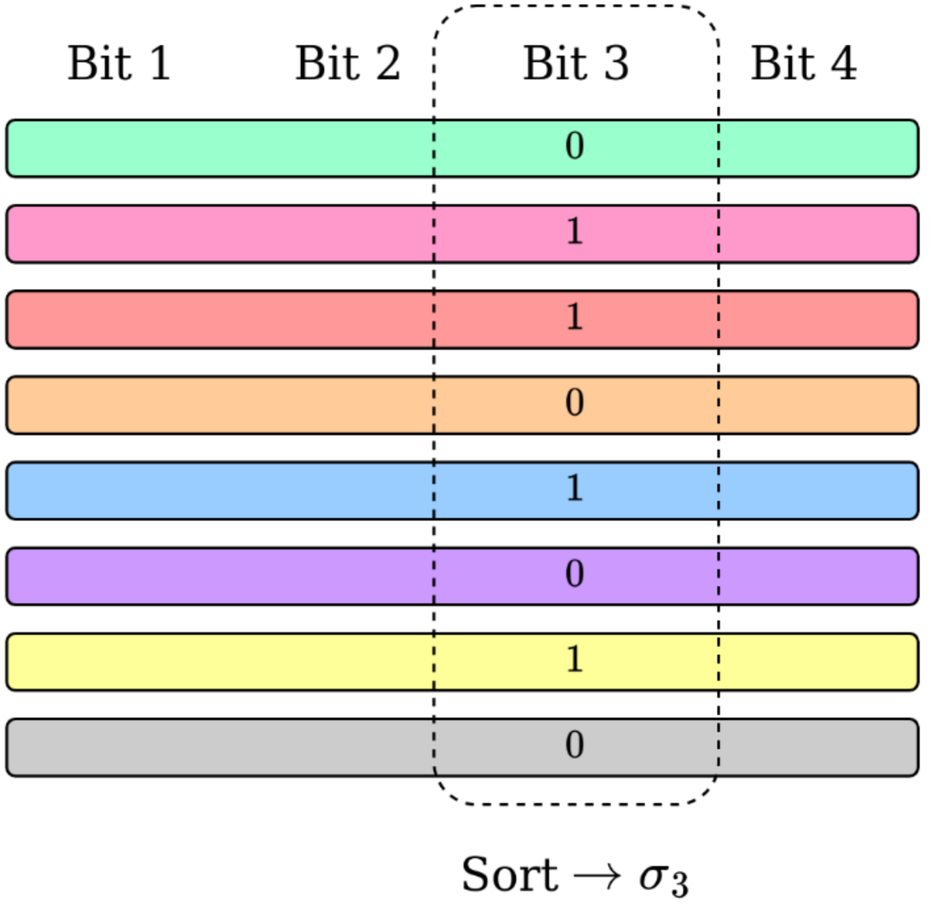
Radix Sort

Single bit sorting is $O(n)$



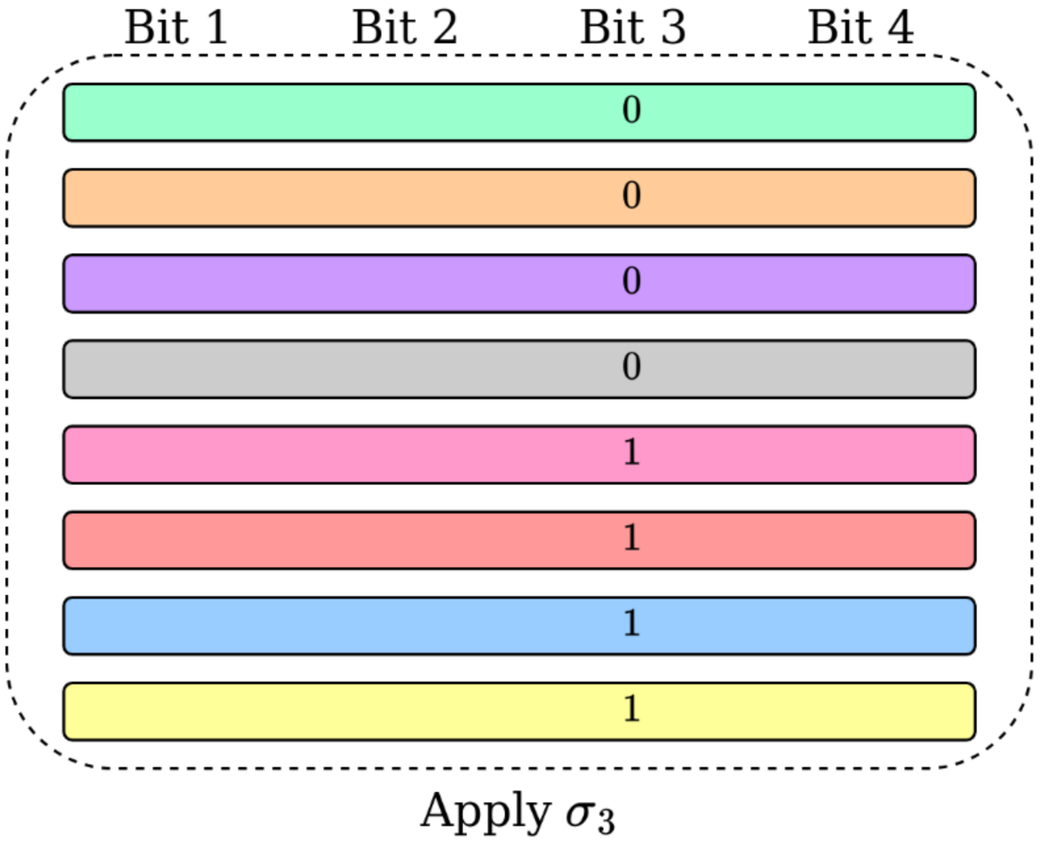
Radix Sort

Single bit sorting is $O(n)$



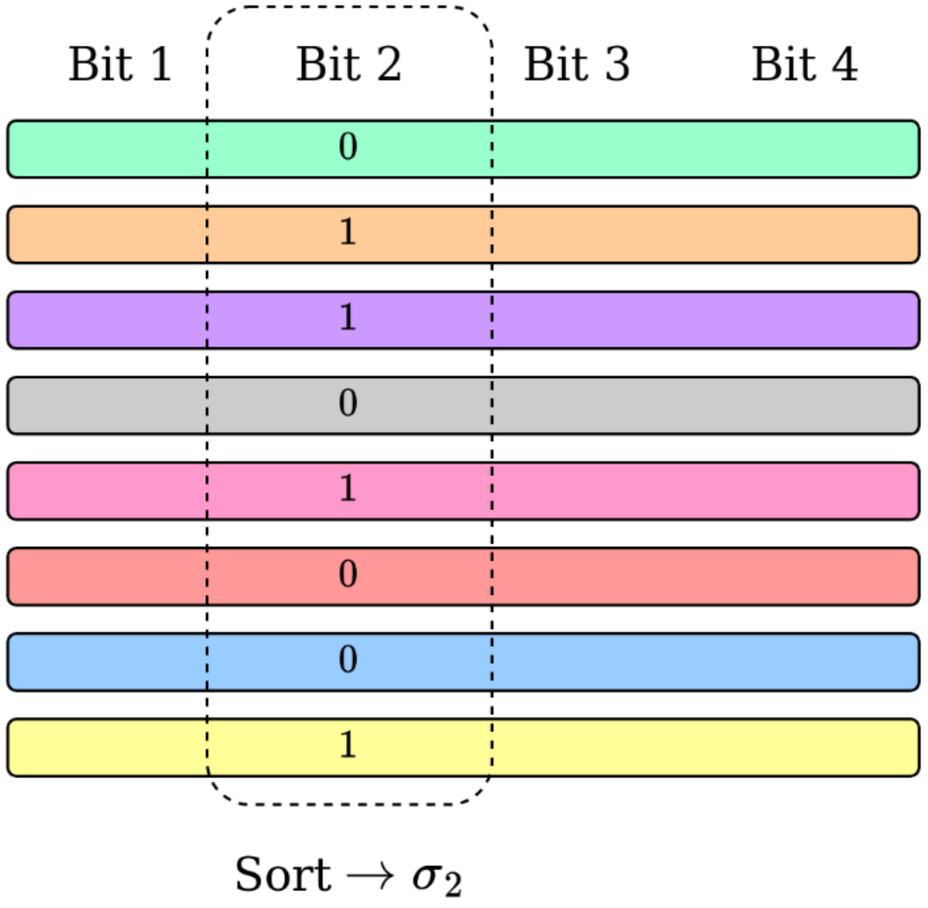
Radix Sort

Single bit sorting is $O(n)$



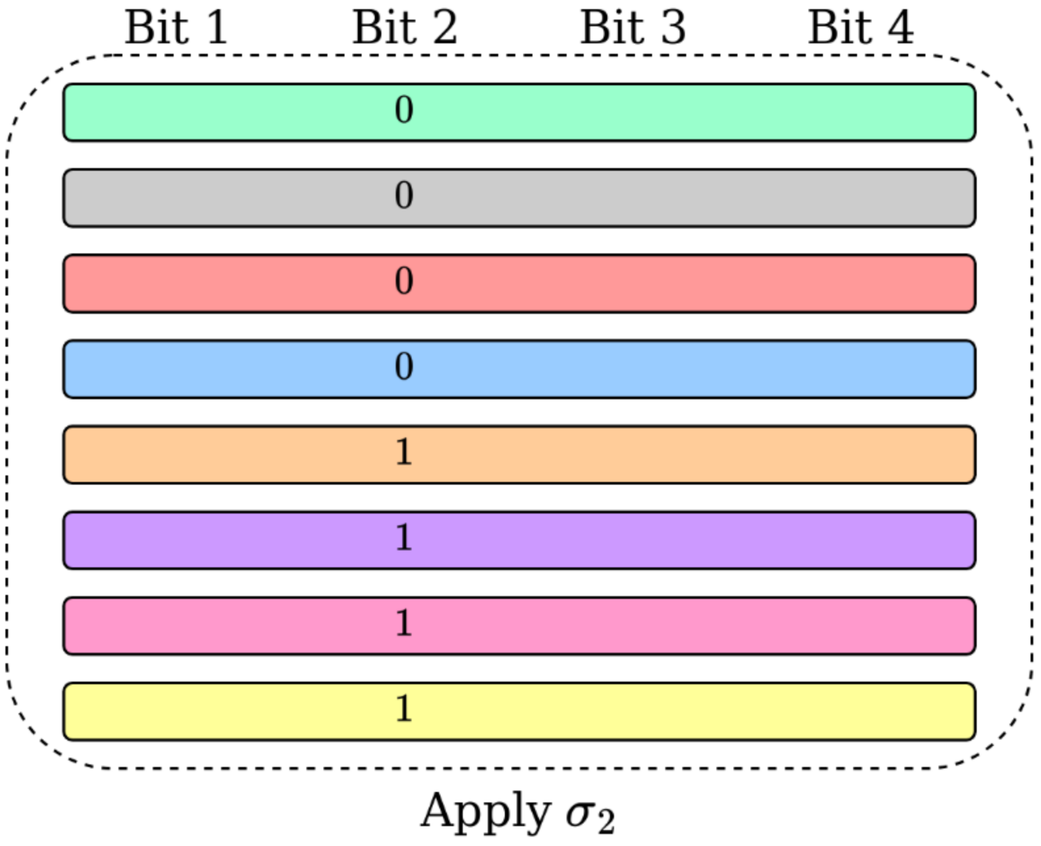
Radix Sort

Single bit sorting is $O(n)$



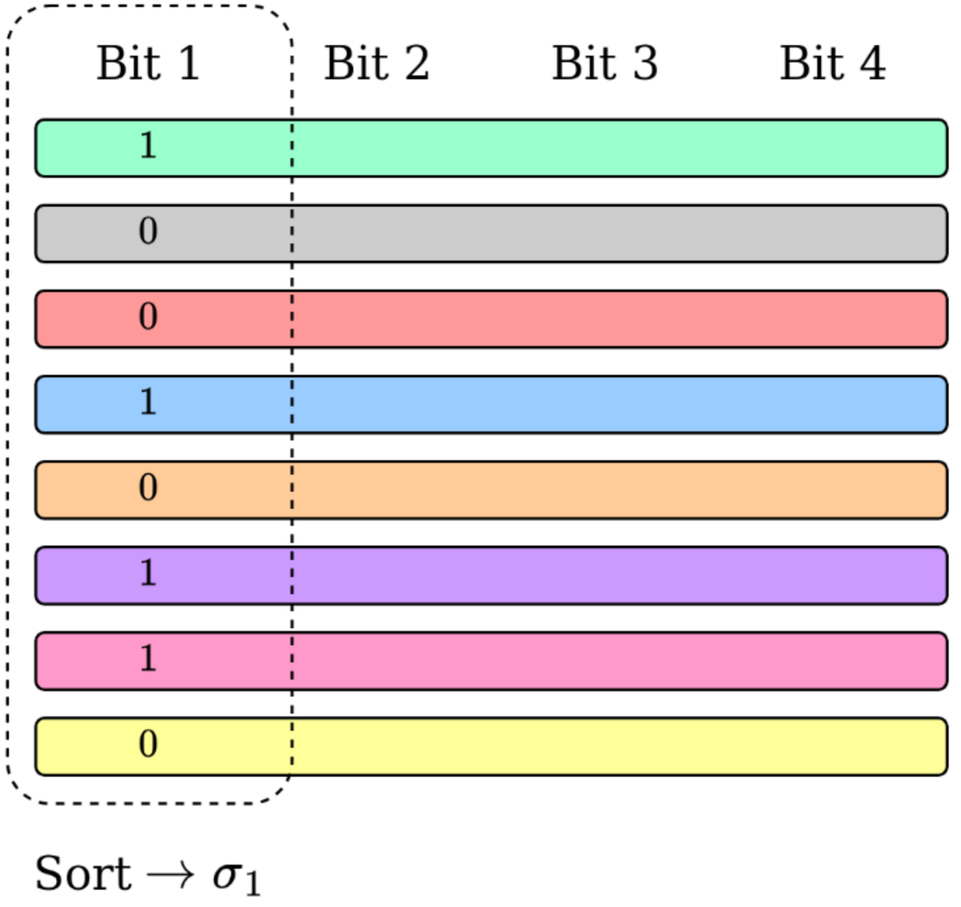
Radix Sort

Single bit sorting is $O(n)$



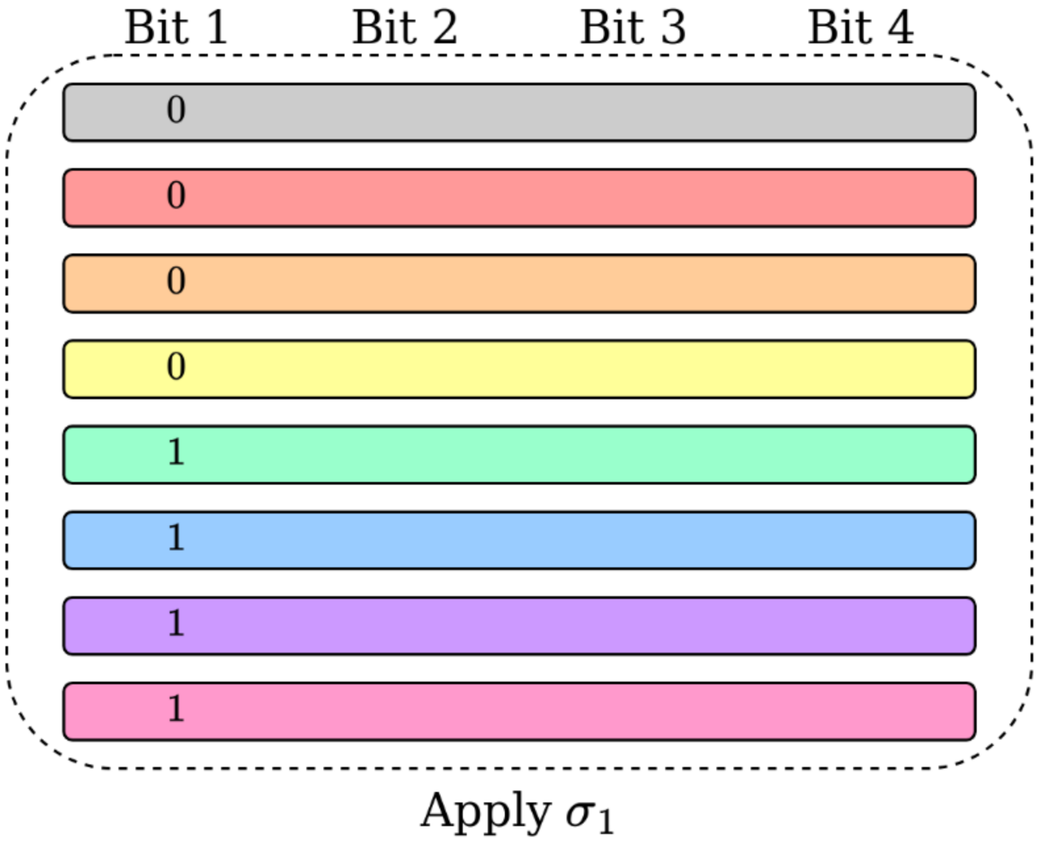
Radix Sort

Single bit sorting is $O(n)$



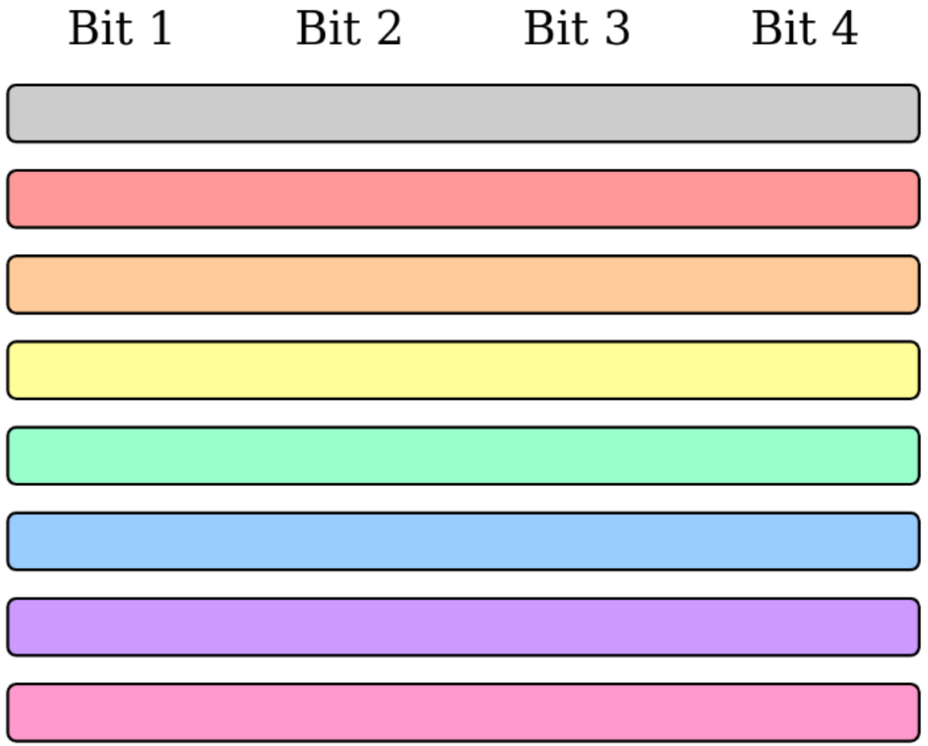
Radix Sort

Single bit sorting is $O(n)$



Radix Sort

Single bit sorting is $O(n)$



$$\sigma = \sigma_1 \circ \sigma_2 \circ \sigma_3 \circ \sigma_4$$

Analyzing Quicksort and Radixsort

Analyzing Quicksort and Radixsort

Quicksort: $O(n \log n)$ operations

Analyzing Quicksort and Radixsort

Quicksort: $O(n \log n)$ operations

Radix sort: $O(\ell n)$ operations

Analyzing Quicksort and Radixsort

Quicksort: $O(n \log n)$ operations

Radix sort: $O(\ell n)$ operations

Each operation has $O(\ell) = \Omega(\log n)$ communication.

Analyzing Quicksort and Radixsort

Quicksort: $O(n \log n)$ operations

Radix sort: $O(\ell n)$ operations

Each operation has $O(\ell) = \Omega(\log n)$ communication.

Algorithm	Communication
Quicksort	$O(\ell n \log n)$
Radix Sort	$O(\ell^2 n)$
Sorting Networks	$O(\ell n \log^2 n)$

Analyzing Quicksort and Radixsort

Quicksort: $O(n \log n)$ operations

Radix sort: $O(\ell n)$ operations

Each operation has $O(\ell) = \Omega(\log n)$ communication.

Algorithm	Communication
Quicksort	$O(\ell n \log n)$
Radix Sort	$O(\ell^2 n)$
Sorting Networks	$O(\ell n \log^2 n)$

Gold standard of oblivious sorting is $O(n \log^2 n)$ communication.

Extracting Sorting Permutations

Extracting Sorting Permutations

Two Challenges

Extracting Sorting Permutations

Two Challenges

- Ensuring stability

Extracting Sorting Permutations

Two Challenges

- Ensuring stability
- Extracting the applied permutation

Extracting Sorting Permutations

Two Challenges

- Ensuring stability
- Extracting the applied permutation

One Solution

Extracting Sorting Permutations

Two Challenges

- Ensuring stability
- Extracting the applied permutation

One Solution

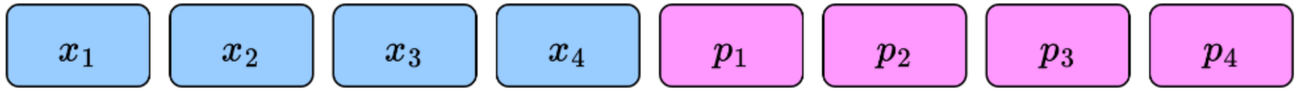
- Input padding

Extracting Sorting Permutations

Two Challenges

- Ensuring stability
- Extracting the applied permutation

One Solution



- Input padding

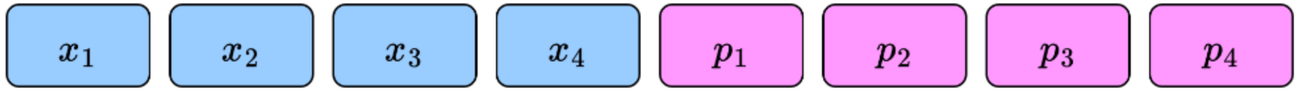
Extracting Sorting Permutations

Two Challenges

- Ensuring stability
- Extracting the applied permutation

One Solution

- Input padding



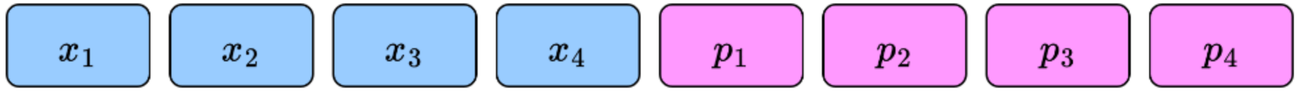
$$x_i \rightarrow x_i||i$$

Extracting Sorting Permutations

Two Challenges

- Ensuring stability
- Extracting the applied permutation

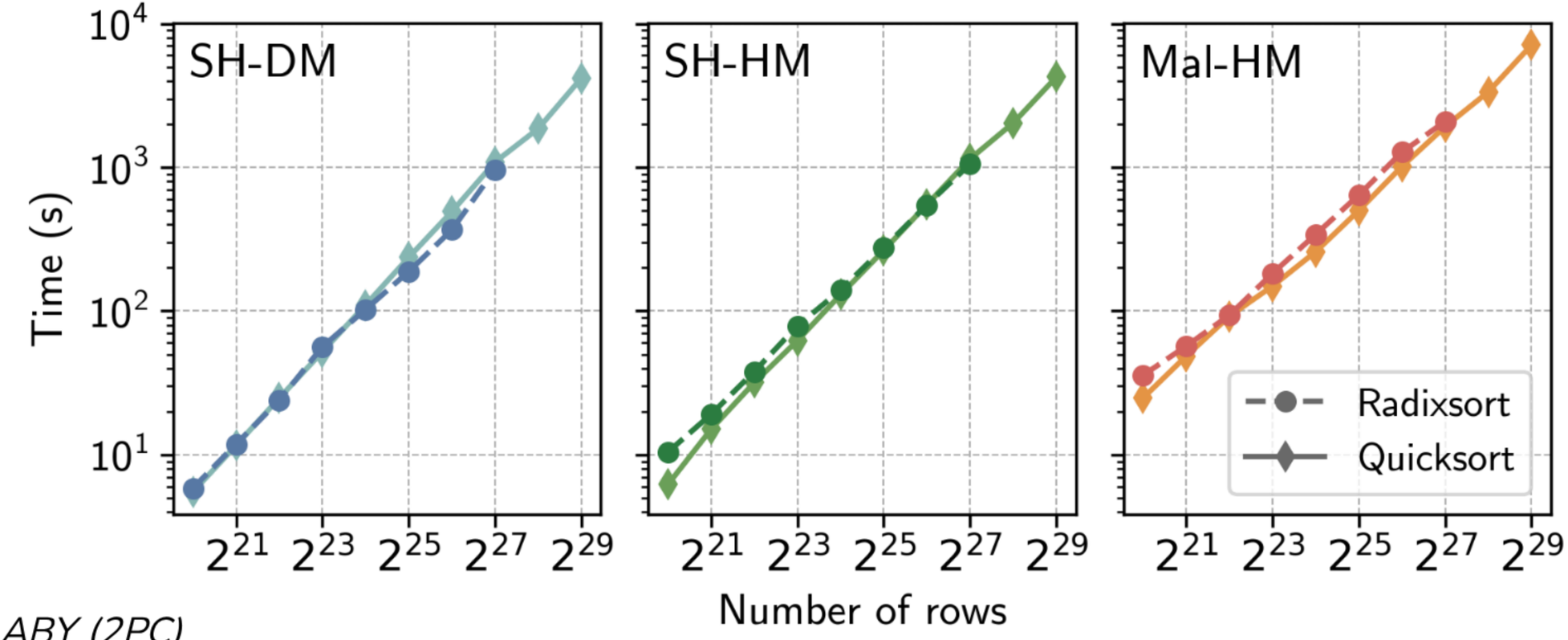
One Solution



- Input padding
- Padded bits represent the applied permutation

$$x_i \rightarrow x_i||i$$

Experimental Results



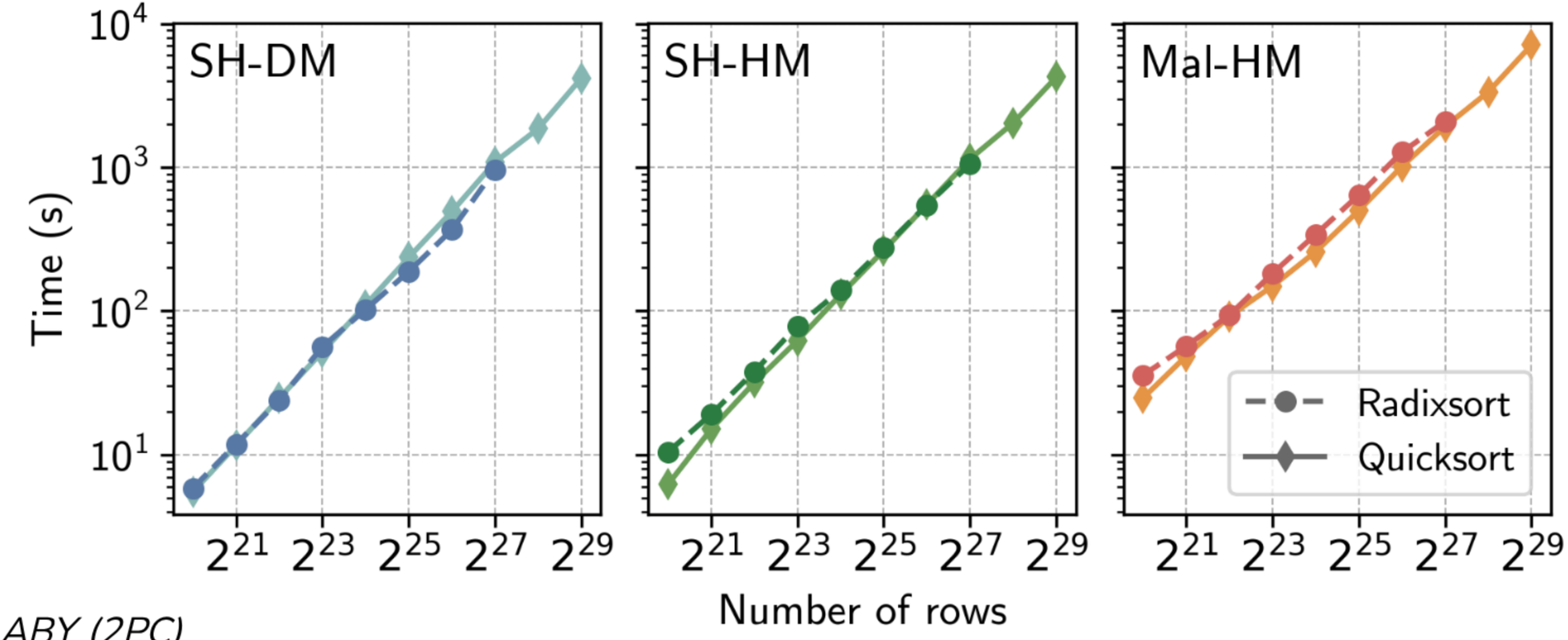
SH-DM: *ABY (2PC)*

SH-HM: *Araki et al. (3PC)*

Mal-HM: *Fantastic Four (4PC)*

Experimental Results

Up to 500m elements!

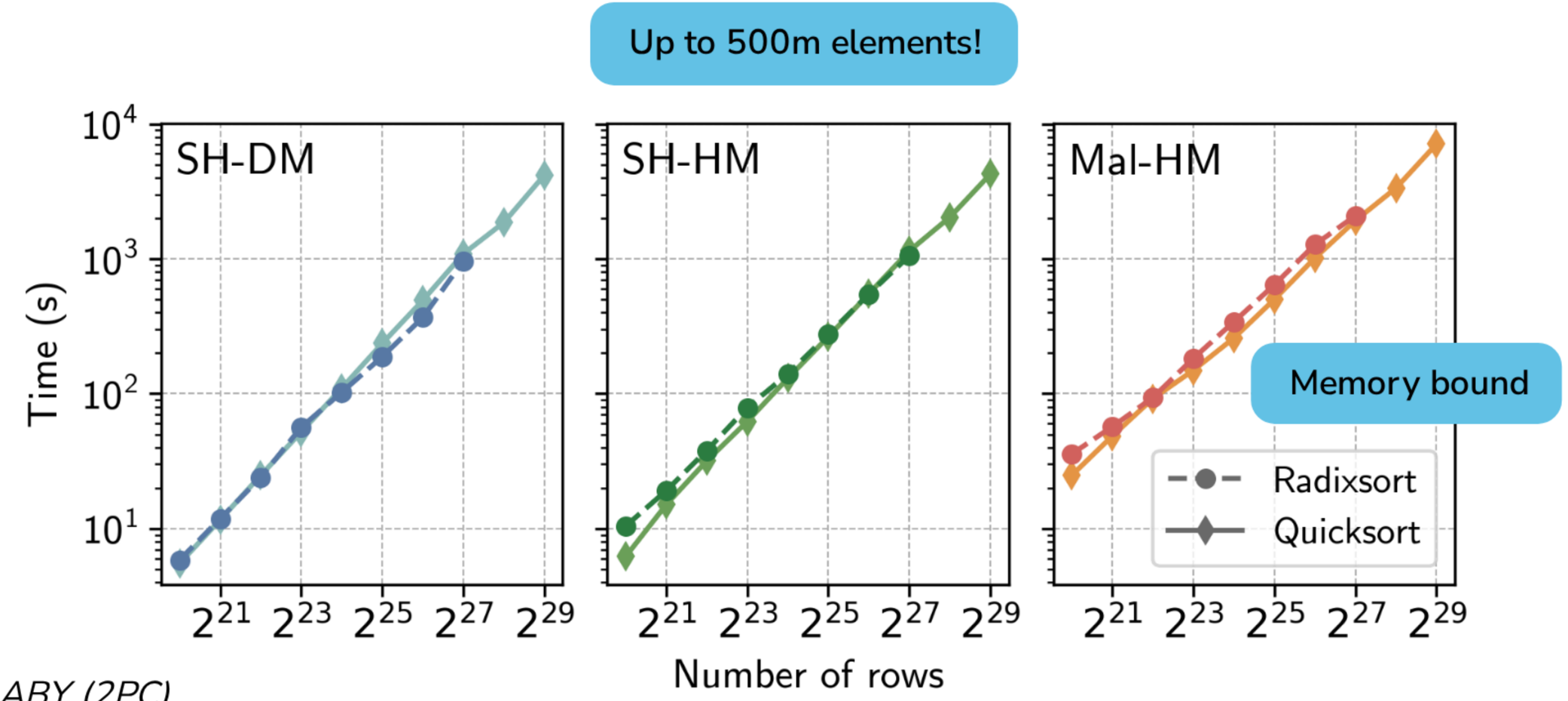


SH-DM: *ABY (2PC)*

SH-HM: *Araki et al. (3PC)*

Mal-HM: *Fantastic Four (4PC)*

Experimental Results

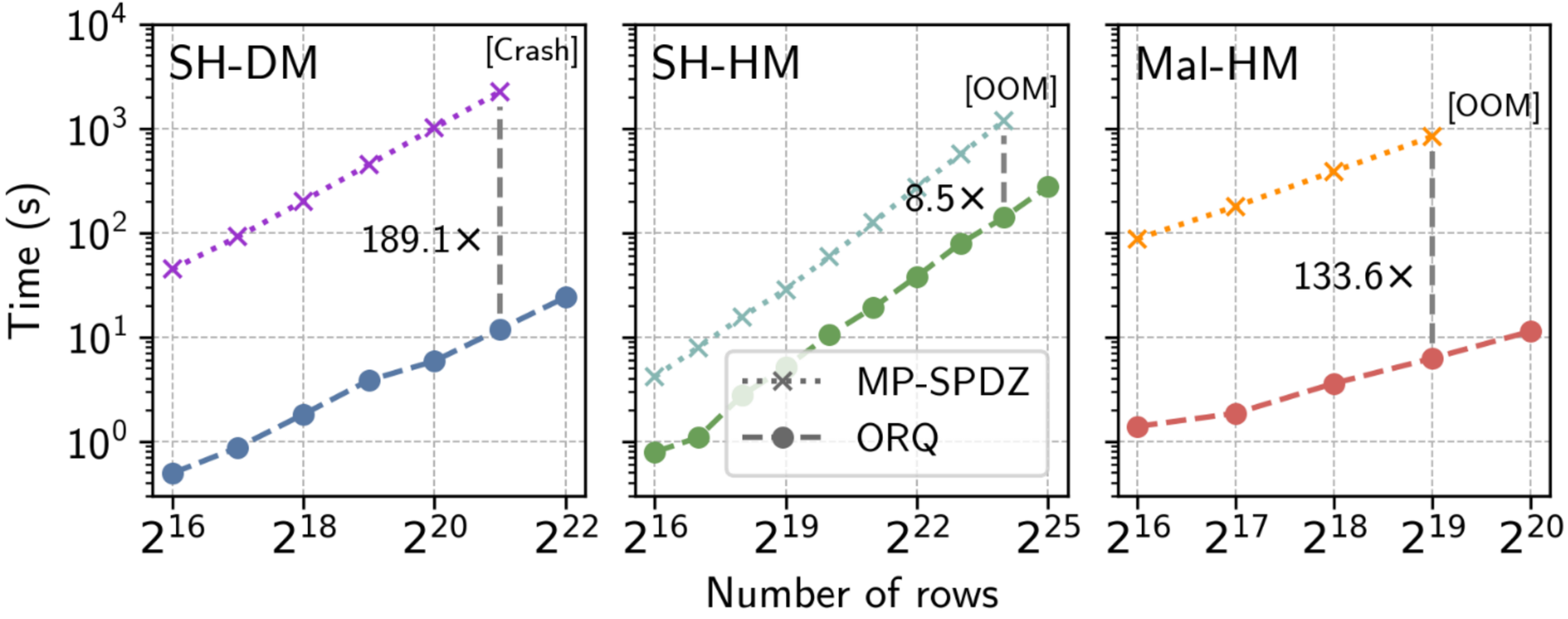


SH-DM: *ABY (2PC)*

SH-HM: *Araki et al. (3PC)*

Mal-HM: *Fantastic Four (4PC)*

Comparison with MP-SPDZ



SH-DM: *ABY (2PC)*

SH-HM: *Araki et al. (3PC)*

Mal-HM: *Fantastic Four (4PC)*

ORQ

Roadmap

~~1. Deployment of MPC for Political Polling~~

~~2. Oblivious Sorting~~

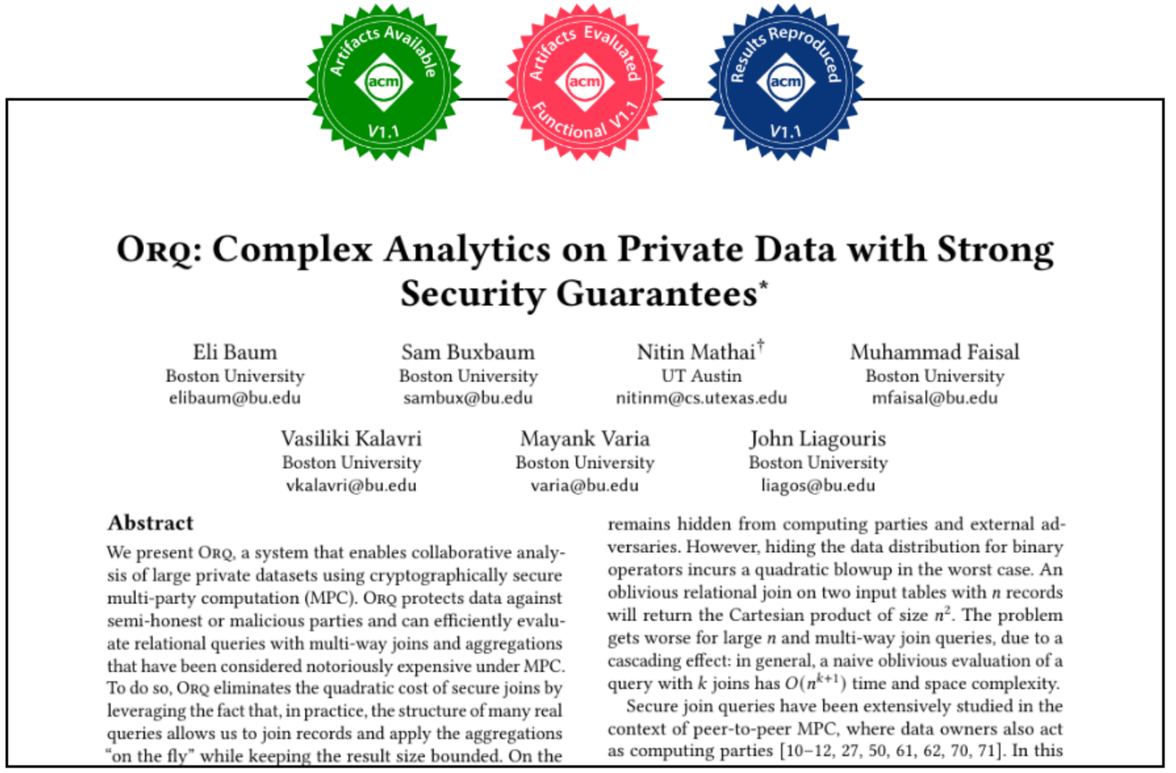
3. ORQ

ORQ: Complex Analytics on Private Data with Strong Security Guarantees

Distinguished Artifact Award at SOSP 25!

Paper

GitHub



ORQ: Complex Analytics on Private Data with Strong Security Guarantees*

Eli Baum
Boston University
elibaum@bu.edu

Sam Buxbaum
Boston University
sambux@bu.edu

Nitin Mathai†
UT Austin
nitinm@cs.utexas.edu

Muhammad Faisal
Boston University
mfaisal@bu.edu

Vasiliki Kalavri
Boston University
vkalavri@bu.edu

Mayank Varia
Boston University
varia@bu.edu

John Liagouris
Boston University
liagos@bu.edu

Abstract

We present ORQ, a system that enables collaborative analysis of large private datasets using cryptographically secure multi-party computation (MPC). ORQ protects data against semi-honest or malicious parties and can efficiently evaluate relational queries with multi-way joins and aggregations that have been considered notoriously expensive under MPC. To do so, ORQ eliminates the quadratic cost of secure joins by leveraging the fact that, in practice, the structure of many real queries allows us to join records and apply the aggregations "on the fly" while keeping the result size bounded. On the remains hidden from computing parties and external adversaries. However, hiding the data distribution for binary operators incurs a quadratic blowup in the worst case. An oblivious relational join on two input tables with n records will return the Cartesian product of size n^2. The problem gets worse for large n and multi-way join queries, due to a cascading effect: in general, a naive oblivious evaluation of a query with k joins has $O(n^{k+1})$ time and space complexity.

Secure join queries have been extensively studied in the context of peer-to-peer MPC, where data owners also act as computing parties [10–12, 27, 50, 61, 62, 70, 71]. In this

Oblivious Analytics

Oblivious Analytics

Secure computation must be *oblivious*.

Oblivious Analytics

Secure computation must be *oblivious*.

Oblivious computation must maintain *worst-case* output sizes.

Oblivious Analytics

Secure computation must be *oblivious*.

Oblivious computation must maintain *worst-case* output sizes.

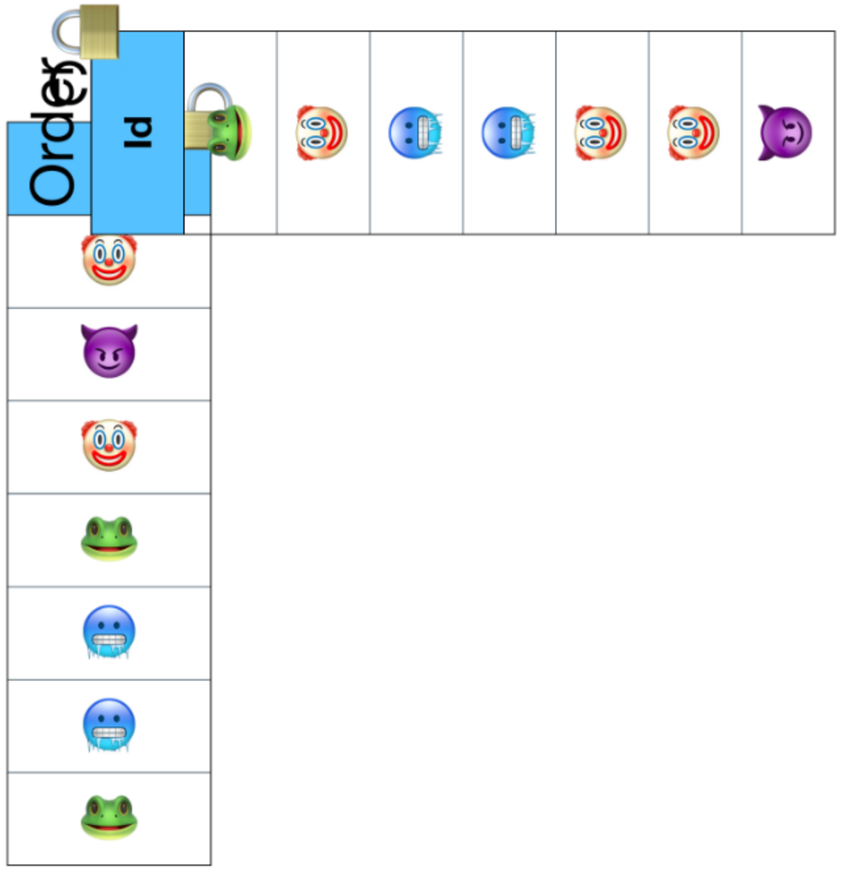
Customer	AcctBalance
🤡	$1000
😈	$2000
🤡	$500
🐸	$60
🥶	$8000
🥶	$700
🐸	-$200

`filter(Customer ≠ 🐸)`

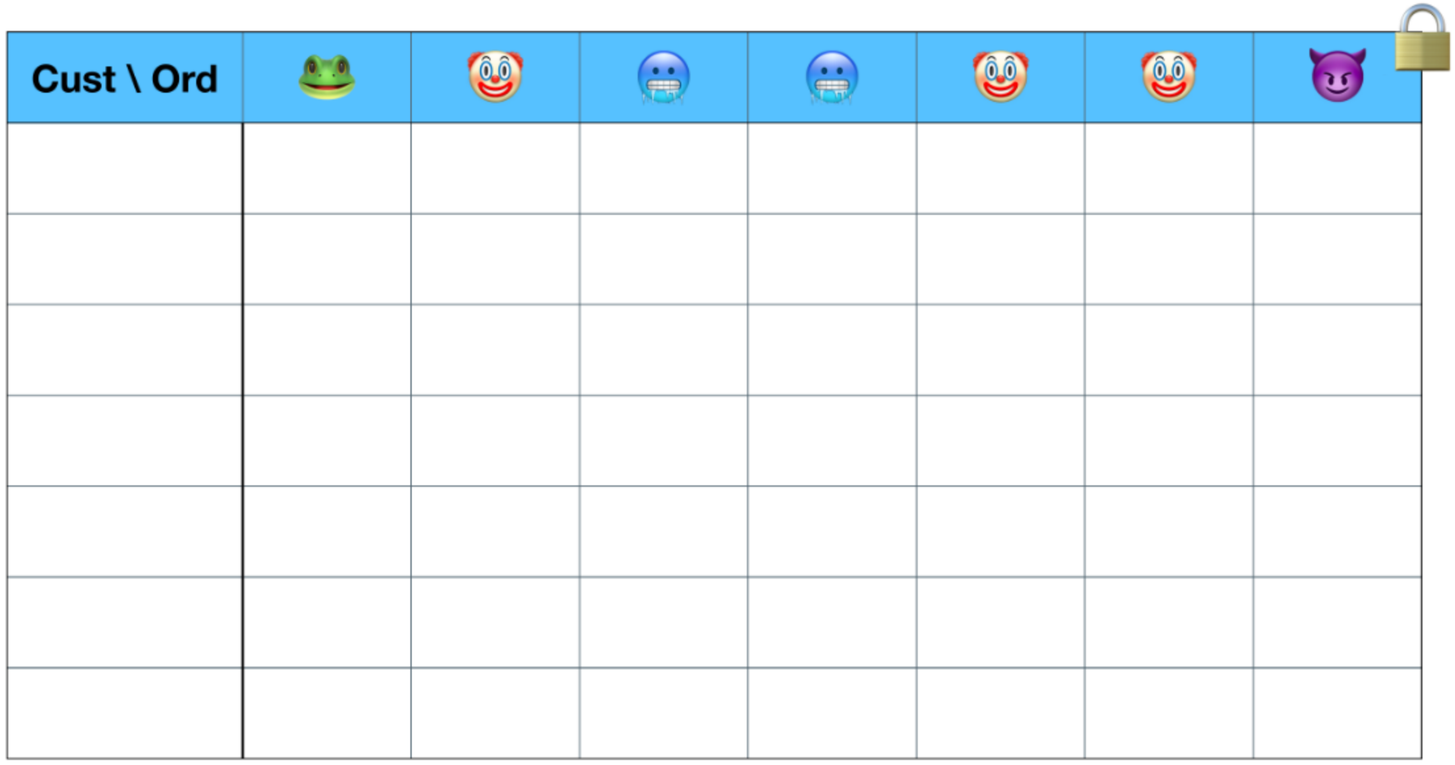
Customer	AcctBalance
▓▓▓	▓▓▓
▓▓▓	▓▓▓
▓▓▓	▓▓▓
▓▓▓	▓▓▓
▓▓▓	▓▓▓
▓▓▓	▓▓▓
▓▓▓	▓▓▓

Oblivious Joins

Oblivious Joins



Oblivious Joins



Oblivious Joins

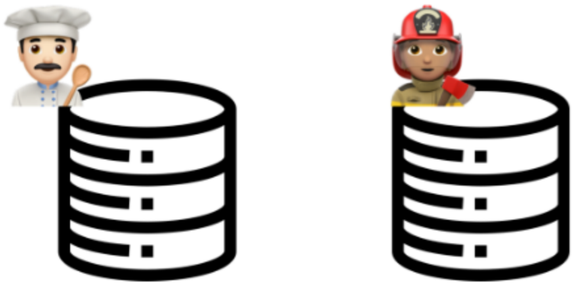
Cust \ Ord	🐸	🤡	🥶	🥶	🤡	🤡	😈 🔒
🤡	❌	✅	❌	❌	✅	✅	❌
😈	❌	❌	❌	❌	❌	❌	✅
🤡	❌	✅	❌	❌	✅	✅	❌
🐸	✅	❌	❌	❌	❌	❌	❌
🥶	❌	❌	✅	✅	❌	❌	❌
🥶	❌	❌	✅	✅	❌	❌	❌
🐸	✅	❌	❌	❌	❌	❌	❌

Oblivious Joins

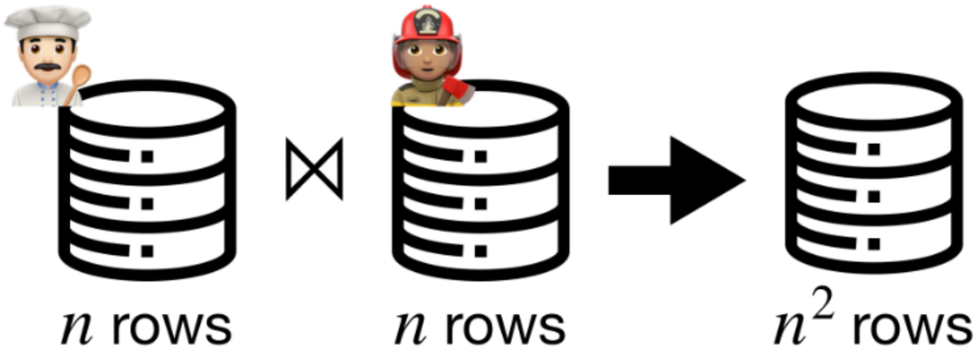
Cust \ Ord	🐸	🤡	🥶	🥶	🤡	🤡	😈 🔒
🤡	❌	✅	❌	❌	✅	✅	❌
😈	❌	❌	❌	❌	❌	❌	✅
🤡	❌	✅	❌	❌	✅	✅	❌
🐸	✅	❌	❌	❌	❌	❌	❌
🥶	❌	❌	✅	✅	❌	❌	❌
🥶	❌	❌	✅	✅	❌	❌	❌
🐸	✅	❌	❌	❌	❌	❌	❌

$O(n^2)$ time and space

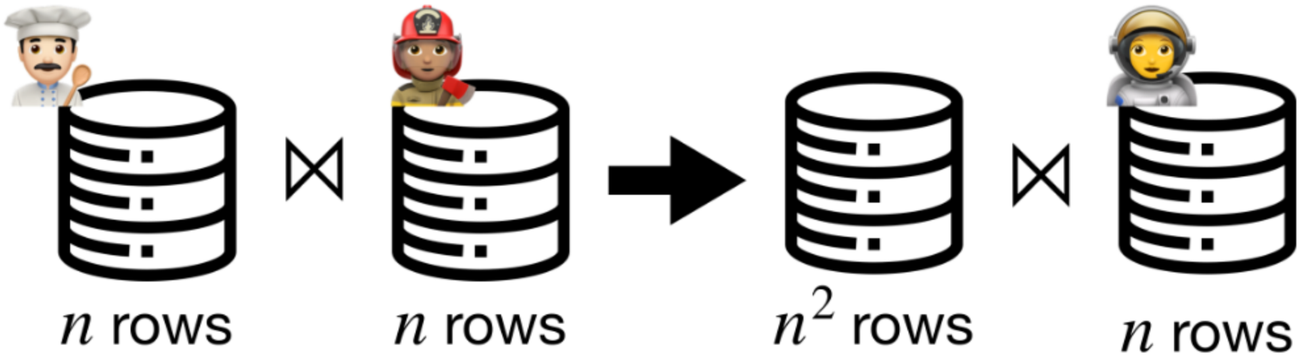
The Problem



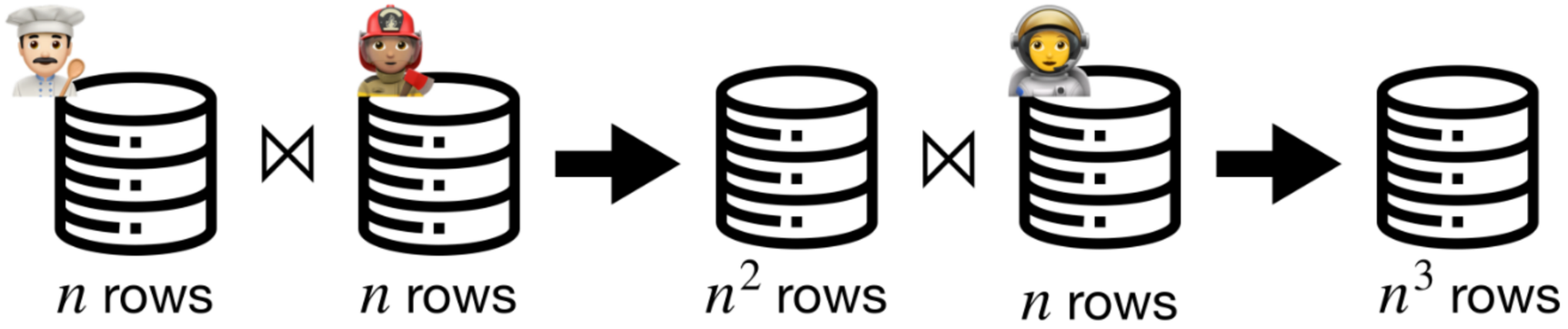
The Problem



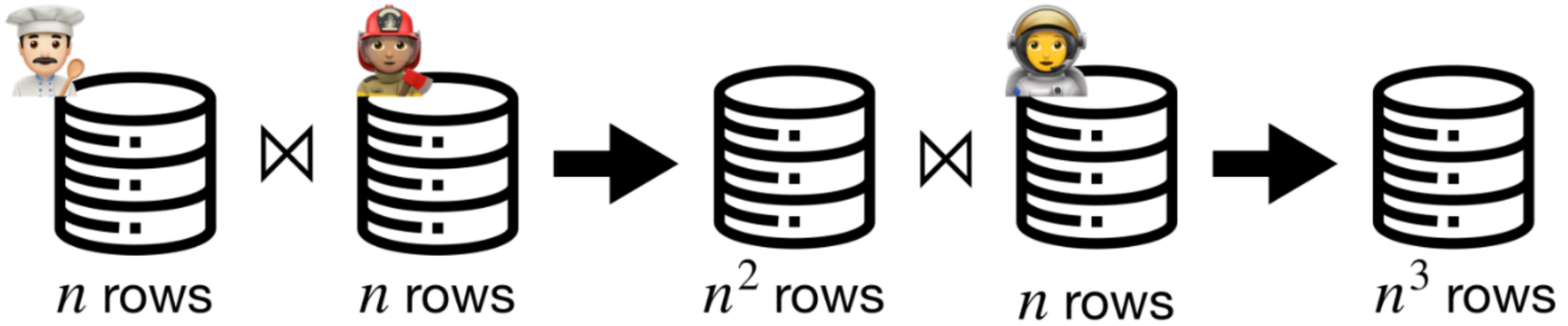
The Problem

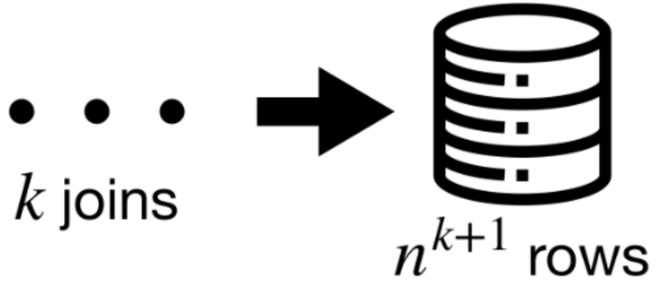


The Problem



The Problem


n rows
$\bowtie$
n rows
n^2 rows
$\bowtie$
n rows
n^3 rows


k joins
n^{k+1} rows

Our Insight

Our Insight

The general problem of oblivious joins is impractically difficult.

Our Insight

The general problem of oblivious joins is impractically difficult.

Practical instances are not!

Our Insight

The general problem of oblivious joins is impractically difficult.

Practical instances are not!

Common queries impose an $O(n)$ bound on the output size.

Our Insight

The general problem of oblivious joins is impractically difficult.

Practical instances are not!

Common queries impose an $O(n)$ bound on the output size.

Joins with incremental aggregations can avoid the quadratic blowup.

Our Insight

The general problem of oblivious joins is impractically difficult.

Practical instances are not!

Common queries impose an $O(n)$ bound on the output size.

Joins with incremental aggregations can avoid the quadratic blowup.

$$f(X, Y) = f(f(X), f(Y))$$

Join-Aggregation Operator

Join-Aggregation Operator

Starting point: one-to-many joins

Join-Aggregation Operator

Starting point: one-to-many joins

```
func join(L, R, keys):
   T := concat*(L, R)
   T.sort(keys)
   T.prefix_aggregate(…)
   return T
```

Join-Aggregation Operator

Starting point: one-to-many joins

```
func join(L, R, keys):
   T := concat*(L, R)
   T.sort(keys)
   T.prefix_aggregate(…)
   return T
```

Reduces to sorting and aggregation!

Supporting Many-to-Many Joins

Supporting Many-to-Many Joins

We can transform a many-to-many join into a one-to-many join.

Supporting Many-to-Many Joins

We can transform a many-to-many join into a one-to-many join.

Pre-aggregate one of the tables and mark duplicate keys as invalid.

Supporting Many-to-Many Joins

We can transform a many-to-many join into a one-to-many join.

Pre-aggregate one of the tables and mark duplicate keys as invalid.

Customer	C.Total	O.Cost
∅	∅	
🤡	$1500	
😈	$2000	
∅	∅	
🐸	-$140	
∅	∅	
🥶	$8700	
🐸		$30
🤡		$20
🥶		$55
🥶		$18
🤡		$70
🤡		$12
😈		$8

Supporting Many-to-Many Joins

We can transform a many-to-many join into a one-to-many join.

Pre-aggregate one of the tables and mark duplicate keys as invalid.

- The aggregation must be decomposable

Customer	C.Total	O.Cost
∅	∅	
🤡	$1500	
😈	$2000	
∅	∅	
🐸	-$140	
∅	∅	
🥶	$8700	
🐸		$30
🤡		$20
🥶		$55
🥶		$18
🤡		$70
🤡		$12
😈		$8

Supporting Many-to-Many Joins

We can transform a many-to-many join into a one-to-many join.

Pre-aggregate one of the tables and mark duplicate keys as invalid.

- The aggregation must be decomposable
- All keys must exist in one of the tables

Customer	C.Total	O.Cost
∅	∅	
🤡	$1500	
😈	$2000	
∅	∅	
🐸	-$140	
∅	∅	
🥶	$8700	
🐸		$30
🤡		$20
🥶		$55
🥶		$18
🤡		$70
🤡		$12
😈		$8

Supporting Many-to-Many Joins

We can transform a many-to-many join into a one-to-many join.

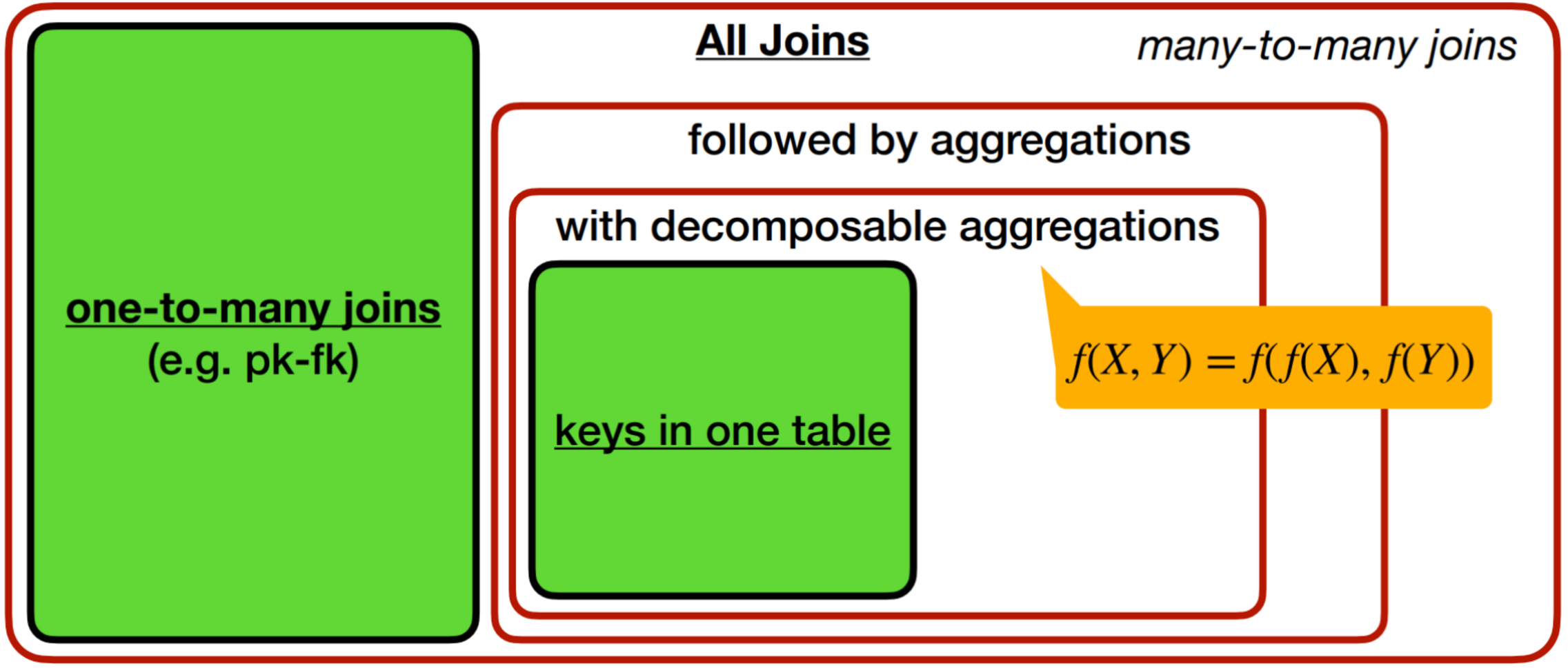
Pre-aggregate one of the tables and mark duplicate keys as invalid.

- The aggregation must be decomposable
- All keys must exist in one of the tables

These constraints hold for all 31 queries from prior work.

Customer	C.Total	O.Cost
∅	∅	
🤡	$1500	
😈	$2000	
∅	∅	
🐸	-$140	
∅	∅	
🥶	$8700	
🐸		$30
🤡		$20
🥶		$55
🥶		$18
🤡		$70
🤡		$12
😈		$8

What Can We Support?



Connection to Oblivious Sorting

Connection to Oblivious Sorting

Queries consist of joins, joins consist of (many) sorts.

Connection to Oblivious Sorting

Queries consist of joins, joins consist of (many) sorts.

- 12 sorts for TPC-H Q21

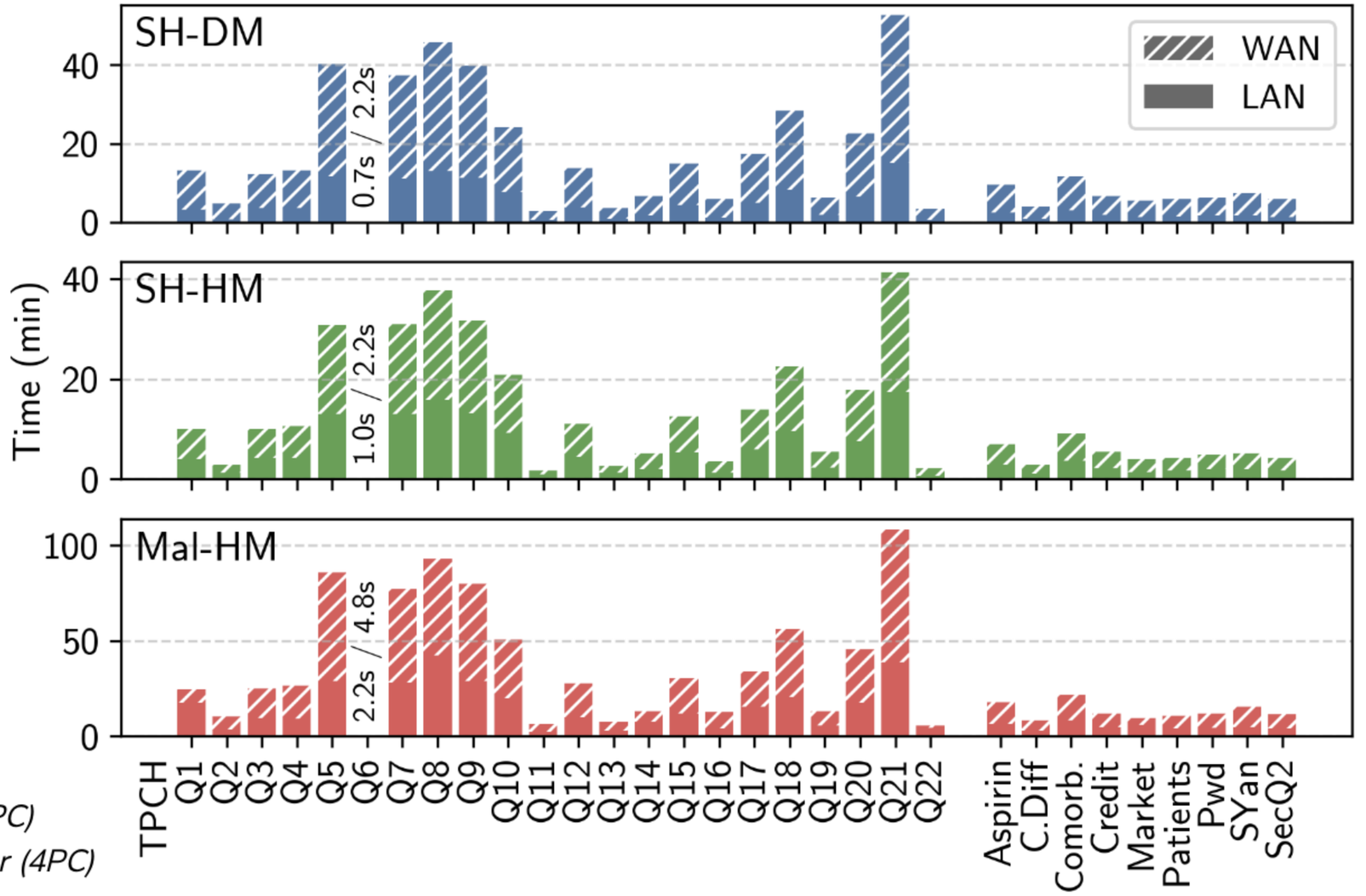
Connection to Oblivious Sorting

Queries consist of joins, joins consist of (many) sorts.

- 12 sorts for TPC-H Q21

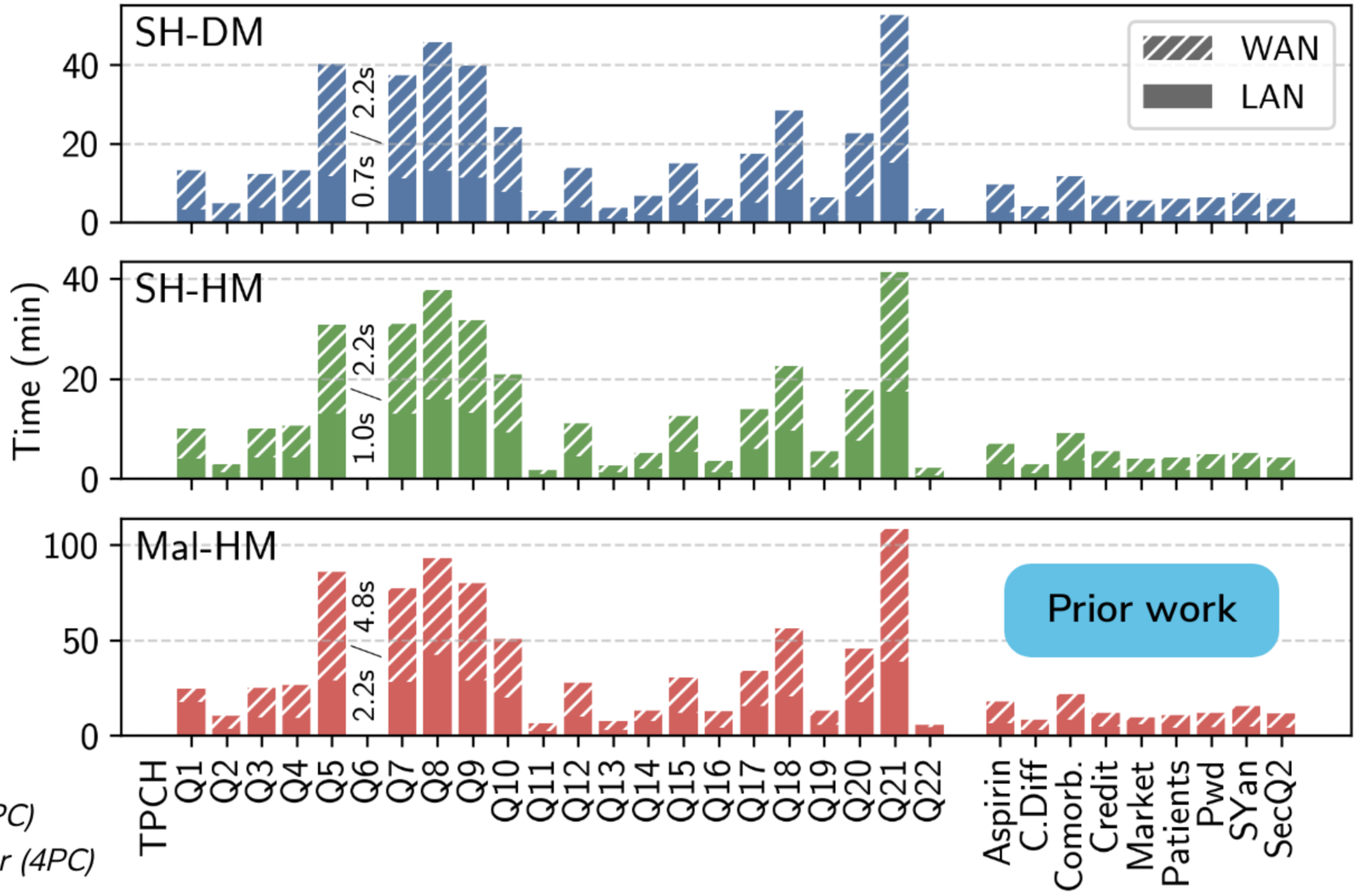
Sorting is no longer the main bottleneck!

Query Benchmarks



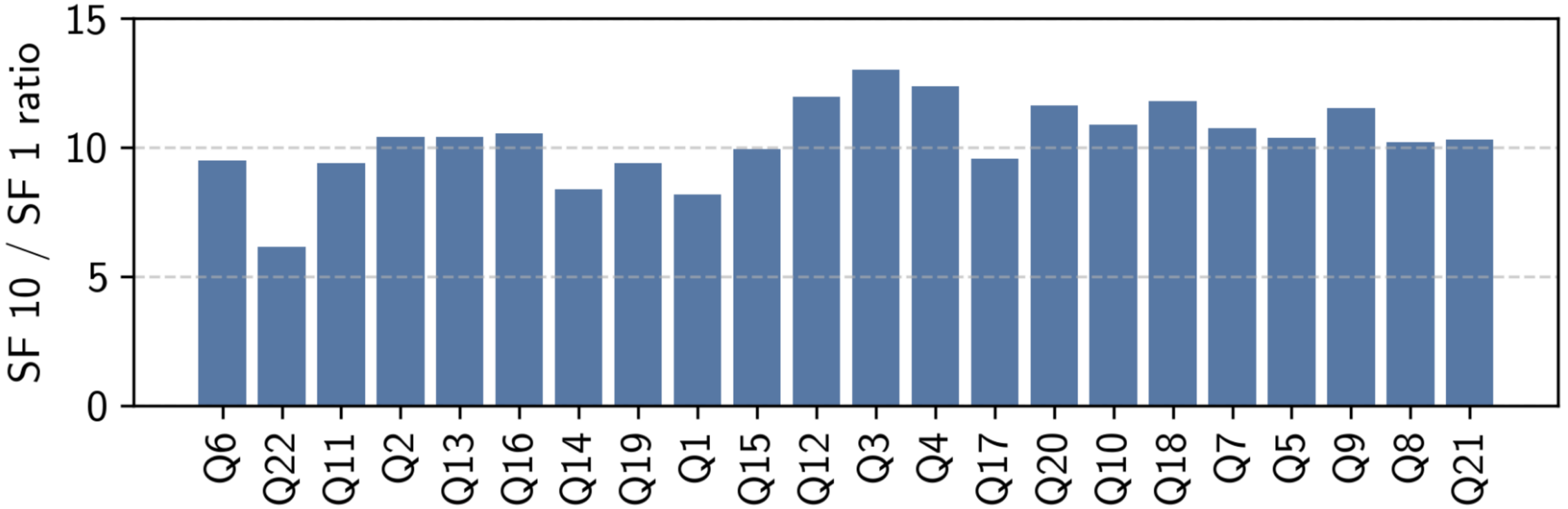
SH-DM: *ABY (2PC)*
SH-HM: *Araki et al. (3PC)*
Mal-HM: *Fantastic Four (4PC)*

Query Benchmarks



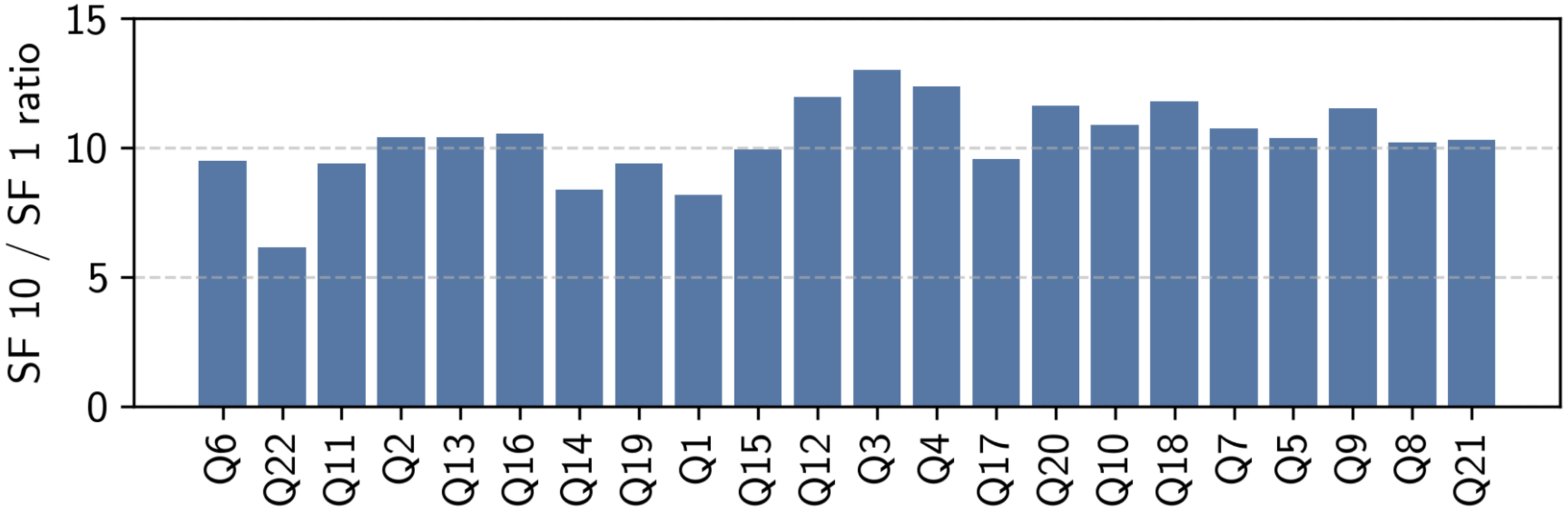
SH-DM: *ABY (2PC)*
SH-HM: *Araki et al. (3PC)*
Mal-HM: *Fantastic Four (4PC)*

Scalability



Scalability

Theoretical overhead $\approx 11\times$



The ORQ System

The ORQ System

- Familiar API

```
auto C = db.getCustomersTable();
auto O = db.getOrdersTable();

O.filter(O["[Comment]"] ≠ 0);

auto T = C.left_outer_join(O,
    {"[CustKey]"},
    {{"Count", "Count", count}});

T.convert_a2b("Count", “[Count]");

auto F = T.aggregate(
    {"[Count]"},
    {{"CustDist", "CustDist", count}});

F.convert_a2b("CustDist", "[CustDist]");
F.sort({"[CustDist]", "[Count]"}, DESC);
```

The ORQ System

- Familiar API
- Protocol-agnostic interface

```
auto C = db.getCustomersTable();
auto O = db.getOrdersTable();

O.filter(O["[Comment]"] ≠ 0);

auto T = C.left_outer_join(O,
    {"[CustKey]"},
    {{"Count", "Count", count}});

T.convert_a2b("Count", “[Count]");

auto F = T.aggregate(
    {"[Count]"},
    {{"CustDist", "CustDist", count}});

F.convert_a2b("CustDist", "[CustDist]");
F.sort({"[CustDist]", "[Count]"}, DESC);
```

The ORQ System

- Familiar API
- Protocol-agnostic interface
  - Support for 2PC, 3PC, 4PC

```
auto C = db.getCustomersTable();
auto O = db.getOrdersTable();

O.filter(O["[Comment]"] ≠ 0);

auto T = C.left_outer_join(O,
    {"[CustKey]"},
    {{"Count", "Count", count}});

T.convert_a2b("Count", “[Count]");

auto F = T.aggregate(
    {"[Count]"},
    {{"CustDist", "CustDist", count}});

F.convert_a2b("CustDist", "[CustDist]");
F.sort({"[CustDist]", "[Count]"}, DESC);
```

The ORQ System

- Familiar API
- Protocol-agnostic interface
  - Support for 2PC, 3PC, 4PC
- Data-parallel runtime

```
auto C = db.getCustomersTable();
auto O = db.getOrdersTable();

O.filter(O["[Comment]"] ≠ 0);

auto T = C.left_outer_join(O,
    {"[CustKey]"},
    {{"Count", "Count", count}});

T.convert_a2b("Count", “[Count]");

auto F = T.aggregate(
    {"[Count]"},
    {{"CustDist", "CustDist", count}});

F.convert_a2b("CustDist", "[CustDist]");
F.sort({"[CustDist]", "[Count]"}, DESC);
```

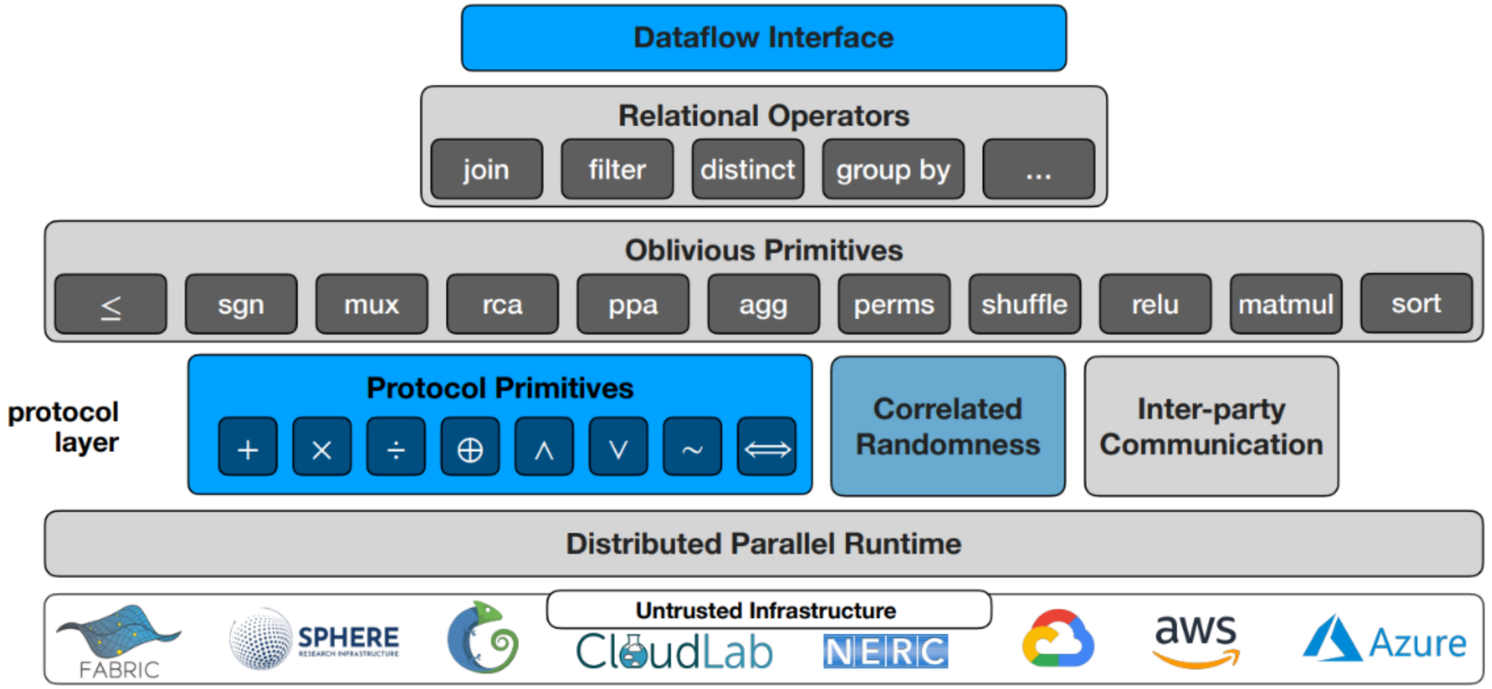
The ORQ System

- Familiar API
- Protocol-agnostic interface
  - Support for 2PC, 3PC, 4PC
- Data-parallel runtime
- Custom TCP Communicator

```
auto C = db.getCustomersTable();
auto O = db.getOrdersTable();

O.filter(O["[Comment]"] ≠ 0);

auto T = C.left_outer_join(O,
    {"[CustKey]"},
    {{"Count", "Count", count}});

T.convert_a2b("Count", “[Count]");

auto F = T.aggregate(
    {"[Count]"},
    {{"CustDist", "CustDist", count}});

F.convert_a2b("CustDist", "[CustDist]");
F.sort({"[CustDist]", "[Count]"}, DESC);
```

The ORQ System

- Familiar API
- Protocol-agnostic interface
  - Support for 2PC, 3PC, 4PC
- Data-parallel runtime
- Custom TCP Communicator



ORQ Summary

ORQ Summary

- A system for relational analytics under MPC

ORQ Summary

- A system for relational analytics under MPC
- Novel join-aggregation operator for oblivious many-to-many joins

ORQ Summary

- A system for relational analytics under MPC
- Novel join-aggregation operator for oblivious many-to-many joins
- State-of-the-art oblivious sorting and shuffling modules

ORQ Summary

- A system for relational analytics under MPC
- Novel join-aggregation operator for oblivious many-to-many joins
- State-of-the-art oblivious sorting and shuffling modules
- Entirely open source

ORQ Summary

- A system for relational analytics under MPC
- Novel join-aggregation operator for oblivious many-to-many joins
- State-of-the-art oblivious sorting and shuffling modules
- Entirely open source
- Under active development

Future Work

Future Work

- Machine learning

Future Work

- Machine learning
- Graph algorithms

Future Work

- Machine learning
- Graph algorithms
- New protocols

Future Work

- Machine learning
- Graph algorithms
- New protocols
  - SPDZ

Future Work

- Machine learning
- Graph algorithms
- New protocols
  - SPDZ
  - Honest majorities with many parties

Future Work

- Machine learning
- Graph algorithms
- New protocols
  - SPDZ
  - Honest majorities with many parties
- 2PC preprocessing

Future Work

- Machine learning
- Graph algorithms
- New protocols
  - SPDZ
  - Honest majorities with many parties
- 2PC preprocessing
  - Beaver triples over $\mathbb{Z}_{2^k}$

Future Work

- Machine learning
- Graph algorithms
- New protocols
  - SPDZ
  - Honest majorities with many parties
- 2PC preprocessing
  - Beaver triples over $\mathbb{Z}_{2^k}$
  - Oblivious shuffling

Thank You!

Polling Paper

Polling GitHub

ORQ Paper

ORQ GitHub